

UNIVERSITÄT
LEIPZIG

Fakultät für Mathematik und Informatik
Mathematisches Institut

Model-Agnostic Interpretability: A Mathematical Approach to Explainable Artificial Intelligence

Diploma thesis by
Moritz Beyer

supervised by
Dr. Iryna Okhrin

Leipzig, December 21, 2025

Ich möchte mich herzlich bei allen bedanken, die mich während der Anfertigung dieser Arbeit unterstützt haben. Mein besonderer Dank gilt meiner Betreuerin Dr. Iryna Okhrin für ihre fachliche Unterstützung, wertvollen Anregungen und ihre Geduld, die maßgeblich zum Gelingen dieser Arbeit beigetragen haben. Ich danke auch Prof. Sayan Mukherjee für inspirierende Gedanken aus seiner Vorlesung, die meine Perspektive auf das Thema geprägt haben.

Weiter möchte ich meinen Eltern danken, die mich während meines Studiums stets unterstützt und ermutigt haben. Außerdem danke ich meinen Freunden und ehemaligen Kommilitonen für den Austausch von Ideen, die moralische Unterstützung in schweren Zeiten, das Korrekturlesen und die tolle LaTeX-Vorlage.

Mein größter Dank gilt meiner Frau Charlotte, die mich während der gesamten Arbeit mit Liebe, Geduld und Zuversicht begleitet hat. Sie hat mich motiviert, mir Halt gegeben und mir Freiräume geschaffen, wenn ich sie gebraucht habe. Ohne ihre Unterstützung hätte ich diese Arbeit nicht fertigstellen können.

Als letztes möchte ich mich beim Universitätsrechenzentrum der Universität Leipzig bedanken, dessen Ressourcen ich teilweise für die Berechnungen in dieser Arbeit nutzen durfte.

Contents

1 Introduction **3**

2 Theoretical Background **4**
2.1 Mathematical Foundations 4
2.2 Taxonomy 5

3 Local Model-Agnostic Methods **7**
3.1 Ceteris Paribus and Individual Conditional Expectation 7
3.2 Shapley Values 11
3.2.1 Shapley Additive Explanations 18

4 Global Model-Agnostic Methods **22**
4.1 Partial Dependence Plots 22
4.2 Accumulated Local Effects 25
4.3 Permutation Feature Interaction 30
4.3.1 Conditional Permutation Feature Importance 31
4.4 Feature Interaction 34

5 Case Study **38**
5.1 Global Analysis 40
5.2 Local Analysis 46
5.3 Discussion and Validity Considerations 58

6 Summary **60**

References **61**

List of Appendices **66**

E Selbstständigkeitserklärung **125**

1 Introduction

Artificial intelligence is commonly understood as the broader goal of enabling machines to perform tasks that would normally require human intelligence. Machine learning is a subfield of AI in which such behavior is not programmed explicitly, but learned from data. In supervised learning one observes pairs of inputs and outputs and fits a prediction rule that generalizes to new inputs. Depending on the type of output, one distinguishes classification problems with discrete labels and regression problems with real-valued labels. This thesis focuses on supervised regression.

Modern supervised learning methods can achieve strong predictive performance, but many successful model classes are difficult to inspect directly. In practice this leads to a tension between accuracy and understanding: a model may be useful while it remains unclear which aspects of the input drive its predictions. This motivates explainable artificial intelligence (XAI). Since XAI is used in very broad ways, we use the narrower term interpretable machine learning (IML) for methods that explain a trained prediction function in terms of its input features.

Modern supervised learning methods can achieve strong predictive performance, but the resulting prediction rules can be difficult to understand. In many applications one therefore wants additional information beyond a single predicted value, for instance which input features are most influential for the model, how feature effects behave across the dataset, and why the model outputs a particular prediction for a given instance.

The main goal of this thesis is to present several explanation techniques for supervised regression models within a unified mathematical framework. The focus is not on proposing new algorithms, but on providing consistent notation and precise definitions that make it possible to state and prove key properties in a single language across different methods.

The methods developed in the theoretical part are applied in a case study on a large dataset of board games, where the task is to predict average user ratings from engineered game metadata. Several model classes are trained, and the explanation methods are used to compare learned patterns across models, to diagnose interaction effects, and to inspect individual predictions.

Chapter 2 introduces the basic supervised learning setup and fixes notation that will be used throughout the thesis, along with a taxonomy of interpretability concepts. Chapter 3 develops local model-agnostic methods, focusing on Ceteris Paribus and ICE curves and on Shapley-based feature attributions. Chapter 4 presents global model-agnostic methods for feature effects, importance and interactions, including PDP, ALE, permutation-based importance variants and an interaction measure. Chapter 5 contains the empirical case study and discusses both global and local findings together with validity considerations. Chapter 6 concludes with a summary of the main theoretical and empirical results.

2 Theoretical Background

2.1 Mathematical Foundations

In this chapter we define basic concepts and notation used throughout this thesis. In the footnotes we provide alternative notations or names that are commonly used in the literature to help readers cross-reference them.

Definition 2.1.1 (Feature Space, instance and feature)**.** The Feature Space[1] $\mathbf{X} \subset \mathbb{R}^p$ is the set of all possible instances[2] $\mathbf{x} = (x_1, \dots, x_p)$. Each entry x_i is called a feature[3].

For the theoretical part of this thesis, we will assume that all features are numerical. In applications, we often have ordinal or categorical features but there are methods to convert them into numerical features, e.g. one-hot encoding. Note that encoding higher dimensional categorical data is non-trivial and an active field of research [30, 25].

Definition 2.1.2 (Feature vector)**.** A Feature vector is the set of all values corresponding to the i-th feature across all instances, i.e.

$$\mathbf{x}_i = \bigcup_{\mathbf{x} \in \mathbf{X}} x_i$$

We sometimes only use the index $i \in \{1, \dots, p\}$ to refer to the feature vector.

Definition 2.1.3 (Label Space and label)**.** The Label Space[4] $\mathbf{Y} \subset \mathbb{R}$ is the set of all possible labels y.

Definition 2.1.4 (True Function)**.** The True Function[5] $f \colon \mathbf{X} \to \mathbf{Y}$ is the function that maps instances $\mathbf{x}^{(i)} \in \mathbf{X}$ to their corresponding labels $y^{(i)} \in \mathbf{Y}$.

Definition 2.1.5 (Dataset)**.** A dataset[6] $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$ consists of N instances $\mathbf{x}^{(i)} \in \mathbf{X}$ and their corresponding labels $f(\mathbf{x}^{(i)}) = y^{(i)} \in \mathbf{Y}$.

We will use $\mathbf{X}_\mathcal{D} = \{\mathbf{x} \in \mathbf{X} \colon \exists y \in \mathbf{Y}, (\mathbf{x}, y) \in \mathcal{D}\}$ to denote the feature space of the dataset and analogously $\mathbf{Y}_\mathcal{D} = \{y \in \mathbf{Y} \colon \exists \mathbf{x} \in \mathbf{X}, (\mathbf{x}, y) \in \mathcal{D}\}$.

The goal of machine learning tasks is to make a prediction about the label y given an instance $\mathbf{x}$. This is easy if $\mathbf{x} \in \mathbf{X}_\mathcal{D}$ since we can just look up the corresponding label in the dataset. However, the more interesting case is when we want to make predictions for unseen instances $\mathbf{x} \in \mathbf{X} \setminus \mathbf{X}_\mathcal{D}$. To accomplish this, we use models that approximate the true function f based on the information contained in the dataset $\mathcal{D}$.

[1] Data Space, Input Space, Attribute Space, Domain
[2] Observation, Sample, Example, Input vector, independent variable, explanatory variable
[3] Attribute, Variable, Predictor, Covariate, Input
[4] Output Space, Response Space, Label Space, Codomain
[5] Target Function, Objective Function, Response Function, True Distribution
[6] Data, Sample, Examples

Definition 2.1.6 (Model)**.** A Model[7] $\hat{f} : \mathbf{X} \to \mathbf{Y}$ is a function that maps instances $\mathbf{x}^{(i)} \in \mathbf{X}$ to their predicted labels $y^{(i)} \in \mathbf{Y}$.

Then we need a way to assess how well a model $\hat{f}$ approximates the true function f. This is done using a loss function that quantifies the discrepancy between the true labels and the predicted labels.

Definition 2.1.7 (Loss Function)**.** A Loss Function[8] is a mapping

$$\ell \colon \mathbf{Y} \times \mathbf{Y} \to \mathbb{R}, \qquad (y, \hat{y}) \mapsto \ell(y, \hat{y}),$$

that satisfies $\ell(y, y) = 0$ for all $y \in \mathbf{Y}$.

We usually also require ℓ to be non-negative. Common examples are the squared loss $\ell(y, \hat{y}) = (y - \hat{y})^2$, the absolute loss $\ell(y, \hat{y}) = |y - \hat{y}|$ or the $0-1-$loss $\ell(y, \hat{y}) = \begin{cases} 0 & y = \hat{y} \\ 1 & y \neq \hat{y} \end{cases}$.

There are many different algorithms to construct models from data. These algorithms will not be the focus of this thesis, but the interested reader can find a few examples in Appendix B.

2.2 Taxonomy

The terms explainable artificial intelligence (XAI) and interpretable machine learning (IML) are often used interchangeably, but they typically refer to different scopes. XAI is commonly understood as an umbrella term for approaches that make the behavior of an AI system understandable to humans; this can include predictive models as well as broader decision pipelines and human–AI interfaces. In recent years, this broader XAI literature has also expanded substantially around large language models (LLMs), both because LLMs themselves require explanation and because they can be used to generate natural-language explanations for other models [5, 8]. In contrast, IML focuses more narrowly on machine learning models in the sense of a prediction function $\hat{f} : \mathbf{X} \to \mathbf{Y}$ and on methods that relate predictions $\hat{f}(\mathbf{x})$ to the input features $\mathbf{x} = (x_1, \ldots, x_p)$. In this thesis we therefore use IML when we analyze the dependence of a model $\hat{f}$ on the feature space.

A useful first distinction is whether a model is interpretable by design or whether explanations are produced after training. In the first case, the model class is chosen so that the fitted model can be inspected directly, for example through a small number of parameters or a simple rule structure. In the second case, a potentially complex predictor is trained and additional methods are applied to obtain explanations of its behavior. We follow the overview and terminology in Molnar [28, pp. 27-36], but only summarize the aspects needed later in this thesis.

[7] Predicted Function, Learned Function, Hypothesis, Predictor, Estimator, Approximation
[8] Cost function, error function

Another common distinction is model-agnostic versus model-specific methods. Model-agnostic methods treat $\hat{f}$ as a black box and only use its input-output behavior, which makes them applicable to a wide range of predictors. Model-specific methods instead exploit the internal structure of a particular model class, such as coefficients for linear models, decision paths for trees, or gradients for neural networks; this can be efficient and insightful, but it narrows the applicability to that model class.

Finally, explanation methods are often described as local or global, depending on their scope. Local explanations target a single instance $\mathbf{x}$ and aim to explain why the model predicts $\hat{f}(\mathbf{x})$ in that case. Global explanations summarize how $\hat{f}$ behaves across many instances and highlight which features or patterns matter overall. In practice these perspectives complement each other: global analyses provide a broad picture and help select interesting cases, while local methods are suited to inspecting individual predictions and their drivers.

In the remainder of this thesis we focus on post-hoc model-agnostic methods with both global and local scope.

3 Local Model-Agnostic Methods

This section discusses model-agnostic methods for explaining individual predictions $\hat{f}(\mathbf{x})$ of a regression model. Local explanations aim to describe why a specific instance receives its predicted value and how this prediction would change under small modifications of the input. Such instance-level explanations are useful for understanding atypical predictions, checking whether a model relies on plausible signals, and diagnosing systematic failure modes.

We focus on two complementary perspectives on local explanations. First, Ceteris Paribus profiles and ICE plots analyse local sensitivity by varying one feature (or a small feature subset) on a grid while keeping all remaining features fixed to their observed values. Second, Shapley-based approaches assign additive feature attributions that decompose a prediction into a baseline plus contributions from individual features. The two viewpoints address different questions: CP/ICE describe how predictions change under targeted interventions, whereas Shapley/SHAP quantify how much each feature contributes to the prediction relative to a reference value.

A broad range of additional local explanation techniques exists, including local surrogate models such as LIME Ribeiro, Singh, and Guestrin [35] and rule-based explanations such as Anchors Ribeiro, Singh, and Guestrin [36], as well as counterfactual explanations and algorithmic recourse methods Wachter, Mittelstadt, and Russell [45] and Karimi et al. [22]. We do not pursue these approaches here because they introduce further modeling choices beyond those considered in this thesis (for example the choice of surrogate model class, distance and locality definitions, discretization and rule constraints, or feasibility and cost assumptions for counterfactuals). Instead, we restrict attention to CP/ICE and Shapley-based attributions, which can be applied to any predictive model via repeated evaluations and which admit a clear mathematical formulation that will be used in the case study.

3.1 Ceteris Paribus and Individual Conditional Expectation

Individual Conditional Expectation (ICE) plots visualize how a model's prediction changes when one (or a small set of) feature(s) is varied, separately for many individual instances. They were introduced by Goldstein et al. [17] as an extension of Partial Dependence Plots (Section 4.1) in order to make heterogeneity visible that can be hidden by the averaging in PDPs.

The basic building block is a curve for a single instance, where all features outside the investigated subset are kept fixed. Such curves are commonly referred to as ceteris paribus (CP) curves; we follow the terminology introduced in Kuźba, Baranowska, and Biecek [23].

Definition 3.1.1 (Ceteris Paribus curve)**.** Let $S \subset \{1, \ldots, p\}$ and $C = \{1, \ldots, p\} \setminus S$. Then $\mathbf{x}_S \subset \mathbf{X}_\mathcal{D}$ denotes the subset of features corresponding to the indices in S, and $\mathbf{x}_C = \mathbf{X}_\mathcal{D} \setminus \mathbf{x}_S$ denotes the remaining features. Then the Ceteris Paribus

curve for an instance $i \in \{1, \dots, N\}$ is defined as:

$$CP^{(i)}(\mathbf{x}_S) = \hat{f}(\mathbf{x}_S, \mathbf{x}_C^{(i)}) \tag{1}$$

where $\mathbf{x}_C^{(i)}$ denotes the i-th observation of the complement features $\mathbf{x}_C$.

A CP profile is a local explanation in the sense that it is tied to a specific instance i: it describes how the prediction would change for that same instance if only the feature(s) in S were modified, while everything else remains as observed. In practice, one usually considers $|S| = 1$ and evaluates the function on a grid of values for the selected feature; the resulting graph is then a one-dimensional curve.

ICE combines many such instance-wise CP curves into a single visualization. This makes ICE a useful middle-ground between local and global explanations: it is not purely local (since it displays multiple instances at once), but it is also not a fully aggregated global summary like PDP or ALE, because the individual curves remain visible. In particular, ICE plots reveal whether feature effects vary substantially across instances (interaction effects or subpopulation behavior), which would be averaged away in global effect curves.

Definition 3.1.2 (Individual Conditional Expectation)**.** The Individual Conditional Expectation (ICE) for the feature subset S is the collection of CP curves across all instances,

$$ICE(\mathbf{x}_S) \;=\; \begin{bmatrix} CP^{(1)}(\mathbf{x}_S) \\ \vdots \\ CP^{(N)}(\mathbf{x}_S) \end{bmatrix}. \tag{2}$$

The ICE plot for S is the set of graphs $\{G(CP^{(i)}(\mathbf{x}_S))\}_{i=1}^{N}$.

In the following example, we will demonstrate how CP and ICE can be used to gain insights into feature interactions learned by a model.

Example 3.1.3. Let $x_1, x_2 \sim \mathcal{N}(0,1)$ i.i.d. and $\varepsilon \sim \mathcal{N}(0, \sigma^2)$. Define a label by

$$y = f(x_1, x_2) = x_1 + 2x_1x_2 + \varepsilon.$$

In this setup the marginal effect of x_1 depends on x_2, since

$$\frac{\partial}{\partial x_1} \mathbb{E}[y \mid x_1, x_2] = 1 + 2x_2.$$

Hence different observations (with different values of x_2) have different local slopes in x_1.

We assume that our model $\hat{f}$ has perfectly learned the true function f (i.e. $\hat{f} = x_1 + 2x_1x_2$) and illustrate how ICE plots can reveal the interaction structure. To visualise the local dependence of $\hat{f}$ on x_1, we compute a Ceteris Paribus (CP) curve for each of a subset of instances: for a grid $\mathcal{G} \subset \mathbb{R}$ we evaluate

$$g^{(i)}(t) \coloneqq \hat{f}\big(t, x_2^{(i)}\big), \qquad t \in \mathcal{G},$$

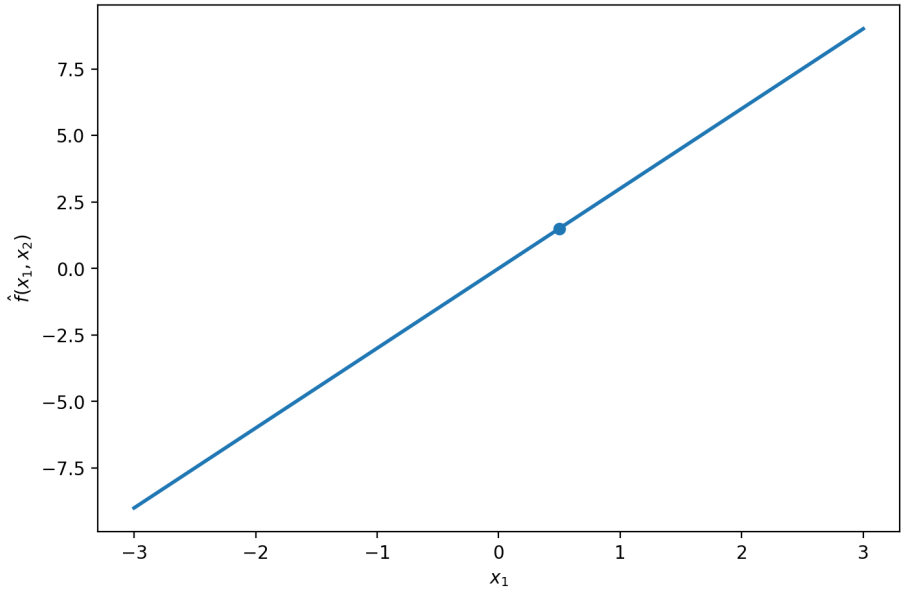


Figure 1: Ceteris Paribus (CP) plot for a single instance with $(x_1, x_2) = (0.5, 1)$ in the same setup as Figure 2. The marker indicates the instance's original location on the curve.

i.e. we vary x_1 along the grid and keep x_2 fixed at its observed value $x_2^{(i)}$. An ICE plot is then obtained by plotting the collection of CP curves $\{g^{(i)}\}_{i \in I}$ for a set of sampled indices I.

Figure 1 displays a single CP curve for one instance, illustrating the local effect of x_1 when x_2 is held fixed at that instance's value. While a single CP curve is informative for an individual prediction, the ICE plot reveals the variability of this effect across instances and therefore makes interactions visible at a glance.

Figure 2 shows the resulting ICE plot for x_1 based on $|I| = 200$ randomly sampled instances. The curves display a pronounced heterogeneity in slope: instances with $x_2 > 0$ tend to exhibit a steeper positive dependence on x_1, whereas instances with $x_2 \leq 0$ can have substantially flatter or even negative slopes. This is a direct consequence of the interaction term $2x_1x_2$. The thick curves highlight the mean ICE over all sampled instances, and separately for the subgroups $x_2 \leq 0$ and $x_2 > 0$.

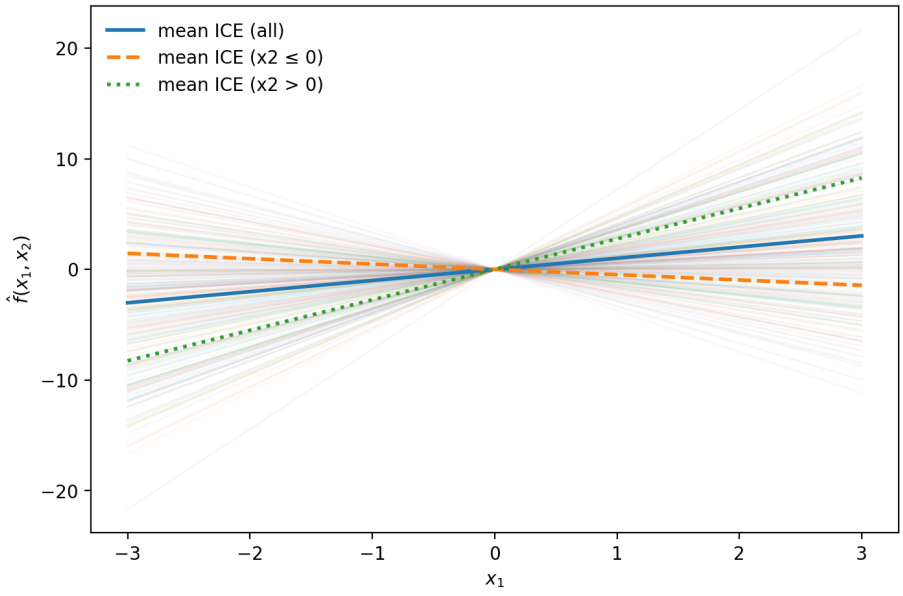


Figure 2: ICE plot for x_1 in the interaction model $f(x_1, x_2) = x_1 + 2x_1x_2$. Each thin curve is a CP curve for one sampled instance (varying x_1 while holding x_2 fixed). The thick solid curve is the mean ICE over all instances; dashed/dotted curves show mean ICE for the subgroups $x_2 \leq 0$ and $x_2 > 0$, illustrating effect heterogeneity induced by the interaction.

3.2 Shapley Values

Many local interpretability methods aim to explain a single prediction $\hat{f}(\mathbf{x})$ by attributing the deviation of this prediction from a baseline value to the individual features of $\mathbf{x}$. A principled way to construct such attributions is given by the Shapley value from cooperative game theory, introduced by Shapley [39] in 1953. In this section we present the mathematical definition and main properties of Shapley values. In the subsequent SHAP subsection, we will define a suitable cooperative game induced by a prediction model $\hat{f}$ and an instance $\mathbf{x}$, following Lundberg and Lee [27].

Definition 3.2.1 (Cooperative game)**.** Let $F \coloneqq \{1, \dots, p\}$ be a finite set of players. A (transferable utility) cooperative game on F is a set function

$$v \colon 2^F \to \mathbb{R}$$

with $v(\emptyset) = 0$. For $S \subseteq F$, the value $v(S)$ is interpreted as the payoff that coalition S can obtain by cooperating.

The Shapley value is a rule that assigns to each game v a payoff allocation $(\phi_1(v), \dots, \phi_p(v)) \in \mathbb{R}^p$. It is characterized by axioms that formalize fairness and coherence requirements. The notation used in the next definition follows Molnar [28, pp. 172-175].

Definition 3.2.2 (Value and Shapley axioms)**.** A value is a mapping Φ that assigns to every game v on F a vector

$$\Phi(v) = (\phi_1(v), \dots, \phi_p(v)) \in \mathbb{R}^p$$

and that satisfies the following axioms:

(A1) (Efficiency)

$$\sum_{i=1}^{p} \phi_i(v) = v(F).$$

(A2) (Symmetry) If $i, j \in F$ satisfy

$$v(S \cup \{i\}) = v(S \cup \{j\}) \qquad \text{for all } S \subseteq F \setminus \{i, j\},$$

then $\phi_i(v) = \phi_j(v)$.

(A3) (Dummy player) If $i \in F$ satisfies

$$v(S \cup \{i\}) = v(S) \qquad \text{for all } S \subseteq F \setminus \{i\},$$

then $\phi_i(v) = 0$.

(A4) (Additivity) For games v, w on F define $(v + w)(S) \coloneqq v(S) + w(S)$. Then

$$\Phi(v + w) = \Phi(v) + \Phi(w).$$

Shapley [39] shows that you can decompose any game v into a linear combination of simpler games called unanimity games.

Definition 3.2.3 (Unanimity game)**.** For $\emptyset \neq T \subseteq F$ the unanimity game u_T is defined by

$$u_T(S) \coloneqq \begin{cases} 1, & T \subseteq S, \\ 0, & \text{otherwise.} \end{cases}$$

Lemma 3.2.4 (Unanimity decomposition)**.** For every game v on F there exist unique coefficients $(a_T)_{\emptyset \neq T \subseteq F}$ such that

$$v \;=\; \sum_{\emptyset \neq T \subseteq F} a_T \, u_T. \tag{3}$$

They are given by the inclusion–exclusion formula

$$a_T \;=\; \sum_{S \subseteq T} (-1)^{|T|-|S|} \, v(S), \qquad \emptyset \neq T \subseteq F. \tag{4}$$

Proof. Fix $\emptyset \neq T \subseteq F$ and define a_T by (4). For any coalition $S \subseteq F$ we evaluate the right-hand side of (3):

$$\sum_{\emptyset \neq T \subseteq F} a_T u_T(S) = \sum_{\emptyset \neq T \subseteq S} a_T,$$

since $u_T(S) = 1 \Leftrightarrow T \subseteq S$. Using (4) and swapping the order of summation gives

$$\begin{aligned} \sum_{\emptyset \neq T \subseteq S} a_T &= \sum_{\emptyset \neq T \subseteq S} \sum_{R \subseteq T} (-1)^{|T|-|R|} v(R) \\ &= \sum_{R \subseteq S} v(R) \sum_{\substack{T \subseteq S \\ R \subseteq T \\ T \neq \emptyset}} (-1)^{|T|-|R|}. \end{aligned}$$

Fix $R \subseteq S$. Write $T = R \cup U$ with $U \subseteq S \setminus R$. Then, using the binomial theorem, the inner sum becomes

$$\sum_{U \subseteq S \setminus R} (-1)^{|U|} = \sum_{k=0}^{|S \setminus R|} \binom{|S \setminus R|}{k} (-1)^k = (1-1)^{|S \setminus R|} = \begin{cases} 1, & R = S, \\ 0, & R \neq S. \end{cases}$$

Hence only the term $R = S$ remains and we obtain

$$\sum_{\emptyset \neq T \subseteq S} a_T = v(S).$$

This shows that Equation (3) holds. Uniqueness follows because the coefficients are determined uniquely by Equation (4). □

We next compute the value of unanimity games under axioms (A1)-(A3).

Lemma 3.2.5. Let Φ satisfy (A1)-(A3). Then for every $\emptyset \neq T \subseteq F$ and every $i \in F$,

$$\phi_i(u_T) = \begin{cases} \frac{1}{|T|}, & i \in T, \\ 0, & i \notin T. \end{cases}$$

Proof. If $i \notin T$ then $u_T(S \cup \{i\}) = u_T(S)$ for all $S \subseteq F \setminus \{i\}$, so i is a dummy player for u_T and (A3) yields $\phi_i(u_T) = 0$.

Now let $i \in T$. For any $i, j \in T$ and any $S \subseteq F \setminus \{i, j\}$ we have

$$\begin{aligned} u_T(S \cup \{i\}) = 1 &\iff T \subseteq S \cup \{i\} \\ &\iff T \setminus \{i\} \subseteq S \\ &\iff T \subseteq S \cup \{j\} \\ &\iff u_T(S \cup \{j\}) = 1, \end{aligned}$$

and similarly $u_T(S \cup \{i\}) = 0 \iff u_T(S \cup \{j\}) = 0$. Hence (A2) implies

$$\phi_i(u_T) = \phi_j(u_T) \quad \text{for all } i, j \in T. \tag{5}$$

Because $\phi_i(u_T) = 0$ for $i \notin T$,

$$\sum_{i \in T} \phi_i(u_T) = \sum_{i \in F} \phi_i(u_T) \overset{(A1)}{=} u_T(F) = 1.$$

Then, because of Equation (5) each $\phi_i(u_T)$ must be $1/|T|$. □

Now, we will state two further lemmas that will help us to prove the Shapley value theorem.

Lemma 3.2.6. Let v be a cooperative game on $F = \{1, \ldots, p\}$ and fix $i \in F$. Let Π be a uniformly random permutation of F and define

$$P_\Pi(i) \coloneqq \{j \in F : \Pi(j) < \Pi(i)\}.$$

Then

$$\mathbb{E}_\Pi\big[v(P_\Pi(i) \cup \{i\}) - v(P_\Pi(i))\big] = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\,(p - |S| - 1)!}{p!} \big(v(S \cup \{i\}) - v(S)\big). \tag{6}$$

Proof. Fix $i \in F$. For any $S \subseteq F \setminus \{i\}$ we compute the probability that $P_\Pi(i) = S$. This event means that the elements of S appear before i, and the elements of $F \setminus (S \cup \{i\})$ appear after i. Among the $p!$ permutations, the number of such permutations is $|S|!\,(p - |S| - 1)!$, where $|S|!$ counts the arrangements of the elements of S before i and $(p - |S| - 1)!$ counts the arrangements of the remaining elements after i. Hence

$$\mathbb{P}\big(P_\Pi(i) = S\big) = \frac{|S|!\,(p - |S| - 1)!}{p!}.$$

Therefore,

$$\begin{aligned}\mathbb{E}_\Pi\big[v(P_\Pi(i)\cup\{i\})-v(P_\Pi(i))\big] &= \sum_{S\subseteq F\setminus\{i\}} \mathbb{P}(P_\Pi(i)=S)\,\big(v(S\cup\{i\})-v(S)\big)\\ &= \sum_{S\subseteq F\setminus\{i\}} \frac{|S|!\,(p-|S|-1)!}{p!}\,\big(v(S\cup\{i\})-v(S)\big).\end{aligned}$$

□

Lemma 3.2.7. Let $F=\{1,\dots,p\}$ and fix $i\in F$. For $R\subseteq F$ define

$$c_i(R) \coloneqq \sum_{\substack{\emptyset\neq T\subseteq F\\ i\in T\\ R\subseteq T}} \frac{(-1)^{|T|-|R|}}{|T|}.$$

Then $c_i(R)$ evaluates to

$$c_i(R)=\begin{cases}\dfrac{(|R|-1)!\,(p-|R|)!}{p!}, & i\in R,\\[2mm] -\dfrac{|R|!\,(p-|R|-1)!}{p!}, & i\notin R.\end{cases}$$

Proof. We first prove the auxiliary identity

$$\sum_{k=0}^{n}\binom{n}{k}\frac{(-1)^k}{m+k}=\frac{(m-1)!\,n!}{(m+n)!},\qquad m\in\mathbb{N},\ n\in\mathbb{N}_0. \tag{7}$$

Using $\frac{1}{m+k}=\int_0^1 x^{m+k-1}\,dx$ and then the binomial theorem, we obtain

$$\begin{aligned}\sum_{k=0}^{n}\binom{n}{k}\frac{(-1)^k}{m+k} &= \int_0^1 x^{m-1}\sum_{k=0}^{n}\binom{n}{k}(-1)^k x^k\,dx\\ &= \int_0^1 x^{m-1}(1-x)^n\,dx,\end{aligned}$$

Set $I_{m,n}\coloneqq\int_0^1 x^{m-1}(1-x)^n\,dx$. Integration by parts with $u=(1-x)^n$ and $dv=x^{m-1}dx$ yields, for $n\geq 1$,

$$I_{m,n}=\frac{n}{m}\int_0^1 x^m(1-x)^{n-1}\,dx=\frac{n}{m}\,I_{m+1,n-1}.$$

Iterating this recursion and using $I_{m,0}=\int_0^1 x^{m-1}dx=\frac{1}{m}$ gives

$$I_{m,n}=\frac{n}{m}\cdot\frac{n-1}{m+1}\cdots\frac{1}{m+n-1}\cdot\frac{1}{m+n}=\frac{n!}{m(m+1)\cdots(m+n)}=\frac{(m-1)!\,n!}{(m+n)!}.$$

This proves (7).

We now evaluate $c_i(R)$. If $i \in R$, then the condition $i \in T$ is redundant and

$$c_i(R) = \sum_{\substack{\emptyset \neq T \subseteq F \\ R \subseteq T}} \frac{(-1)^{|T|-|R|}}{|T|}.$$

Write $m \coloneqq |R|$ and $n \coloneqq p - m$. Every $T \supseteq R$ can be written uniquely as $T = R \cup U$ with $U \subseteq F \setminus R$. Hence $|T| = m + |U|$ and $|T| - |R| = |U|$, and grouping by $k \coloneqq |U|$ yields

$$c_i(R) = \sum_{k=0}^{n} \binom{n}{k} \frac{(-1)^k}{m+k}.$$

By (7) this equals

$$\frac{(m-1)!\, n!}{(m+n)!} = \frac{(|R|-1)!\,(p-|R|)!}{p!}.$$

If $i \notin R$, then $R \subseteq T$ and $i \in T$ is equivalent to $R \cup \{i\} \subseteq T$, hence

$$c_i(R) = \sum_{\substack{\emptyset \neq T \subseteq F \\ R \cup \{i\} \subseteq T}} \frac{(-1)^{|T|-|R|}}{|T|}.$$

Write $m \coloneqq |R|$ and $n \coloneqq p-m-1$. Every $T \supseteq R \cup \{i\}$ can be written uniquely as $T = R \cup \{i\} \cup U$ with $U \subseteq F \setminus (R \cup \{i\})$, so $|T| = m+1+|U|$ and $|T|-|R| = 1+|U|$. Grouping by $k \coloneqq |U|$ yields

$$c_i(R) = -\sum_{k=0}^{n} \binom{n}{k} \frac{(-1)^k}{m+1+k}.$$

Applying (7) with m replaced by $m+1$ gives

$$c_i(R) = -\frac{m!\, n!}{(m+1+n)!} = -\frac{|R|!\,(p-|R|-1)!}{p!}.$$

□

We can now state and prove the Shapley Value theorem.

Theorem 3.2.8 (Shapley value)**.** For every cooperative game v there exists a unique value Φ satisfying (A1)-(A4). We call Φ the Shapley value and it is given componentwise by

$$\phi_i(v) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\,(p-|S|-1)!}{p!} \big(v(S \cup \{i\}) - v(S)\big), \qquad i \in F. \tag{8}$$

Proof. First, we will prove existence by showing that defining Φ as in (8) satisfies (A1)-(A4). Additivity (A4) follows immediately from linearity in v.

If i is a dummy player, then $v(S \cup \{i\}) - v(S) = 0$ for all $S \subseteq F \setminus \{i\}$, so $\phi_i(v) = 0$ and (A3) holds.

Symmetry (A2) holds since the weights in (8) depend only on $|S|$ and the hypothesis $v(S \cup \{i\}) = v(S \cup \{j\})$ for all $S \subseteq F \setminus \{i, j\}$ implies that the terms in the sums for $\phi_i(v)$ and $\phi_j(v)$ can be matched by relabeling i and j.

To prove efficiency (A1), we invoke Lemma 3.2.6, which yields the representation

$$\phi_i(v) = \mathbb{E}_\Pi\big[v(P_\Pi(i) \cup \{i\}) - v(P_\Pi(i))\big].$$

Summing over $i \in F$ and using linearity of expectation gives

$$\sum_{i \in F} \phi_i(v) = \mathbb{E}_\Pi \left[\sum_{i \in F} \big(v(P_\Pi(i) \cup \{i\}) - v(P_\Pi(i))\big) \right].$$

Fix a permutation Π and write its ordering as $(j_1, \ldots, j_p)$. Then $P_\Pi(j_k) = \{j_1, \ldots, j_{k-1}\}$ for $k = 1, \ldots, p$, and hence the inner sum telescopes:

$$\begin{aligned} \sum_{i \in F} \big(v(P_\Pi(i) \cup \{i\}) - v(P_\Pi(i))\big) &= \sum_{k=1}^{p} \big(v(\{j_1, \ldots, j_k\}) - v(\{j_1, \ldots, j_{k-1}\})\big) \\ &= v(F) - v(\emptyset) = v(F), \end{aligned}$$

since $v(\emptyset) = 0$. Taking expectations over Π yields $\sum_{i \in F} \phi_i(v) = v(F)$, which is (A1).

Next, we prove uniqueness. Let Φ satisfy (A1)-(A4). By Lemma 3.2.4 we can write

$$v = \sum_{\emptyset \neq T \subseteq F} a_T \, u_T.$$

By additivity (A4),

$$\phi_i(v) = \sum_{\emptyset \neq T \subseteq F} a_T \, \phi_i(u_T).$$

By Lemma 3.2.5 we have $\phi_i(u_T) = 0$ if $i \notin T$ and $\phi_i(u_T) = \frac{1}{|T|}$ if $i \in T$, hence

$$\phi_i(v) = \sum_{\substack{\emptyset \neq T \subseteq F \\ i \in T}} \frac{a_T}{|T|}. \tag{9}$$

Insert (4) into (9):

$$\phi_i(v) = \sum_{\substack{\emptyset \neq T \subseteq F \\ i \in T}} \frac{1}{|T|} \sum_{R \subseteq T} (-1)^{|T| - |R|} v(R).$$

We regroup the finite double sum by the subsets $R \subseteq F$, i.e. we collect all terms multiplying the same $v(R)$ and write

$$\phi_i(v) = \sum_{R \subseteq F} c_i(R) \, v(R), \tag{10}$$

where

$$c_i(R) \coloneqq \sum_{\substack{\emptyset \neq T \subseteq F \\ i \in T \\ R \subseteq T}} \frac{(-1)^{|T|-|R|}}{|T|}. \tag{11}$$

By Lemma 3.2.7 this yields

$$\phi_i(v) = \sum_{\substack{R \subseteq F \\ i \in R}} \frac{(|R|-1)!\,(p-|R|)!}{p!}\, v(R) \;-\; \sum_{\substack{R \subseteq F \\ i \notin R}} \frac{|R|!\,(p-|R|-1)!}{p!}\, v(R).$$

In the first sum substitute $R = S \cup \{i\}$ with $S \subseteq F \setminus \{i\}$. Then $|R| - 1 = |S|$ and $p - |R| = p - |S| - 1$, and we obtain

$$\phi_i(v) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\,(p-|S|-1)!}{p!}\, v(S \cup \{i\}) \;-\; \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\,(p-|S|-1)!}{p!}\, v(S),$$

which is (8). In particular, any value satisfying (A1)-(A4) must coincide with (8), and therefore is unique. □

Example 3.2.9 (Carpenter and architect)**.** Two people consider whether to build a small wooden house. Player 1 is a carpenter and can build a basic version alone, worth 2 (in suitable units). Player 2 is an architect and cannot build the house alone, so alone they create value 0. If they cooperate, the architect improves the design, increasing the value to 3. This is modeled by the game $v\colon 2^{\{1,2\}} \to \mathbb{R}$ given by

$$v(\emptyset) = 0, \qquad v(\{1\}) = 2, \qquad v(\{2\}) = 0, \qquad v(\{1,2\}) = 3.$$

Using the Shapley value formula (8), we obtain for player 1

$$\phi_1(v) = \sum_{S \subseteq F \setminus \{1\}} \frac{|S|!\,(2-|S|-1)!}{2!} \big(v(S \cup \{1\}) - v(S)\big).$$

Since $F \setminus \{1\} = \{2\}$, the subsets are $S = \emptyset$ and $S = \{2\}$, and the weights are

$$\frac{0!\,1!}{2!} = \frac{1}{2}, \qquad \frac{1!\,0!}{2!} = \frac{1}{2}.$$

Hence

$$\phi_1(v) = \frac{1}{2}\big(v(\{1\}) - v(\emptyset)\big) + \frac{1}{2}\big(v(\{1,2\}) - v(\{2\})\big) = \frac{1}{2}(2-0) + \frac{1}{2}(3-0) = \frac{5}{2}.$$

Similarly, for player 2 we have $F \setminus \{2\} = \{1\}$, and therefore

$$\phi_2(v) = \frac{1}{2}\big(v(\{2\}) - v(\emptyset)\big) + \frac{1}{2}\big(v(\{1,2\}) - v(\{1\})\big) = \frac{1}{2}(0-0) + \frac{1}{2}(3-2) = \frac{1}{2}.$$

$\phi_1(v)$ and $\phi_2(v)$ can be understood as fair payoffs for the carpenter and the architect if they cooperate: the carpenter receives 2.5 and the architect 0.5.

The Shapley value is linear in v by construction and distributes the total value $v(F)$ among players by averaging marginal contributions across all coalitions (or, as per lemma 3.2.6, across all player orderings). For local feature attributions, we will in the next section define a game $v_{\hat{f},\mathbf{x}}$ whose coalitional values represent model outputs when only a subset of the features of $\mathbf{x}$ is revealed, and then use $\phi_i(v_{\hat{f},\mathbf{x}})$ as the attribution of feature i for the prediction at $\mathbf{x}$.

3.2.1 Shapley Additive Explanations

Let $F \coloneqq \{1, \dots, p\}$ denote the index set of features. Shapley values require evaluating $v(S)$ for all $S \subseteq F$ and are therefore infeasible when p is large. Štrumbelj and Kononenko [44] proposed to estimate Shapley values for explaining individual predictions by Monte Carlo sampling, based on the permutation form from Lemma 3.2.6. The key practical difficulty is that, for prediction models, coalition values must represent the model output when only a subset of features is available. Since typical models $\hat{f}$ cannot be evaluated on inputs with missing entries, Štrumbelj and Kononenko [44] replace the missing features by values sampled from a background distribution.

For $S \subseteq F$ we write $-S \coloneqq F \setminus S$. If $a, b \in \mathbf{X}$, then $(a_S, b_{-S}) \in \mathbf{X}$ denotes the instance whose j-th component equals a_j for $j \in S$ and b_j for $j \in -S$.

Definition 3.2.10 (Coalition evaluation by feature replacement)**.** Let $\mathbf{x} \in \mathbf{X}$ be the instance to be explained and let Z be an $\mathbf{X}$-valued random variable. For $S \subseteq F$ define

$$m_{\hat{f},\mathbf{x}}(S) \coloneqq \mathbb{E}\big[\hat{f}(\mathbf{x}_S, Z_{-S})\big]. \tag{12}$$

If Z is distributed according to the empirical distribution of $\mathbf{X}_{\mathcal{D}}$, then $m_{\hat{f},\mathbf{x}}(S)$ is the expected prediction when the features in S are fixed to the values of $\mathbf{x}$ and the remaining features are replaced by a randomly drawn observation. This yields a cooperative game by centering at the empty coalition.

Definition 3.2.11 (Prediction game)**.** Let $\mathbf{x} \in \mathbf{X}$ and let $m_{\hat{f},\mathbf{x}}$ be given by (12). Define $v_{\hat{f},\mathbf{x}} \colon 2^F \to \mathbb{R}$ by

$$v_{\hat{f},\mathbf{x}}(S) \coloneqq m_{\hat{f},\mathbf{x}}(S) - m_{\hat{f},\mathbf{x}}(\emptyset),$$

with $v_{\hat{f},\mathbf{x}}(\emptyset) = 0$.

Štrumbelj and Kononenko [44] estimate $\phi_i(v_{\hat{f},\mathbf{x}})$ by sampling permutations and marginal contributions along these permutations.

Definition 3.2.12 (Shapley sampling estimator)**.** Fix $i \in F$. Let $\Pi^{(1)}, \dots, \Pi^{(M)}$ be i.i.d. uniformly random permutations of F and $Z^{(1)}, \dots, Z^{(M)}$ i.i.d. copies of Z. For each t let $P_{\Pi^{(t)}}(i)$ be defined as in Lemma 3.2.6. Define

$$\widehat{\phi}_i(\mathbf{x}) \coloneqq \frac{1}{M} \sum_{t=1}^{M} \Big(\hat{f}(\mathbf{x}_{P_{\Pi^{(t)}}(i) \cup \{i\}}, Z^{(t)}_{-(P_{\Pi^{(t)}}(i) \cup \{i\})}) - \hat{f}(\mathbf{x}_{P_{\Pi^{(t)}}(i)}, Z^{(t)}_{-P_{\Pi^{(t)}}(i)}) \Big). \tag{13}$$

The estimator (13) follows directly from the permutation form of Shapley values and is model-agnostic, but its definition depends on a choice of coalition evaluation scheme and requires many model evaluations to achieve small variance. This motivates methods that enable more efficient computation. The SHAP framework of Lundberg and Lee [27] can achieve this by making two additional conceptual steps. First, it formulates explanations as an additive model in a simplified binary space indicating feature presence. Second, it identifies Shapley values as the unique feature attributions within this class that satisfy the natural properties of local accuracy, missingness, and consistency. We now introduce these concepts more formally.

Definition 3.2.13 (Additive explanation model)**.** Let $\mathbf{x} \in \mathbf{X}$ be fixed. An additive explanation model is a function $g\colon \{0,1\}^p \to \mathbb{R}$ of the form

$$g(\mathbf{z}) = \phi_0 + \sum_{i=1}^{p} \phi_i z_i, \tag{14}$$

where $\mathbf{z} = (z_1, \ldots, z_p) \in \{0,1\}^p$ encodes whether a feature is present ($z_i = 1$) or absent ($z_i = 0$).

Definition 3.2.14 (Local SHAP properties)**.** Let $f_{\hat{f},\mathbf{x}}\colon \{0,1\}^p \to \mathbb{R}$ be a target function in the simplified space. An additive explanation model g as in (14) satisfies:

(P1) (Local accuracy) $g(\mathbf{1}) = f_{\hat{f},\mathbf{x}}(\mathbf{1})$.

(P2) (Missingness) If $z_i = 0$, then the attribution of feature i is 0, i.e. $\phi_i = 0$ whenever the i-th simplified input is absent.

(P3) (Consistency) If for two target functions $f_{\hat{f},\mathbf{x}}$ and $f'_{\hat{f},\mathbf{x}}$ we have

$$f'_{\hat{f},\mathbf{x}}(\mathbf{z}) - f'_{\hat{f},\mathbf{x}}(\mathbf{z}^{\setminus i}) \geq f_{\hat{f},\mathbf{x}}(\mathbf{z}) - f_{\hat{f},\mathbf{x}}(\mathbf{z}^{\setminus i}) \qquad \text{for all } \mathbf{z} \in \{0,1\}^p,$$

where $\mathbf{z}^{\setminus i}$ denotes $\mathbf{z}$ with the i-th component set to 0, then $\phi_i(f'_{\hat{f},\mathbf{x}}) \geq \phi_i(f_{\hat{f},\mathbf{x}})$.

In SHAP, the target function $f_{\hat{f},\mathbf{x}}$ is defined by a conditional expectation that replaces absent features by their distribution given the present ones. In the special case where feature dependence is ignored, this reduces to Definition 3.2.10.

Definition 3.2.15 (SHAP coalition values)**.** Let X be an $\mathbf{X}$-valued random variable. For $S \subseteq F$ define

$$m^{\mathrm{cond}}_{\hat{f},\mathbf{x}}(S) \coloneqq \mathbb{E}\big[\hat{f}(X) \mid X_S = \mathbf{x}_S\big], \qquad v^{\mathrm{cond}}_{\hat{f},\mathbf{x}}(S) \coloneqq m^{\mathrm{cond}}_{\hat{f},\mathbf{x}}(S) - m^{\mathrm{cond}}_{\hat{f},\mathbf{x}}(\emptyset). \tag{15}$$

Definition 3.2.16 (SHAP values)**.** Let $\mathbf{x} \in \mathbf{X}$. The SHAP values of $\hat{f}$ at $\mathbf{x}$ are the Shapley values of the game $v^{\text{cond}}_{\hat{f},\mathbf{x}}$ from (15). We write

$$\phi_i(\hat{f}, \mathbf{x}) \coloneqq \phi_i\big(v^{\text{cond}}_{\hat{f},\mathbf{x}}\big), \qquad i \in F,$$

and set the baseline $\phi_0(\hat{f}, \mathbf{x}) \coloneqq m^{\text{cond}}_{\hat{f},\mathbf{x}}(\emptyset) = \mathbb{E}[\hat{f}(X)]$.

Since $v^{\text{cond}}_{\hat{f},\mathbf{x}}(\emptyset) = 0$, Theorem 3.2.8 yields

$$\sum_{i=1}^{p} \phi_i(\hat{f}, \mathbf{x}) = v^{\text{cond}}_{\hat{f},\mathbf{x}}(F).$$

Moreover, $m^{\text{cond}}_{\hat{f},\mathbf{x}}(F) = \mathbb{E}[\hat{f}(X) \mid X = \mathbf{x}] = \hat{f}(\mathbf{x})$ and $m^{\text{cond}}_{\hat{f},\mathbf{x}}(\emptyset) = \mathbb{E}[\hat{f}(X)]$, so $v^{\text{cond}}_{\hat{f},\mathbf{x}}(F) = \hat{f}(\mathbf{x}) - \mathbb{E}[\hat{f}(X)]$. With $\phi_0(\hat{f}, \mathbf{x}) \coloneqq \mathbb{E}[\hat{f}(X)]$ we obtain

$$\hat{f}(\mathbf{x}) = \phi_0(\hat{f}, \mathbf{x}) + \sum_{i=1}^{p} \phi_i(\hat{f}, \mathbf{x}). \tag{16}$$

Moreover, Lundberg and Lee [27] show that within the class of additive explanation models, the values $\phi_i(\hat{f}, \mathbf{x})$ are uniquely characterized by Properties (P1)-(P3).

Finally, SHAP proposes estimators that compute $\phi_i(\hat{f}, \mathbf{x})$ efficiently. Besides the permutation estimator (13), the model-agnostic KernelSHAP method fits an additive explanation model by a weighted least squares problem.

Definition 3.2.17 (KernelSHAP objective)**.** Let $\mathbf{x} \in \mathbf{X}$ and let $f_{\hat{f},\mathbf{x}} \colon \{0,1\}^p \to \mathbb{R}$ be defined by

$$f_{\hat{f},\mathbf{x}}(\mathbf{1}_S) \coloneqq m^{\text{cond}}_{\hat{f},\mathbf{x}}(S), \qquad S \subseteq F,$$

where $\mathbf{1}_S \in \{0,1\}^p$ denotes the indicator vector of S. Let $\mathcal{Z} \subseteq \{0,1\}^p$ be a finite set of masks and let $\pi \colon \{0,1\}^p \to (0, \infty]$ be a weight function. KernelSHAP computes $(\phi_0, \phi_1, \ldots, \phi_p)$ by minimizing

$$\sum_{\mathbf{z} \in \mathcal{Z}} \pi(\mathbf{z}) \left(f_{\hat{f},\mathbf{x}}(\mathbf{z}) - g(\mathbf{z})\right)^2 \tag{17}$$

over additive models g of the form $g(\mathbf{z}) = \phi_0 + \sum_{i=1}^{p} \phi_i z_i$. The minimizer $(\phi_0, \phi_1, \ldots, \phi_p)$ defines the KernelSHAP attribution, where ϕ_i is the coefficient of z_i in the fitted additive model g.

Definition 3.2.18 (Shapley kernel)**.** Let $\mathbf{z} \in \{0,1\}^p$ and write $|\mathbf{z}| \coloneqq \sum_{i=1}^{p} z_i$. The Shapley kernel is the weight function π defined by

$$\pi(\mathbf{z}) \coloneqq \begin{cases} \infty, & |\mathbf{z}| \in \{0, p\}, \\ \dfrac{p-1}{\binom{p}{|\mathbf{z}|}\, |\mathbf{z}|\, (p - |\mathbf{z}|)}, & |\mathbf{z}| \in \{1, \ldots, p-1\}. \end{cases}$$

With this kernel Lundberg and Lee [27] show that, if $\mathcal{Z} = \{0,1\}^p$ and $f_{\hat{f},\mathbf{x}}(\mathbf{z})$ is evaluated exactly for every $\mathbf{z}$, the minimizer of (17) coincides with the SHAP values from Definition 3.2.16. The infinite weights for $|\mathbf{z}| \in \{0, p\}$ enforce the constraints $g(\mathbf{0}) = f_{\hat{f},\mathbf{x}}(\mathbf{0})$ and $g(\mathbf{1}) = f_{\hat{f},\mathbf{x}}(\mathbf{1})$.

In practice, $\mathcal{Z}$ is obtained by sampling masks $\mathbf{z}$ (equivalently, subsets $S = \{i \in F : z_i = 1\}$), typically with preference for small and large coalition sizes because of the kernel weights. For each sampled mask, the value $f_{\hat{f},\mathbf{x}}(\mathbf{z})$ is approximated from data by estimating $m^{\text{cond}}_{\hat{f},\mathbf{x}}(S)$, for example via conditional sampling if feasible, or by sampling the missing features from a background dataset. The weighted least squares problem (17) is then solved on these sampled pairs $(\mathbf{z}, f_{\hat{f},\mathbf{x}}(\mathbf{z}))$ to obtain $(\phi_0, \phi_1, \ldots, \phi_p)$.

Note that Lundberg and Lee [27] also propose specialized algorithms for linear models and deep neural networks and later Lundberg, Erion, and Lee [26] introduced an algorithm for tree-based models. However, since these are not model-agnostic, we do not cover them here.

4 Global Model-Agnostic Methods

...

4.1 Partial Dependence Plots

Partial Dependence Plots (PDPs) are widely used in XAI to visualize the marginal effect of a subset of input features on the predicted outcome of a model. They provide insight into how features interact with a model and can help detect complex relationships, such as non-linear or non-monotonic effects. PDPs have been introduced in Friedman [15] but some of the notation here is taken from Molnar [28, pp. 198-205].

Definition 4.1.1 (Partial Dependence Function)**.** Let $S \subset \{1, \ldots, p\}$ and $C = \{1, \ldots, p\} \setminus S$. Then $\mathbf{x}_S \subset \mathbf{X}_\mathcal{D}$ denotes the subset of features corresponding to the indices in S, and $\mathbf{x}_C = \mathbf{X}_\mathcal{D} \setminus \mathbf{x}_S$ denotes the remaining features. The partial dependence function for $\mathbf{x}_S$ is defined as:

$$PD_S(\mathbf{x}_S) = \mathbb{E}_{\mathbf{x}_C}[\hat{f}(\mathbf{x}_S, \mathbf{x}_C)] = \int \hat{f}(\mathbf{x}_S, \mathbf{x}_C) d\mathbb{P}\mathbf{x}_C \tag{18}$$

The partial dependence function captures the average effect of the features in $\mathbf{x}_S$ on the model $\hat{f}(\cdot)$ while marginalizing over the complementary features $\mathbf{x}_C$. In practice, we do not know the underlying distribution $\mathbb{P}\mathbf{x}_C$. Instead, we estimate the partial dependence function:

$$\widehat{PD}_S(\mathbf{x}_S) = \frac{1}{N} \sum_{i=1}^{N} \hat{f}(\mathbf{x}_S, \mathbf{x}_C^{(i)}), \tag{19}$$

where $\mathbf{x}_C^{(i)}$ denotes the i-th observation of the complement features $\mathbf{x}_C$.

Definition 4.1.2 (Partial Dependence Plot)**.** A Partial Dependence Plot is the Graph $G(\widehat{PD}_S)$.

To visualize the PDP, we restrict ourselves to the univariate or bivariate case, i.e. $|S| \leq 2$. Also, if N is very large, we may chose quantiles or equally spaced points in the range of $\mathbf{x}_S$ to make the PDP more interpretable.

As highlighted in Goldstein et al. [17], PDPs rely on the assumption of independence between the feature subsets $\mathbf{x}_S$ and $\mathbf{x}_C$. This assumption often limits their utility in practical applications, where features are rarely independent. When $\mathbf{x}_S$ and $\mathbf{x}_C$ are dependent, the estimator $\widehat{PD}_S$ averages $\hat{f}(\mathbf{x}_S, \mathbf{x}_C^{(i)})$ over values $\mathbf{x}_C^{(i)}$ that may be unlikely (or even infeasible) for the fixed $\mathbf{x}_S$, so the PDP can reflect extrapolation artifacts rather than a meaningful marginal effect [17]. Moreover, the averaging over instances can hide substantial heterogeneity, for example when the effect of $\mathbf{x}_S$ varies across subpopulations [28, p. 205] or due to interactions with $\mathbf{x}_C$ (as can be seen in Figure 2 in Example 3.1.3, where the Mean ICE would be the same as the PDP). Nonetheless, when the underlying model is additive (with regard to $\mathbf{x}_S$), PDPs are capable of capturing the marginal effect of $\mathbf{x}_S$ precisely, irrespective of the distribution of $\mathbf{x}_C$:

Proposition 4.1.3. Let $\hat{f}$ be an additive model, i.e. $\hat{f}(\mathbf{x}) = g(x_S) + h(x_C)$, for some functions $g : \mathbb{R}^{|S|} \to \mathbb{R}$ and $h : \mathbb{R}^{|C|} \to \mathbb{R}$. Then the partial dependence function $PD(x_S)$ is given by $g(x_S) + c$ for a constant $c \in \mathbb{R}$.

Proof.

$$\begin{aligned} PD_S(x_S) &= \mathbb{E}_{x_C}[\hat{f}(x_S, x_C)] \\ &= \mathbb{E}_{x_C}[g(x_S) + h(x_C)] && \text{(Additivity)} \\ &= \mathbb{E}_{x_C}[g(x_S)] + \mathbb{E}_{x_C}[h(x_C)] && \text{(Linearity)} \\ &= g(x_S) + c \end{aligned}$$

Since $g(x_S)$ does not depend on x_C and the second term is constant in regard to x_S.

□

We will demonstrate this property in the following example.

Example 4.1.4. Let $x_1, x_2 \sim \mathcal{N}(0, 1)$ i.i.d. and $\varepsilon \sim \mathcal{N}(0, \sigma^2)$. Define a label by

$$y = f(x_1, x_2) = g(x_1) + 0.6x_2 + \varepsilon, \qquad \text{where} \qquad g(t) = \sin(1.5t) + 0.25t^2.$$

This model is additive in x_1 and x_2 and has a nonlinear marginal effect in x_1.

We assume that the fitted prediction model has perfectly learned the conditional mean, i.e.

$$\hat{f}(x_1, x_2) = \mathbb{E}[y \mid x_1, x_2] = g(x_1) + 0.6x_2.$$

Consider the univariate partial dependence function for x_1, i.e. $S = \{1\}$ and $C = \{2\}$:

$$PD_{\{1\}}(t) = \mathbb{E}_{X_2}\big[\hat{f}(t, X_2)\big] = \mathbb{E}_{X_2}\big[g(t) + 0.6X_2\big] = g(t) + 0.6\,\mathbb{E}[X_2] = g(t),$$

since $\mathbb{E}[X_2] = 0$. Hence the PDP for x_1 recovers the nonlinear effect g.

Figure 3 shows the resulting PDP for x_1.

Finally, we also consider the bivariate PDP for $S = \{1, 2\}$. In this two-feature setup we have $C = \emptyset$, hence $PD_{\{1,2\}}(t_1, t_2) = \hat{f}(t_1, t_2)$ and the bivariate PDP coincides with the response surface of $\hat{f}$. Figure 4 visualises this surface as a contour plot.

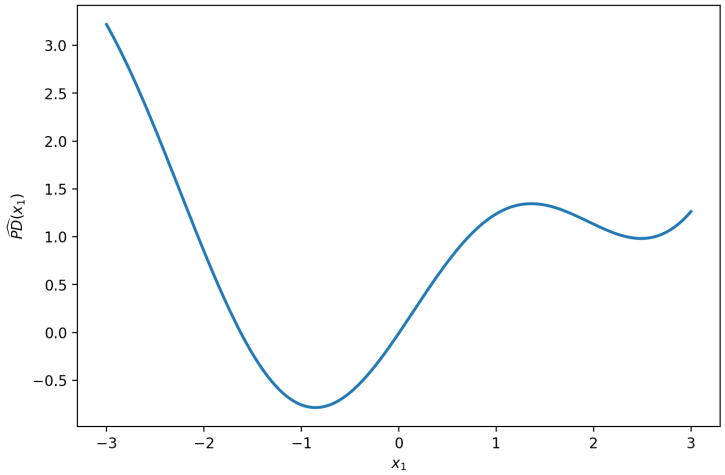


Figure 3: Partial Dependence Plot for x_1 in the additive model $f(x_1, x_2) = g(x_1) + 0.6x_2 + \varepsilon$ with $g(t) = \sin(1.5t) + 0.25t^2$ and $x_1, x_2 \sim \mathcal{N}(0, 1)$.

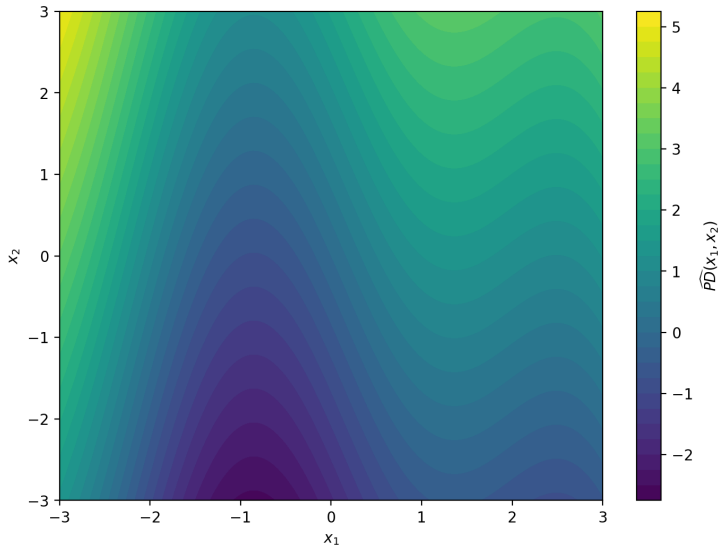


Figure 4: Bivariate PDP for (x_1, x_2) in the same setup as Figure 3. Since $C = \emptyset$ in this two-feature example, the bivariate PDP coincides with the response surface $\hat{f}(x_1, x_2) = g(x_1) + 0.6x_2$.

4.2 Accumulated Local Effects

Accumulated Local Effects (ALE) plots have been proposed by Apley and Zhu [2] as an alternative to PDPs for visualizing the effect of individual predictors in the model-agnostic setting. ALE plots aim to retain the interpretability of PDPs while avoiding their main drawbacks by aggregating local changes of the model $\hat{f}$ along the empirical distribution of the features. They therefore only require model evaluations in regions where data are actually observed. This makes ALE plots more robust to feature correlations and less prone to extrapolation issues than PDPs, as we will illustrate in Example 4.2.10 below.

We restrict ourselves to the univariate case and consider the effect of a single feature x_i on the prediction $\hat{f}(\mathbf{x})$. Information on Multivariate ALE effects and higher-order interaction ALEs can be found in Apley and Zhu [2].

Let $X = (X_1, \ldots, X_p)$ be a random vector taking values in the feature space $\mathbf{X} \subset \mathbb{R}^p$ with distribution $\mathbb{P}_X$. For a fixed $i \in \{1, \ldots, p\}$ we write

$$X_{-i} \coloneqq (X_1, \ldots, X_{i-1}, X_{i+1}, \ldots, X_p) \text{ and analogously}$$

$$x_{-i} \coloneqq (x_1, \ldots, x_{i-1}, x_{i+1}, \ldots, x_p).$$

We denote by $\mathbb{P}_{X_i}$ the marginal distribution of X_i and by $\mathbb{P}_{X_{-i}|X_i=x_i}$ the conditional distribution of X_{-i} given $X_i = x_i$. We assume that X_i is continuous with support $[a_i, b_i]$ and that $\hat{f}$ is differentiable with respect to x_i on this support.

Definition 4.2.1 (Local effect of feature i)**.** The local effect of feature i at (x_i, x_{-i}) is

$$\frac{\partial}{\partial x_i} \hat{f}(x_i, x_{-i}).$$

ALE aggregates these local effects in two steps: first, we average them over $\mathbb{P}_{X_{-i}|X_i=z}$, and second, we integrate these averaged local effects along the x_i-axis. For this we need to state a few regularity conditions:

(R1) For every x_{-i}, the mapping $x_i \mapsto \hat{f}(x_i, x_{-i})$ is differentiable on $[a_i, b_i]$.

(R2) The partial derivative $\frac{\partial}{\partial x_i} \hat{f}(x_i, x_{-i})$ is continuous in (x_i, x_{-i}) on the support of (X_i, X_{-i}).

(R3) The function

$$z \mapsto \mathbb{E}\left[\frac{\partial}{\partial x_i} \hat{f}(z, X_{-i}) \,\middle|\, X_i = z \right]$$

exists and is continuous on $[a_i, b_i]$.

Definition 4.2.2 (Uncentered ALE function)**.** Assume that R1–R3 hold. Then for $x_i \in [a_i, b_i]$ the uncentered accumulated local effect of feature i is defined as

$$g_i(x_i) \coloneqq \int_{a_i}^{x_i} \mathbb{E}\left[\tfrac{\partial}{\partial x_i} \hat{f}(z, X_{-i}) \,\middle|\, X_i = z \right] dz. \tag{20}$$

Definition 4.2.3 (Univariate ALE function)**.** The ALE function of feature i is given by

$$ALE_i(x_i) \coloneqq g_i(x_i) - \mathbb{E}\big[g_i(X_i)\big], \tag{21}$$

so that $\mathbb{E}\big[ALE_i(X_i)\big] = 0$ under the marginal distribution of X_i.

In additive settings ALE functions capture the true effect of x_i on the model, regardless of any dependence between x_i and x_{-i}, as was the case for PDPs. This is formalized in the following proposition.

Proposition 4.2.4. Let $\hat{f}(\mathbf{x}) = s(x_i) + h(x_{-i})$ for some functions s and h such that (R1)-(R3) hold. Then, for all $x_i \in [a_i, b_i]$,

$$ALE_i(x_i) = s(x_i) + c$$

for some constant $c \in \mathbb{R}$.

Proof. We have $\frac{\partial}{\partial x_i}\hat{f}(x_i, X_{-i}) = s'(x_i)$ because $\frac{\partial}{\partial x_i}h(x_{-i}) = 0$

$$\Rightarrow \mathbb{E}\big[\tfrac{\partial}{\partial x_i}\hat{f}(z, X_{-i}) \mid X_i = z\big] = \mathbb{E}\big[s'(z) \mid X_i = z\big] = s'(z)$$

$$\begin{aligned}
\Rightarrow\ g_i(x_i) &= \int_{a_i}^{x_i} \mathbb{E}\big[\tfrac{\partial}{\partial x_i}\hat{f}(z, X_{-i}) \mid X_i = z\big]\, dz \\
&= \int_{a_i}^{x_i} s'(z)\, dz \\
&= s(x_i) - s(a_i)
\end{aligned}$$

$$\begin{aligned}
\Rightarrow\ ALE_i(x_i) &= g_i(x_i) - \mathbb{E}[g_i(X_i)] \\
&= \big(s(x_i) - s(a_i)\big) - \mathbb{E}\big[s(X_i) - s(a_i)\big] \\
&= s(x_i) - \mathbb{E}[s(X_i)] \\
&= s(x_i) + c, \text{ where } c = -\mathbb{E}[s(X_i)]
\end{aligned}$$

□

Definition 4.2.2 relies on the differentiability of $\hat{f}$. Common models such as decision trees and random forests do not exhibit the regularity that is needed. Apley and Zhu [2] therefore define ALE functions via finite differences and limits over increasingly fine partitions:

Let $[a_i, b_i]$ be partitioned into K intervals $(z_{k-1}, z_k]$ such that

$$a_i = z_0 < z_1 < \cdots < z_K = b_i.$$

For $x_i \in (z_{k-1}, z_k]$ denote by $k(x_i) = k$ the index of the interval containing x_i and let $k(a_i) = 1$.

Definition 4.2.5 (Uncentered Finite-difference ALE function)**.** For a fixed partition $\{z_k\}_{k=0}^{K}$ define

$$g_i^{(K)}(x_i) \coloneqq \sum_{k=1}^{k(x_i)} \mathbb{E}\big[\hat{f}(z_k, X_{-i}) - \hat{f}(z_{k-1}, X_{-i}) \,\big|\, X_i \in (z_{k-1}, z_k]\big]. \tag{22}$$

If, as $K \to \infty$ and $\max_k(z_k - z_{k-1}) \to 0$, the functions $g_i^{(K)}$ converge to a limit g_i that does not depend on the particular sequence of partitions, this limit is called the uncentered ALE function of feature i. The centered ALE function emerges as in Definition 4.2.3 by subtracting the marginal expectation of g_i.

Sufficient conditions for the existence and uniqueness of this limit are given in the supplementary material of Apley and Zhu [2]. For sufficiently regular models Definition 4.2.2 and Definition 4.2.5 coincide. This theorem and its proof are adapted from Apley and Zhu [2].

Theorem 4.2.6. Assume that $\hat{f}$ is differentiable with respect to x_i on $[a_i, b_i]$ and that (R1)-(R3) hold. Then, for all $x_i \in [a_i, b_i]$,

$$\lim_{K\to\infty} g_i^{(K)}(x_i) \;=\; \int_{a_i}^{x_i} \mathbb{E}\big[\tfrac{\partial}{\partial x_i}\hat{f}(z, X_{-i}) \mid X_i = z\big]\, dz. \tag{23}$$

Proof. Fix i and let $\{z_k\}_{k=0}^{K}$ be a partition of $[a_i, b_i]$ with $\delta_K \coloneqq \max_{k=1,\dots,K}\{z_k - z_{k-1}\} \to 0$. For $x_i \in [a_i, b_i]$ let $k(x_i)$ be such that $x_i \in (z_{k(x_i)-1}, z_{k(x_i)}]$ and set

$$g_i^{(K)}(x_i) = \sum_{k=1}^{k(x_i)} \mathbb{E}\big[\hat{f}(z_k, X_{-i}) - \hat{f}(z_{k-1}, X_{-i}) \,\big|\, X_i \in (z_{k-1}, z_k]\big].$$

Define

$$m(z) \coloneqq \mathbb{E}\big[\tfrac{\partial}{\partial x_i}\hat{f}(z, X_{-i}) \mid X_i = z\big], \qquad z \in [a_i, b_i].$$

$$\begin{aligned}
g_i^{(K)}(x_i) &= \sum_{k=1}^{k(x_i)} \mathbb{E}\big[\hat{f}(z_k, X_{-i}) - \hat{f}(z_{k-1}, X_{-i}) \,\big|\, X_i \in (z_{k-1}, z_k]\big] \\
&= \sum_{k=1}^{k(x_i)} \mathbb{E}\Big[\int_{z_{k-1}}^{z_k} \tfrac{\partial}{\partial x_i}\hat{f}(t, X_{-i})\, dt \;\Big|\; X_i \in (z_{k-1}, z_k]\Big] \quad \text{(by the FTC)} \\
&= \sum_{k=1}^{k(x_i)} \int_{z_{k-1}}^{z_k} \mathbb{E}\big[\tfrac{\partial}{\partial x_i}\hat{f}(t, X_{-i}) \,\big|\, X_i \in (z_{k-1}, z_k]\big]\, dt \text{ (Fubini's theorem)}
\end{aligned}$$

By Assumption (R3), m is continuous on $[a_i, b_i]$, hence uniformly continuous, and

$$\sup_{t\in(z_{k-1},z_k]} \Big|\mathbb{E}\big[\tfrac{\partial}{\partial x_i}\hat{f}(t, X_{-i}) \mid X_i \in (z_{k-1}, z_k]\big] - m(t)\Big| \to 0 \qquad (K \to \infty),$$

uniformly in k. Thus, for every $\varepsilon > 0$ there exists K_0 such that for all $K \geq K_0$, for all $k \leq k(x_i)$ and all $t \in (z_{k-1}, z_k]$,

$$\left| \mathbb{E}\big[\tfrac{\partial}{\partial x_i} \hat{f}(t, X_{-i}) \mid X_i \in (z_{k-1}, z_k] \big] - m(t) \right| \leq \varepsilon. \tag{24}$$

Therefore, for $K \geq K_0$,

$$\begin{aligned}
\left| g_i^{(K)}(x_i) - \sum_{k=1}^{k(x_i)} \int_{z_{k-1}}^{z_k} m(t)\, dt \right| &\leq \sum_{k=1}^{k(x_i)} \int_{z_{k-1}}^{z_k} \varepsilon\, dt \quad \text{(because (24))} \\
&= \varepsilon \sum_{k=1}^{k(x_i)} (z_k - z_{k-1}) \\
&= \varepsilon\, (z_{k(x_i)} - a_i) \\
&\leq \varepsilon\, (b_i - a_i).
\end{aligned}$$

$$\begin{aligned}
\Rightarrow g_i^{(K)}(x_i) &\to \sum_{k=1}^{k(x_i)} \int_{z_{k-1}}^{z_k} m(t)\, dt \\
&= \int_{a_i}^{z_{k(x_i)}} m(t)\, dt.
\end{aligned} \tag{25}$$

Since $|z_{k(x_i)} - x_i| \leq \delta_K \to 0$ and m is continuous,

$$\int_{a_i}^{z_{k(x_i)}} m(t)\, dt \to \int_{a_i}^{x_i} m(t)\, dt. \tag{26}$$

Combining (25) and (26) yields

$$\lim_{K \to \infty} g_i^{(K)}(x_i) = \int_{a_i}^{x_i} \mathbb{E}\big[\tfrac{\partial}{\partial x_i} \hat{f}(z, X_{-i}) \mid X_i = z \big]\, dz.$$

□

The finite difference formulation of ALE functions allows us to estimate the ALE function (20) from practical data (where the underlying distributions are unknown) and also in a model-agnostic way (where $\hat{f}$ may not be differentiable). The following estimator is also proposed in Apley and Zhu [2]. Let

$$a_i = z_0 < z_1 < \cdots < z_K = b_i$$

be a fixed partition of the observed values $\{x_i^{(n)}\}_{n=1}^N$ of feature i into K intervals $N_i(k) = (z_{k-1}, z_k]$, $k = 1, \ldots, K$. Again, let $a_i \in N_i(1)$ and denote by

$$n_i(k) \coloneqq \left| \{ n : x_i^{(n)} \in N_i(k) \} \right|$$

the number of observations in the k-thinterval.

Definition 4.2.7 (Empirical uncentered ALE)**.** For $x_i \in (z_{k-1}, z_k]$ define

$$\hat{g}_i(x_i) \coloneqq \sum_{\ell=1}^{k(x_i)} \frac{1}{n_i(\ell)} \sum_{n:x_i^{(n)} \in N_i(\ell)} \big(\hat{f}(z_\ell, x_{-i}^{(n)}) - \hat{f}(z_{\ell-1}, x_{-i}^{(n)})\big). \tag{27}$$

Thus, for each bin ℓ we compute the average change in prediction when we move x_i from the left endpoint $z_{\ell-1}$ to the right endpoint z_ℓ, keeping x_{-i} fixed at its observed values within that bin. The function $\hat{g}_i$ is the cumulative sum of these average local effects over all bins up to the value x_i.

Definition 4.2.8 (Empirical ALE function)**.** The empirical ALE function of feature i is given by

$$\widehat{ALE}_i(x_i) \coloneqq \hat{g}_i(x_i) - \frac{1}{N} \sum_{n=1}^{N} \hat{g}_i(x_i^{(n)}). \tag{28}$$

Definition 4.2.9 (ALE plot)**.** An ALE plot for feature i is the graph of the empirical ALE function, i.e. the graph $G(\widehat{ALE}_i)$. In practice, $G(\widehat{ALE}_i)$ is visualized by evaluating $\widehat{ALE}_i(x_i)$ on a grid of values x_i in the observed range of feature i (e.g. empirical quantiles of $\{x_i^{(n)}\}_{n=1}^N$).

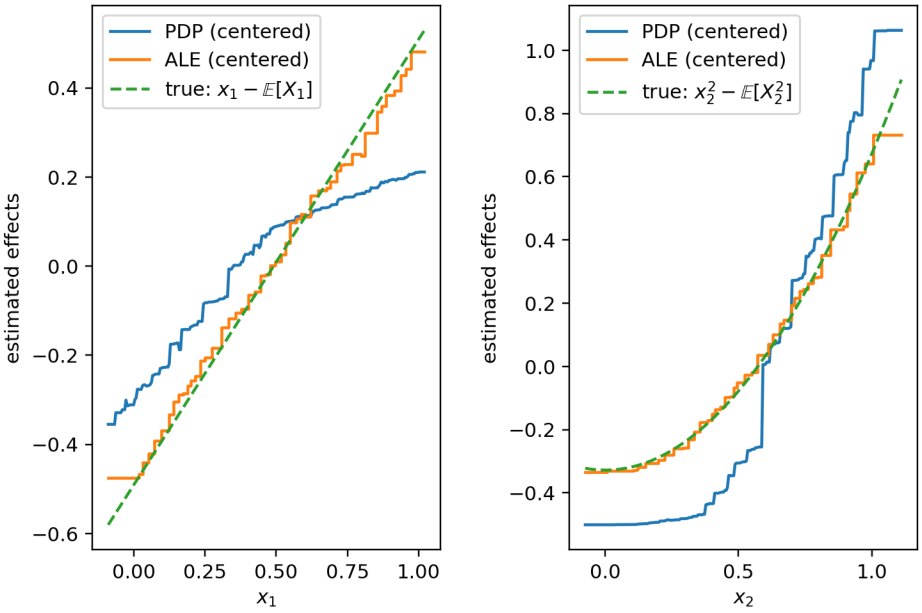
We now illustrate how ALE plots can be more reliable than PDPs in the presence of strong feature correlations, following the conceptual setup of Apley and Zhu [2].

Example 4.2.10. Let $Z \sim \mathcal{U}[0,1]$ and $\varepsilon_1, \varepsilon_2 \sim \mathcal{N}(0, \sigma^2)$ i.i.d. and define two features and a label by

$$x_1 = Z + \varepsilon_1, \qquad x_2 = Z + \varepsilon_2, \qquad y = f(x_1, x_2) = x_1 + x_2^2$$

We see that x_1 and x_2 are strongly correlated through the common variable Z. We train a Decision Tree regressor $\hat{f}$ (`max_depth`=30) on $N = 1000$ samples from this model, with $\sigma = 0.05$. We compare the one-dimensional PDP and ALE for x_1 and x_2. As reference, we also plot the true centered main effects shown as dashed lines in Figure 5.

Since x_1 and x_2 are correlated, the PDP evaluates $\hat{f}$ at many feature combinations $(x_i, x_{-i}^{(n)})$ that are far away from underlying data, which distorts the estimation. In contrast, ALE averages finite differences within bins of x_i, using observations with x_i in the same bin, thereby staying closer to the distribution of the data. If we did not know the underlying true function, the ALE plot would give us a much better idea of the actual effect of x_1 and x_2 on the model predictions than the PDP.



(a) Effects of x_1.

(b) Effects of x_2.

Figure 5: comparison of centered PDP and ALE plot for x_1 (left) and x_2 (right). The dashed curves show the corresponding true centered main effects $x_1 - \mathbb{E}[X_1]$ and $x_2^2 - \mathbb{E}[X_2^2]$.

4.3 Permutation Feature Interaction

Feature Importance methods aim to quantify the contribution of individual input variables to a model's predictive performance. One method for this is Permutation Feature Importance (PFI), where the performance of the model on a test set is evaluated before and after permuting the values of a specific feature. This permutation breaks the association between that feature and the label, allowing us to assess how much the model relies on that feature for its predictions. In contrast to Leave-One-Feature-Out Importance [24], PFI does not require retraining of the model, making it computationally efficient. PFI was first introduced by Breiman [6] for Random Forests and later generalized to model-agnostic settings by Fisher, Rudin, and Dominici [12].

Definition 4.3.1. Let $\mathbf{x}_i = (\mathbf{x}_i^{(1)}, \ldots, \mathbf{x}_i^{(N)})$ a feature vector and π a permutation of $\{1, \ldots, N\}$. Then we define the permuted feature vector $\mathbf{x}_{i,\pi}$ as follows:

$$\mathbf{x}_{i,\pi} = (\mathbf{x}_i^{(\pi(1))}, \ldots, \mathbf{x}_i^{(\pi(N))})$$

We also introduce the following useful short-hands:

$$\mathbf{x}_{-i} \coloneqq (\mathbf{x}_1, \ldots, \mathbf{x}_{i-1}, \mathbf{x}_{i+1}, \ldots, \mathbf{x}_p)$$

$$f(\mathbf{x}_{-i}, \mathbf{x}_{i,\pi}) \coloneqq f(\mathbf{x}_1, \dots, \mathbf{x}_{i-1}, \mathbf{x}_{i,\pi}, \mathbf{x}_{i+1}, \dots, \mathbf{x}_p)$$

Definition 4.3.2 (Permutation Feature Importance (PFI))**.** Let $\hat{f}\colon \mathbf{X} \to \mathbf{Y}$ a model, ℓ a loss function and $\mathbf{x}_i$ a feature vector. We define Permutation Feature Importance (PFI) for feature $i \in \{1, \dots, p\}$ as

$$PFI(i) = \mathbb{E}_{\mathbf{x}}[\ell(y, \hat{f}(\mathbf{x}))] - \mathbb{E}_{\mathbf{x}_{-i}}[\ell(y, \hat{f}(\mathbf{x}_{-i}, \mathbf{x}_{i,\pi}))]. \tag{29}$$

In practice, we do not know the underlying distribution $\mathbb{P}_{\mathbf{x}}$. Also, the results depend on the chosen permutation π, especially for small N. Because of this, we randomly sample m permutations $\pi_1, \dots, \pi_m$ and estimate the PFI as follows:

$$\hat{PFI}(i) = \frac{1}{m} \sum_{k=1}^{m} \Big(\frac{1}{N} \sum_{j=1}^{N} \ell(y^{(j)}, \hat{f}(\mathbf{x})^{(j)}) - \frac{1}{N} \sum_{j=1}^{N} \ell(y^{(j)}, \hat{f}(\mathbf{x}_{-i}, \mathbf{x}_{i,\pi_k})^{(j)}) \Big) \tag{30}$$

Note that we use the difference of losses here, while traditionally the ratio is used ([6, 12]). Both approaches are equally valid and represent an absolute or relative decrease in model performance, respectively.

4.3.1 Conditional Permutation Feature Importance

The definition of the Permutation Feature Importance given above relies on permuting feature values independently of all other features. This procedure destroys any statistical dependencies between the permuted feature $\mathbf{x}_i$ and the remaining features $\mathbf{x}_{-i}$, such that the resulting joint distribution generally no longer resembles the true data-generating distribution. Consequently, the model $\hat{f}$ is evaluated on feature combinations that may be highly unlikely or even impossible. The resulting quantity therefore measures a marginal importance of $\mathbf{x}_i$, which captures the contribution of $\mathbf{x}_i$ under small perturbations but not its importance conditional on other features.

To address this issue, conditional variants of the PFI have been proposed ([43, 20, 29]). These aim to preserve the dependence structure between features and thereby quantify the importance of $\mathbf{x}_i$ given $\mathbf{x}_{-i}$.

Definition 4.3.3 (Conditional Permutation Feature Importance)**.** Let $\tilde{\mathbf{x}}_i$ be a random variable sampled from the conditional distribution $\mathbb{P}_{\mathbf{x}_i|\mathbf{x}_{-i}}$. The conditional Permutation Feature Importance (cPFI) of feature i is defined as

$$cPFI(i) = \mathbb{E}_{\mathbf{x}}[\ell(y, \hat{f}(\mathbf{x})) - \mathbb{E}[\ell(y, \hat{f}(\tilde{\mathbf{x}}_i, \mathbf{x}_{-i})) \,|\, \mathbf{x}_{-i}]] \tag{31}$$

We will illustrate the difference between marginal PFI and conditional PFI in the following example.

Example 4.3.4. Let $Z \sim \mathcal{N}(0, 1)$ and $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ be independent. We define two features and a label by

$$\mathbf{x}_1 = Z, \qquad \mathbf{x}_2 = Z, \qquad y = Z + \varepsilon.$$

Obviously, $\mathbf{x}_2$ is highly correlated with $\mathbf{x}_1$ (it is a copy).

We consider the squared error loss $\ell(y, \hat{y}) = (y - \hat{y})^2$ and the prediction model

$$\hat{f}(\mathbf{x}_1, \mathbf{x}_2) = \frac{\mathbf{x}_1 + \mathbf{x}_2}{2}.$$

This means that $\hat{f}(\mathbf{x}_1, \mathbf{x}_2) = Z$, so our model predicts the true function perfectly (without noise ε). The expected loss of our model is

$$\mathbb{E}_{\mathbf{x}}[(y - \hat{f}(\mathbf{x}_1, \mathbf{x}_2))^2] = \mathbb{E}_{\mathbf{x}}[(Z + \varepsilon - Z)^2] = \mathbb{E}_{\mathbf{x}}[\varepsilon^2] = \mathrm{Var}(\varepsilon) = \sigma^2.$$

Next, we will compute (marginal) PFI for feature $\mathbf{x}_2$.

$$PFI(2) = \sigma^2 - \mathbb{E}_{\mathbf{x}_1}[\ell(y, \hat{f}(\mathbf{x}_1, \mathbf{x}_{2,\pi}))],$$

We can view the permuted feature $\mathbf{x}_{2,\pi}$ as an independent copy of $\mathbf{x}_2$, denoted by $Z' \sim \mathcal{N}(0, 1)$, which is independent of both Z and ε.

$$\begin{aligned}
\Rightarrow \hat{f}(\mathbf{x}_1, \mathbf{x}_{2,\pi}) &= \frac{Z + Z'}{2} \\
\Rightarrow PFI(2) &= \sigma^2 - \mathbb{E}_{\mathbf{x}_1}[((Z + \varepsilon) - \frac{Z + Z'}{2})^2] \\
&= \sigma^2 - \mathbb{E}_{\mathbf{x}_1}[(\varepsilon + \frac{Z - Z'}{2})^2] \\
&= \sigma^2 - (\mathbb{E}_{\mathbf{x}_1}[\varepsilon^2] + \frac{1}{4}\mathbb{E}_{\mathbf{x}_1}[(Z - Z')^2]) \\
&= \sigma^2 - (\sigma^2 + \frac{1}{4}(\mathbb{E}_{\mathbf{x}_1}(Z^2) - \mathbb{E}_{\mathbf{x}_1}(Z'^2))) \\
&= \frac{1}{2}
\end{aligned}$$

Hence, the standard (marginal) PFI attributes positive importance to $\mathbf{x}_2$, even though $\mathbf{x}_2$ is just a duplicate of $\mathbf{x}_1$ and does not provide additional information beyond $\mathbf{x}_1$. In contrast, we will now compute the cPFI for feature $\mathbf{x}_2$.

$$cPFI(2) = \mathbb{E}_{\mathbf{x}}[\ell(y, \hat{f}(\mathbf{x})) - \mathbb{E}[\ell(y, \hat{f}(\mathbf{x}_1, \tilde{\mathbf{x}}_2)) \,|\, \mathbf{x}_1]],$$

where $\tilde{\mathbf{x}}_2$ is drawn from the conditional distribution $\mathbb{P}_{\mathbf{x}_2|\mathbf{x}_1}$. However, we know that

$$\begin{aligned}
\mathbb{P}_{\mathbf{x}_2|\mathbf{x}_1}(\mathbf{x}_2 = \mathbf{x}_1 \,|\, \mathbf{x}_1) &= 1 \\
\Rightarrow \tilde{\mathbf{x}}_2 &= \mathbf{x}_1 \text{ almost surely} \\
\Rightarrow \hat{f}(\mathbf{x}_1, \tilde{\mathbf{x}}_2) &= \frac{\mathbf{x}_1 + \tilde{\mathbf{x}}_2}{2} = \frac{\mathbf{x}_1 + \mathbf{x}_1}{2} = Z = \hat{f}(\mathbf{x}_1, \mathbf{x}_2) \text{ a.s.} \\
\Rightarrow cPFI(2) &= \mathbb{E}_{\mathbf{x}}[\ell(y, \hat{f}(\mathbf{x})) - \mathbb{E}[\ell(y, \hat{f}(\mathbf{x})) \,|\, \mathbf{x}_1]] \\
&= \mathbb{E}_{\mathbf{x}}[\ell(y, \hat{f}(\mathbf{x}))] - \mathbb{E}_{\mathbf{x}}[\mathbb{E}[\ell(y, \hat{f}(\mathbf{x})) \,|\, \mathbf{x}_1]] \\
&= \mathbb{E}_{\mathbf{x}}[\ell(y, \hat{f}(\mathbf{x}))] - \mathbb{E}_{\mathbf{x}}[\ell(y, \hat{f}(\mathbf{x}))] \qquad \text{(tower rule)} \\
&= 0
\end{aligned}$$

Thus, in this example the conditional PFI correctly assigns zero importance to the surrogate feature $\mathbf{x}_2$. The PFI on the other hand evaluates model performance on unrealistic feature combinations (where $\mathbf{x}_2 \neq \mathbf{x}_1$) and therefore attributes a strictly positive importance ($\text{PFI}(2) = \frac{1}{2}$).

So we demonstrated that the cPFI provides a more realistic estimate of feature importance in the presence of correlated features. However, we need to estimate (31) because $\mathbb{P}_{\mathbf{x}_i|\mathbf{x}_{-i}}$ is unknown. This is more difficult than estimating (29). We will present two different strategies for this: Conditional Subgroups and Knockoff Sampling.

The key idea of the **conditional subgroup (CS)** approach of Molnar et al. [29] is to partition the feature space into subgroups in which $\mathbf{x}_i$ and $\mathbf{x}_{-i}$ are approximately independent. Within such a subgroup, permuting $\mathbf{x}_i$ provides a reasonable approximation to sampling from the conditional distribution.

Definition 4.3.5 (Subgroup Partition)**.** A subgroup partition for feature i is a collection $G = \{G_1, \ldots, G_K\}$ of disjoint sets whose union is the feature space $\mathbf{X}$ with a mapping

$$g_i : \mathbf{X} \to \{1, \ldots, K\}, \qquad g_i(\mathbf{x}) = k \iff \mathbf{x} \in G_k,$$

such that $g_i(\mathbf{x})$ depends only on $\mathbf{x}_{-i}$.

Definition 4.3.6 (Conditional subgroup PFI)**.** Let $g_i(\mathbf{x})$ induce a partition of the feature space into subgroups $G_1, \ldots, G_K$ and let $n_k = |G_k|$. We then define

$$\hat{PFI}^k_{CS}(i) = \frac{1}{m}\sum_{h=1}^{m}\Big(\frac{1}{n_k}\sum_{j:\mathbf{x}^{(j)}\in G_k}\left(\ell(y^{(j)}, \hat{f}(\mathbf{x})^{(j)}) - \ell(y^{(j)}, \hat{f}(\mathbf{x}_{-i}, \mathbf{x}_{i,\pi_h})^{(j)})\right)\Big) \tag{32}$$

where $\mathbf{x}_{i,\pi_h}$ is a permutation of $\mathbf{x}_i$ restricted to the subgroup G_k. the cPFI is then estimated as

$$\hat{cPFI}_{CS}(i) = \frac{1}{N}\sum_{k=1}^{K} n_k \hat{PFI}^k_{CS}(i). \tag{33}$$

In practice, the partition g_i is obtained by fitting a decision tree that predicts $\mathbf{x}_i$ from the remaining features $\mathbf{x}_{-i}$. The tree recursively splits the space of $\mathbf{x}_{-i}$ so that the conditional distribution of $\mathbf{x}_i$ becomes more homogeneous within each terminal node. It is recommended in Molnar et al. [29] to use transformation trees [21], which test for general distributional changes in $\mathbf{x}_i$ when splitting on $\mathbf{x}_{-i}$. However, simpler CART trees may also be used. The resulting subgroups serve as interpretable regions in which the residual dependence between $\mathbf{x}_i$ and $\mathbf{x}_{-i}$ is weak enough that within-group permutations approximate draws from $\mathbb{P}_{\mathbf{x}_i|\mathbf{x}_{-i}}$. Note that it is shown in Molnar et al. [29] that the estimator in (33) is equal to the true cPFI (31) if you have constructed perfect subgroups such that $\mathbf{x}_i$ and $\mathbf{x}_{-i}$ are independent within each subgroup.

Another approach to estimate (31) is the **Conditional Predictive Impact (CPI)** framework of Watson and Wright [47] which introduced the use of

knockoffs to estimate conditional feature importance. Instead of explicitly approximating the conditional distribution $\mathbb{P}_{\mathbf{x}_i|\mathbf{x}_{-i}}$, CPI introduces knockoff copies of the features and then measures how much predictive performance deteriorates when the original feature is replaced by its knockoff, similar to how PFI measures the deterioration after replacing a feature by its permuted version.

Definition 4.3.7 (Model-X Knockoffs, after Candès et al. [9])**.** Let $\mathbf{x} = (X_1, \ldots, X_p)$ be a vector of random variables. A random vector $\tilde{\mathbf{x}} = (\tilde{X}_1, \ldots, \tilde{X}_p)$ is called a Model-X knockoff of $\mathbf{x}$ if the following two properties hold:

(P1) (Pairwise exchangeability) For any subset $A \subseteq \{1, \ldots, p\}$,

$$(\mathbf{x}, \tilde{\mathbf{x}}) \overset{d}{=} (\mathbf{x}, \tilde{\mathbf{x}})_{\text{swap}(A)},$$

where $(\mathbf{x}, \tilde{\mathbf{x}})_{\text{swap}(A)}$ is defined by exchanging X_j and $\tilde{X}_j$ for every $j \in A$.

(P2) (Conditional independence from the response)

$$\tilde{\mathbf{x}} \perp\!\!\!\perp y \mid \mathbf{x}.$$

That is, knockoffs may depend on $\mathbf{x}$ but must carry no additional information about y.

Property (P1) states that swapping any coordinates of $\mathbf{x}$ with their corresponding knockoff coordinates leaves the joint distribution invariant. Together, (P1) and (P2) ensure that $\tilde{\mathbf{x}}$ mimics the dependence structure of $\mathbf{x}$ but plays the role of a negative control when assessing conditional importance. We will not go into further details about the construction of knockoffs, but there are various methods to generate knockoff variables depending on the data distribution (see [9, 38, 37, 3, 42]).

To estimate (31) using knockoffs, we just replace the permuted feature with the knockoff:

$$\hat{cPFI}_{CPI}(i) \coloneqq \frac{1}{N} \sum_{j=1}^{N} \left(\ell(y^{(j)}, \hat{f}(\mathbf{x})^{(j)}) - \ell(y^{(j)}, \hat{f}(\mathbf{x}_{-i}, \tilde{\mathbf{x}}_i)^{(j)}) \right) \tag{34}$$

4.4 Feature Interaction

The global methods introduced so far primarily describe main effects, i.e. how $\hat{f}(\mathbf{x})$ changes on average when varying one feature while accounting for the remaining features. In many applications, however, the influence of one feature depends on the value of another feature. Such feature interactions are important for interpretation because statements of the form "increasing x_j increases the prediction" may only be valid within a certain context (values of other predictors).

A useful conceptual device is the decomposition discussed by Friedman [15], which separates a function into main effects, pairwise interactions, and higher-order terms. Heuristically,

$$\hat{f}(\mathbf{x}) \;\approx\; \sum_{j=1}^{p} f_j(x_j) \;+\; \sum_{j<k} f_{jk}(x_j, x_k) \;+\; \cdots, \tag{35}$$

so that the presence of interactions corresponds to non-negligible f_{jk} terms. Note that modelling influences the presence of interaction terms: linear models do not exhibit interactions, because the structure of the model only allows for additive contributions of each feature. In (boosted) regression trees, by restricting the depth of the individual trees, one implicitly restricts the maximum order of interaction terms contributed by each tree (and thus by the ensemble) [15].

To measure interactions in a model-agnostic way, Friedman and Popescu [14] propose statistics based on partial dependence functions. This thesis introduced them in Definition 4.1.1, and we use that notation here.

In section 4.1 we showed that whenever the model is additive with respect to a feature subset $\mathbf{x}_S$, the corresponding partial dependence function $PD_S(\mathbf{x}_S)$ recovers that component up to an additive constant (Proposition 4.1.3). We now apply this idea to a feature pair (x_j, x_k). Assume that there is no interaction between x_j and x_k, which means that the model can be written as

$$\hat{f}(\mathbf{x}) = g_j(x_j) + g_k(x_k) + h(x_{-\{j,k\}})$$

for some functions g_j, g_k and a remainder h that does not depend on (x_j, x_k) jointly. By Proposition 4.1.3 this implies that the bivariate PD is

$$PD_{jk}(\mathbf{x}_j, \mathbf{x}_k) = PD_j(\mathbf{x}_j) + PD_k(\mathbf{x}_k) + c,$$

for some constant $c \in \mathbb{R}$. When switching to the estimated partial dependence function $\widehat{PD}$ we can remove this constant by centering the PDs.

Definition 4.4.1 (Centered PD)**.** Let $S \subset \{1, \ldots, p\}$. The centered partial dependence function for $\mathbf{x}_S$ is defined as

$$\widetilde{PD}_S(\mathbf{x}_S) = \widehat{PD}_S(\mathbf{x}_S) - \frac{1}{N}\sum_{i=1}^{N} \widehat{PD}_S(\mathbf{x}^{(i)}{}_S). \tag{36}$$

We now have $\widetilde{PD}_{jk}(\mathbf{x}_{\{j,k\}}) = \widetilde{PD}_j(\mathbf{x}_j) + \widetilde{PD}_k(\mathbf{x}_k)$. Based on this observation, Friedman and Popescu [14] define a measure of interaction strength as follows:

Definition 4.4.2 (Friedman's H-statistic (pairwise))**.** Let $j \neq k$. Using the centered partial dependence estimates $\widetilde{PD}_{jk}, \widetilde{PD}_j, \widetilde{PD}_k$, the (squared) interaction strength between $\mathbf{x}_j$ and $\mathbf{x}_k$ is defined as

$$H_{jk}^2 \coloneqq \frac{\sum_{i=1}^{N}\left[\widetilde{PD}_{jk}(\mathbf{x}_j^{(i)}, \mathbf{x}_k^{(i)}) - \widetilde{PD}_j(\mathbf{x}_j^{(i)}) - \widetilde{PD}_k(\mathbf{x}_k^{(i)})\right]^2}{\sum_{i=1}^{N} \widetilde{PD}_{jk}(\mathbf{x}_j^{(i)}, \mathbf{x}_k^{(i)})^2}. \tag{37}$$

We can see that $H_{jk}^2 \geq 0$, where values close to 0 indicate little interaction (because $\widetilde{PD}_{jk}(\mathbf{x}_{jk}) \approx \widetilde{PD}_j(\mathbf{x}_j) + \widetilde{PD}_k(\mathbf{x}_k)$), and larger values indicate a stronger interaction. Friedman and Popescu [14] also provide the following statistic to measure the interaction strength of one feature with all other features.

Definition 4.4.3 (Friedman's H-statistic (feature-wise))**.** Let $j \in \{1, \ldots, p\}$ and denote by x_{-j} all components of $\mathbf{x}$ except x_j. Using centered partial dependence estimates $\widetilde{PD}_j$ and $\widetilde{PD}_{-j}$, define

$$H_j^2 \coloneqq \frac{\sum_{i=1}^N \big[\hat{f}(\mathbf{x}^{(i)}) - \widetilde{PD}_j(\mathbf{x}_j^{(i)}) - \widetilde{PD}_{-j}(\mathbf{x}^{(i)}{}_{-j})\big]^2}{\sum_{i=1}^N \hat{f}(\mathbf{x}^{(i)})^2}. \tag{38}$$

In the following example we illustrate how H_{jk}^2 changes as we vary the interaction strength in a simple model.

Example 4.4.4. Let $X_1 \sim \mathcal{N}(0, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(0, \sigma_2^2)$ be independent. Define two features and a model via

$$x_1 = X_1, \quad x_2 = X_2, \quad \hat{f}(x_1, x_2) = x_1 + x_2 + \alpha x_1 x_2,$$

For the partial dependence of x_1 we average over the marginal distribution of X_2.

$$\begin{aligned} PD_1(x_1) &= \mathbb{E}_{X_2}\big[\hat{f}(x_1, X_2)\big] \\ &= \mathbb{E}\big[x_1 + X_2 + \alpha x_1 X_2\big] \\ &= x_1 + \mathbb{E}[X_2] + \alpha x_1 \mathbb{E}[X_2] \\ &= x_1 \qquad\qquad (\mathbb{E}[X_2] = 0) \end{aligned}$$

Analogously, for $PD_2(x_2)$ we obtain

$$PD_2(x_2) = \mathbb{E}_{X_1}\big[\hat{f}(X_1, x_2)\big] = \mathbb{E}\big[X_1 + x_2 + \alpha X_1 x_2\big] = x_2.$$

For the bivariate partial dependence there are no complement features to average out, hence

$$PD_{12}(x_1, x_2) = \hat{f}(x_1, x_2) = x_1 + x_2 + \alpha x_1 x_2.$$

Therefore the interaction remainder is

$$PD_{12}(x_1, x_2) - PD_1(x_1) - PD_2(x_2) = \alpha x_1 x_2.$$

We know that $\mathbb{E}[X_1] = \mathbb{E}[X_2] = 0 \Rightarrow \mathbb{E}[X_1 X_2] = 0 \Rightarrow \mathbb{E}[PD_{12}(x_1, x_2)] = 0$. Therefore we do not need to center the PDs in this example and can plug them

directly into Equation (37) to get

$$
\begin{aligned}
H_{12}^2 &= \frac{\sum_{i=1}^{N}(\alpha X_1^{(i)} X_2^{(i)})^2}{\sum_{i=1}^{N}(X_1^{(i)} + X_2^{(i)} + \alpha X_1^{(i)} X_2^{(i)})^2} \\
&= \frac{\alpha^2 \frac{1}{N}\sum_{i=1}^{N}(X_1^{(i)} X_2^{(i)})^2}{\frac{1}{N}\sum_{i=1}^{N}(X_1^{(i)} + X_2^{(i)} + \alpha X_1^{(i)} X_2^{(i)})^2} \\
&= \frac{\alpha^2 \mathbb{E}[(X_1 X_2)^2]}{\mathbb{E}[(X_1 + X_2 + \alpha X_1 X_2)^2]} \qquad (X_1^{(i)} X_2^{(i)} \text{ are i.i.d. } \forall i\text{; LLN for } N \to \infty) \\
&= \frac{\alpha^2\, \mathbb{E}[X_1^2]\mathbb{E}[X_2^2]}{\mathbb{E}[X_1^2] + \mathbb{E}[X_2^2] + \alpha^2\, \mathbb{E}[X_1^2]\mathbb{E}[X_2^2]} \quad \text{(cross-terms in denominator vanish)} \\
&= \frac{\alpha^2 \sigma_1^2 \sigma_2^2}{\sigma_1^2 + \sigma_2^2 + \alpha^2 \sigma_1^2 \sigma_2^2}.
\end{aligned}
$$

We see that $H_{12}^2 = 0 \Leftrightarrow \alpha = 0$, and $H_{12}^2 \to 1$ as $|\alpha| \to \infty$. This means that if there is no interaction ($\alpha = 0$), the interaction strength is 0, and for very strong interactions ($|\alpha| \to \infty$) the interaction strength approaches its maximum value of 1.

5 Case Study

For the case study a dataset about board games was chosen. See Appendix A for a detailed description of how the data was collected and preprocessed. This final dataset has a total of 61,530 games and 388 features. The label for all prediction tasks is `avg_rating`, the average of all user ratings for a game on a scale from 1 (worst) to 10 (best). Figure 6 shows the empirical distribution of `avg_rating`, which resembles a slightly skewed normal distribution. Two of the 388 features are identifying features (`name` and `bgg_id` of the game) and thus not relevant for the ML tasks. Then there are 13 numerical features and 373 boolean features. Most of the boolean features are quite sparse, i.e. they have the value 1 for only a few games and are 0 for the vast majority.

The dataset is derived from BoardGameGeek user ratings and metadata. Consequently, the label `avg_rating` should be interpreted as the mean rating within the BGG user community rather than as an estimate of the general population's preferences. In particular, rating activity on BGG is subject to self-selection: users who rate games are more likely to be highly engaged boardgame enthusiasts. This may induce systematic preference shifts (e.g., toward higher strategic depth and lower reliance on chance) and can lead models trained on this label to overemphasize features correlated with "enthusiast" tastes such as complexity, long play times, or specific mechanics.

Further, the data may exhibit temporal and survivorship-related selection effects. Older games recorded on BGG are more likely to be titles that remained available, discussed, or culturally salient, which can bias observed ratings upward relative to the full historical set of released games. Conversely, newer games may be influenced by recency effects in the rating process. In addition, several features (categories/mechanics) are community-curated and sparse, implying potential measurement noise and missingness.

Finally, some variables can act as proxies for selection mechanisms rather than intrinsic game quality. For example, expansions typically exist only for base games that were commercially successful or well-received, so the indicator `category_Expansion for Base-game` may partly capture a selection effect. Overall, these limitations should be kept in mind when interpreting global and local explanations: they describe patterns learned from the BGG ecosystem and do not necessarily generalize to broader player populations. The following analyses should thereby be viewed as descriptive rather than causal.

It was necessary to have different models to compare the performance of the interpretability methods and to check whether the results are consistent across different models. We chose the following models which are commonly used for tabular data:

- **Linear Regression**
- **Lasso Regression**
- **Random Forest**
- **AdaBoost**

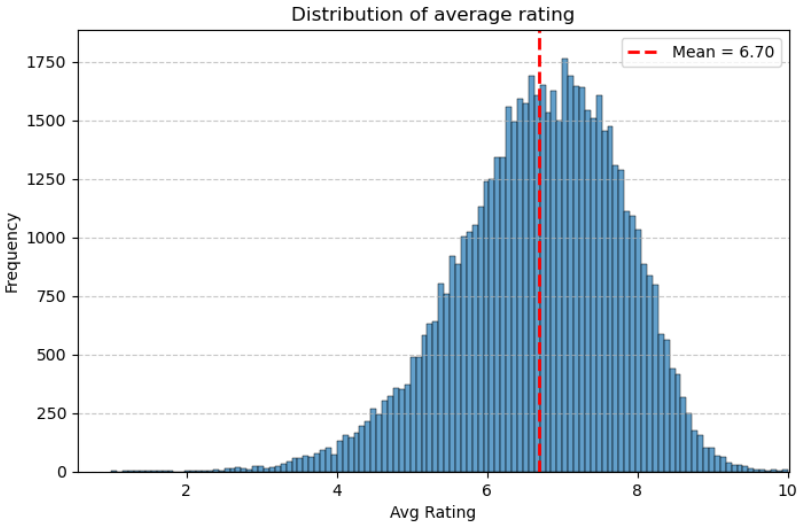


Figure 6: Empirical distribution of the label `avg_rating` in the board games dataset.

- **XGBoost**
- **NN1** (Shallow Neural Network)
- **NN3** (Neural Network with two hidden layers and one dropout layer for regularization)

We also have a very simple baseline model that always predicts the mean of the training labels, called **Dummy Regression**. The performance of these models on the test set is shown in Table 1. We can see that the tree-based

Table 1: Performance of the trained regression models on the test set

Model	RMSE	MAE	R^2
Dummy Regression	1.088	0.871	0.000
Lasso Regression	0.776	0.584	0.491
Linear Regression	0.766	0.576	0.505
NN1	0.706	0.522	0.578
NN3	0.689	0.514	0.599
AdaBoost	0.659	0.477	0.633
Random Forest	0.644	0.462	0.649
XGBoost	0.628	0.452	0.667

models (Random Forest, AdaBoost and XGBoost) perform best, followed by

the neural networks and finally the linear models. All models perform better than the Dummy Regression baseline which shows that there is indeed learnable structure in the data. We will now apply some of the interpretability methods discussed in this thesis to try to understand what structures the models have learned.

5.1 Global Analysis

In this section we will first analyse the trained models from Table 1 using permutation feature importance (PFI). Recall from Section 4.3 that PFI measures the decrease in predictive performance when the values of a feature are randomly permuted, thereby breaking its association with the target while keeping all other features fixed.

For each model and each feature i we compute the PFI on the test set as in Equation (30), where the chosen loss ℓ is the MSE on the test set and we average over $m = 10$ permutations. The tables with the top 20 features for each model can be found in Appendix C.1, while the full tables can be found in the repository [4]. We also calculate the standard deviation across permutations and a normed PFI by dividing by the sum of PFI over all features. This allows better comparison between models with different overall error levels.

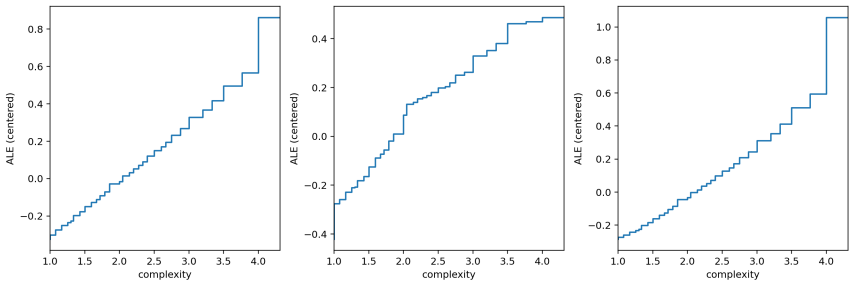
Across all models a very consistent picture emerges. The three most influential features with respect to the prediction of `avg_rating` are the publication year `year`, the indicator `category_Expansion for Base-game`, and the complexity measure `complexity`. These features appear among the top twenty PFI scores for every model and typically account for a large share of the total (normalised) importance.

Other features that are present in the top twenty across all models includes the indicators `category_Wargame`, `mechanic_Roll_/_Spin_and_Move`, `category_Miniatures`, `mechanic_Dice_Rolling`, `mechanic_Cooperative_Game` and basic constraints such as `min_age`, `max_time` and `min_players`. Overall, this suggests that the rating of a game is largely determined by when it was published, whether it is an expansion or a base game, how complex it is, and which broad category and mechanics it belongs to.

It is interesting that the linear models rank `year` much lower (rank 17 and 20). A possible explanation might be that its effect on the label is nonlinear or that it is correlated with other features. We will investigate this further using ALE plots.

The full set of ALE plots for all models is provided in Appendix C (Figures 22 to 28). Here we will discuss the most consistent (dis)agreements across models.

For numerical features, the ALE is evaluated on a discretised grid. The resulting curve is therefore piecewise constant, with steps at bin boundaries. The grid was chosen such that each bin contains a similar number of samples. This is why the bin width varies substantially and explains why the `year` plots typically show narrower bins for recent years: the data is more concentrated there, yielding smaller intervals on the x-axis even though each bin contains a similar number of samples.



(a) Linear Regression (b) Random Forest (c) NN1

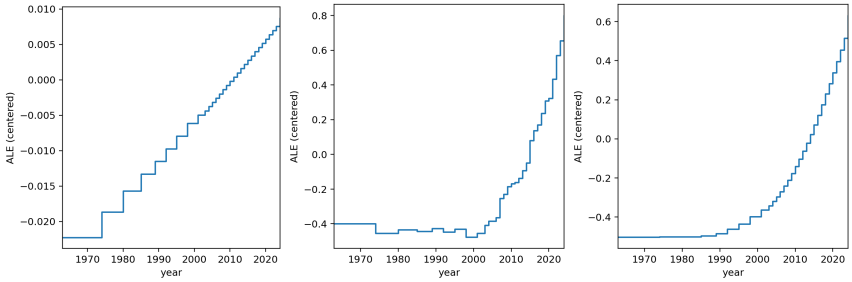
Figure 7: Comparison of ALE plots for `complexity`. All models suggest a strong linear relationship.

Across all model classes, two numerical features exhibit a remarkably stable pattern: (i) `year` has a positive effect on the predicted rating, and (ii) `complexity` is monotonically increasing, indicating that more complex games tend to receive higher predicted ratings. This agreement supports the PFI results and suggests that these two variables encode robust, model-invariant signals in the data.

For binary indicators, the sign of the effect is also highly consistent across models. In particular, `category_Expansion for Base-game` shows a strong positive shift in predicted rating, while `mechanic_Roll_/_Spin_and_Move` shows a consistent negative shift. Further, `category_Wargame`, `category_Miniatures`, and `mechanic_Cooperative_Game` tend to have moderately positive effects, whereas `mechanic_Dice_Rolling` is close to zero in all models. This suggests that some broad category/mechanic signals are captured similarly regardless of model. One explanation for the high impact of `category_Expansion for Base-game` is that expansions are typically made for games that are already popular and well-rated. This is consistent with a selection bias. On the other hand, Roll/Spin and Move mechanics are commonly associated with low-strategy, high-luck designs often aimed at younger audiences, which may coincide with lower average ratings in this dataset.

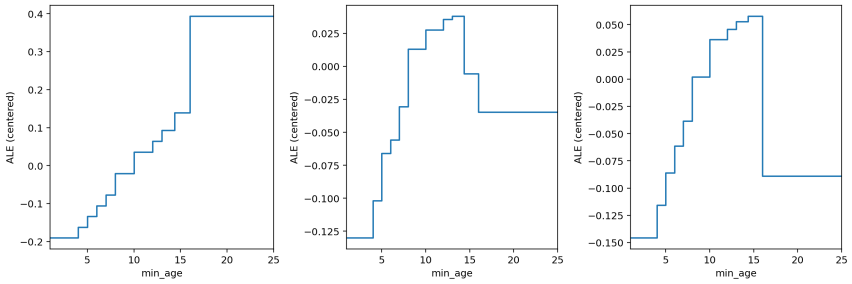
For the other features we can see substantial differences in the form and magnitude of the ALE plot. This is particularly evident for `year`: while Linear Regression and Lasso assign a relatively small, approximately linear increase over time, tree-based ensembles and neural networks show a pronounced acceleration in later years. A representative comparison is shown in Figure 8. This provides a plausible explanation for the lower PFI rank of `year` in the linear models: when the true relationship is strongly non-linear (and potentially correlated with other attributes), a single global slope can underrepresent its contribution and redistribute importance to correlated covariates.

A second notable divergence concerns `min_age`. Linear models suggest an almost monotone increase, whereas most non-linear models exhibit an inverted U-shape. Predicted ratings increase from very low minimum ages to roughly the



(a) Linear Regression (b) XGBoost (c) NN3

Figure 8: Comparison of ALE plots for `year`. Linear models reflect a small near-linear trend, whereas non-linear models capture a strong, non-linear increase in recent decades.



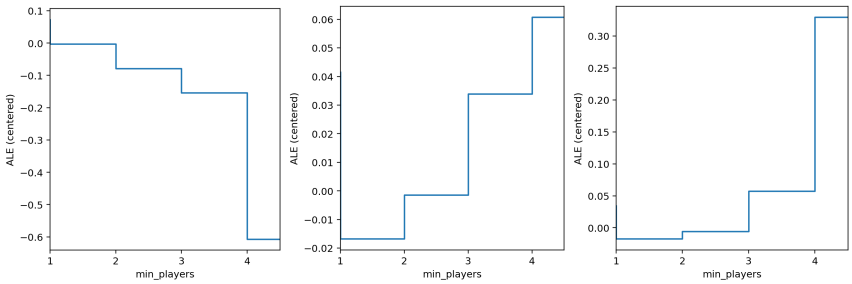
(a) Lasso Regression (b) AdaBoost (c) NN3

Figure 9: Comparison of ALE plots for `min_age`. Linear models reflect a small near-linear trend, whereas non-linear models capture an incline and decline towards teenage years.

teenage range and then flatten or decrease for very high age requirements. We can see this in figure 9. This might correlate with complexity, as more complex games tend to have a higher minimum age stamp. Very high `min_age` values are rare and may represent a distinct subset; the decline in the tail could reflect either a genuine preference pattern in the data or higher estimation noise in sparse regions.

Similarly, `max_time` is close to flat in the linear models but shows a positive trend in the tree-based ensembles and the deeper network suggesting diminishing returns: increases in play time correlate with higher predicted ratings up to a point, beyond which additional time adds little. A possible interpretation is that `max_time` shares variance with `complexity`; linear models may attribute most of the shared signal to `complexity`, while non-linear models can still recover a residual, non-linear time effect.

The largest qualitative discrepancy is found for `min_players`. Linear Regression and Lasso show a strong negative effect as the minimum required player



(a) Lasso Regression (b) AdaBoost (c) NN3

Figure 10: Comparison of ALE plots for `min_players`. Linear models imply a negative trend, while non-linear models suggest a high effect for one or 5+ players.

count increases. In contrast, non-linear models show a pronounced jump at `min_players`=5. Figure 10 illustrates this disagreement. A possible explanation for this is that there are relatively few games for minimum 5+ players, meaning the tail of the distribution is noisy.

Taken together, the ALE analysis complements the PFI results by revealing that the most influential variables have consistent global importance and some of them have non-linear impact. These non-linear patterns are consistent with the improved predictive performance of non-linear models in Table 1. `year` and `complexity` show robust positive effects across all models, while `min_age`, `max_time`, and especially `min_players` highlight meaningful differences between linear and non-linear models. We find that `category_Expansion for Base-game` and `mechanic_Roll_/_Spin_and_Move` have a high positive respectively negative effect on the predicted rating across all models.

To complement the main-effect analysis, we now quantify feature interactions using Friedman's H-statistic introduced in Section 4.4. We compute H_j for the features found in Table 8 as a first screening step, and then compute H_{jk} for selected feature sets to identify which pairs are responsible for the interaction signal. Full H_j tables for all models are provided in Appendix C.3.

As a sanity check, the linear models yield interaction scores essentially equal to zero for all screened features, consistent with their additive structure (Table 11). In contrast, tree ensembles and the deeper neural network produce non-negligible interaction strengths, with AdaBoost exhibiting the largest overall interaction mass, followed by Random Forest and XGBoost. This aligns with the model-class discussion: tree ensembles can represent interactions through sequential splits, whereas the linear models cannot. The feature-wise statistic H_j identifies `year`, `complexity`, and `category_Expansion for Base-game` as the most interaction-relevant variables in the tree ensembles, indicating that the marginal effect of these globally important features depends on other attributes of a game.

To understand which interactions drive this signal, we compute pairwise H_{jk}

statistics. When restricting attention to basic profile variables[9], the strongest pairwise interactions involve combinations such as `min_time` with `max_players` and `min_time` with `min_age`. These patterns are intuitively plausible because such variables encode feasibility constraints and typical design regimes. However, these pairs are also strongly dependent in the data, so the corresponding PDP-based interaction estimates should primarily be interpreted as interaction effects in the fitted model rather than as evidence of a causal interaction in the data-generating process. The full pairwise H_{jk} table for XGBoost on this profile-variable set is given in Table 18.

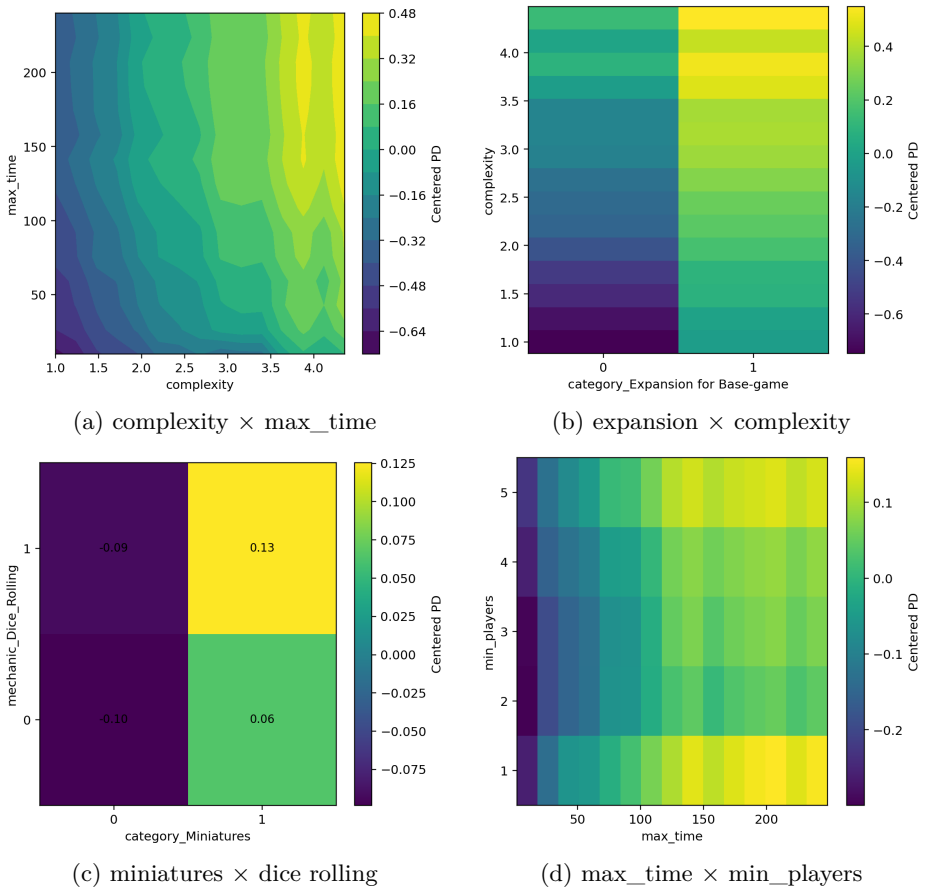
To obtain more domain-interpretable interactions, we therefore compute H_{jk} on a second feature set that combines the most influential numeric variables with representative category and mechanic indicators[10]. For XGBoost, the most relevant and interpretable interaction candidates include `complexity` with `max_time`, `category_Expansion for Base-game` with `complexity`, `category_Miniatures` with `mechanic_Dice_Rolling`, and `max_time` with `min_players`. These interactions are also among the most prominent pairs in Random Forest and AdaBoost, indicating that they are robust across tree-based model classes. Tables 19 and 20 show the full pairwise H_{jk} for XGBoost on this feature set.

Figure 11 visualizes selected interaction patterns using two-dimensional partial dependence plots for XGBoost. The plot for `complexity` and `max_time` shows that both variables have a positive main effect, but the effect of complexity depends on the time regime: high complexity is predicted particularly favorably for longer games, whereas short games with low complexity are predicted substantially below average. The interaction between `category_Expansion for Base-game` and `complexity` indicates that expansions receive an overall positive shift in predicted rating, but also that the dependence on complexity differs between base games and expansions, consistent with expansions often being attached to already successful and well-received products. The interaction between `category_Miniatures` and `mechanic_Dice_Rolling` shows a clear non-additive pattern: dice rolling has little effect when miniatures are absent, but becomes distinctly positive when miniatures are present, suggesting that the model distinguishes between dice rolling as a core mechanic in miniature-heavy designs and dice rolling in other contexts. Finally, the interaction between `max_time` and `min_players` shows that the predicted benefit of long play times varies by the minimum required group size, reinforcing the interpretation that the model learns different quality regimes for games aimed at solo play versus larger groups.

Overall, the H-statistic analysis refines the global picture obtained from PFI and ALE. The dominant predictors (`year`, `complexity`, and `category_Expansion for Base-game`) exhibit strong and largely consistent main effects across model classes. At the same time, the non-linear models are not purely

[9] `year`, `complexity`, `category_Expansion for Base-game`,`min_age`, `min_time`, `max_time`, `min_players`, `max_players`

[10] `year`, `complexity`, `category_Expansion for Base-game`,`max_time`, `min_players`, `category_Wargame`, `category_Miniatures`,`mechanic_Cooperative_Game`, `mechanic_Dice_Rolling`, `mechanic_Roll_/_Spin_and_Move`



(a) complexity × max_time

(b) expansion × complexity

(c) miniatures × dice rolling

(d) max_time × min_players

Figure 11: Two-dimensional partial dependence plots for selected interaction pairs in XGBoost. The values shown are centered partial dependence estimates like in Equation (36).

additive: the relatively large H_j scores indicate that the contribution of these features depends on the remaining feature set. Importantly, a high H_j does not imply the existence of a single dominant pairwise interaction (j, k). Rather, the interaction mass can be distributed across many weak-to-moderate interactions (especially when j is correlated with numerous sparse mechanics/category indicators), so that no individual H_{jk} stands out strongly within the restricted screening sets. The pairwise H_{jk} analysis and the selected 2D-PDPs therefore highlight a small number of interpretable interaction patterns (e.g. `complexity`×`max_time` and `expansion`×`complexity`) while the remaining context-dependence of `year`, `complexity`, and `expansion` is plausibly spread over many smaller effects.

5.2 Local Analysis

The global analysis identified a small set of robust, model-invariant signals: `year`, `complexity`, and `category_Expansion for Base-game` emerged as the most influential variables across all model classes, while `mechanic_Roll_/_Spin_and_Move` showed a consistently negative effect. To complement these aggregate findings, we now study individual test-set predictions using local methods introduced earlier (CP and SHAP). We have two goals: (i) verify whether the global patterns indeed explain concrete predictions for specific games, and (ii) diagnose typical failure modes, where the model extrapolates or systematically over-/underestimates a game's rating.

We focus on XGBoost and NN3 as models because they represent two non-linear model families, and performed best within their model class (see Table 1). This choice provides a useful contrast between a tree ensemble and a neural network while keeping the analysis manageable.

Local explanations are necessarily instance-based. Rather than sampling arbitrarily, we select games from the test set that represent extremes along several interpretable criteria computed on the test set:

- highest/lowest predictions among games with `mechanic_Roll_/_Spin_and_Move`=1

  Thunder Road: Vendetta - Turbo Tina's Swag Pack (high) and How to Be a Complete Bastard (low)

- highest/lowest predicted rating Overall

  Frosthaven (high) and Lotto (low)

- largest positive/negative residuals $(\hat{y} - y)$.

  Karambolage (large underprediction) and Dominion: Fan-Edition I (large overprediction)

- extreme outlier in terms of true rating

  Rock Paper Scissors Game

Table 2: Selected test-set instances for the local analysis. Residuals are defined as $r = \hat{y} - y$ (positive: overprediction, negative: underprediction).

Game	y	$\hat{y}_{\text{XGB}}$	$\hat{y}_{\text{NN3}}$	r_{XGB}	r_{NN3}
Thunder Road: Vendetta – Turbo Tina's Swag Pack	8.300	8.176	7.771	-0.124	-0.529
How to Be a Complete Bastard	4.675	4.011	4.095	-0.664	-0.580
Frosthaven	8.792	8.475	8.176	-0.317	-0.616
Lotto	3.736	4.266	3.807	+0.530	+0.072
Karambolage	6.790	4.723	4.893	-2.067	-1.897
Dominion: Fan-Edition I	4.463	7.095	7.360	+2.631	+2.896
Rock Paper Scissors Game	1.274	5.217	5.273	+3.943	+3.999
Dominion	7.598	7.465	7.297	-0.132	-0.301

We additionally include Dominion as a comparison point for the misestimated fan-edition variant, so that we can compare an expansion to its base game. Table 2 summarises the selected cases.

A useful reference point is the SHAP baseline $\phi_0 = \mathbb{E}[\hat{y}(X)]$. For the fitted models we obtain $\phi_0 \approx 6.67$ (XGBoost) and $\phi_0 \approx 6.52$ (NN3) on the chosen background distribution. Recall from Figure 6 that we already know that $\mathbb{E}[y(X)] = 6.70$. For each model, the explanation can be interpreted as the deviation from this ϕ_0 induced by the feature attributions. We will now take an in-depth look at three representative instances that illustrate main patterns.

Systematic overprediction: Dominion: Fan-Edition I. Both XGBoost and NN3 overpredict the fan edition by roughly 2.6–2.9 rating points (Table 2). The SHAP decompositions in Figures 12 and 13 show that this is not driven by many small effects, but by a small number of large, high-level signals:

- `category_Expansion for Base-game`=1 contributes strongly positively (largest positive attribution in both models).
- `compilation_count`=2 also contributes strongly positively.
- Additional positive contributions come from "high-status" publisher / designer indicators and from deck-building mechanics.

These signals are sufficiently strong that negative contributions from numerical features (e.g. moderate `year`, moderate `complexity`, short `max_time`) do not offset them. This behaviour is consistent with the global finding that expansions have a large positive effect and high importance.

To probe whether this misprediction can be resolved by plausible changes in the most important (by PFI) numerical features, we inspect CP curves for `year` and `complexity` (Figure 14). Even when varying these features across broad but realistic ranges, the predicted rating remains well above the true $y = 4.463$ (minimum CP predictions are still above roughly 6.8 for both models). Hence,

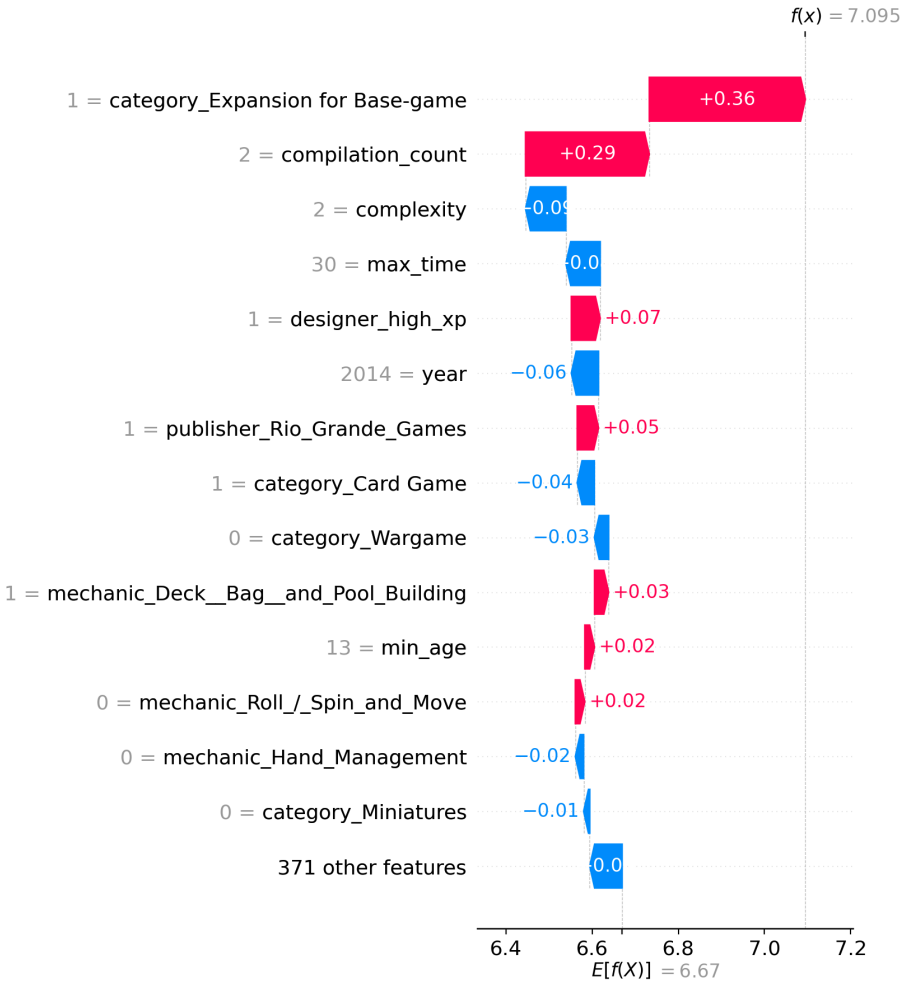


Figure 12: SHAP explanations for Dominion: Fan-Edition I for XGBoost. The value for `complexity` was missing in the data and imputed; SHAP explains the model's prediction for the imputed representation.

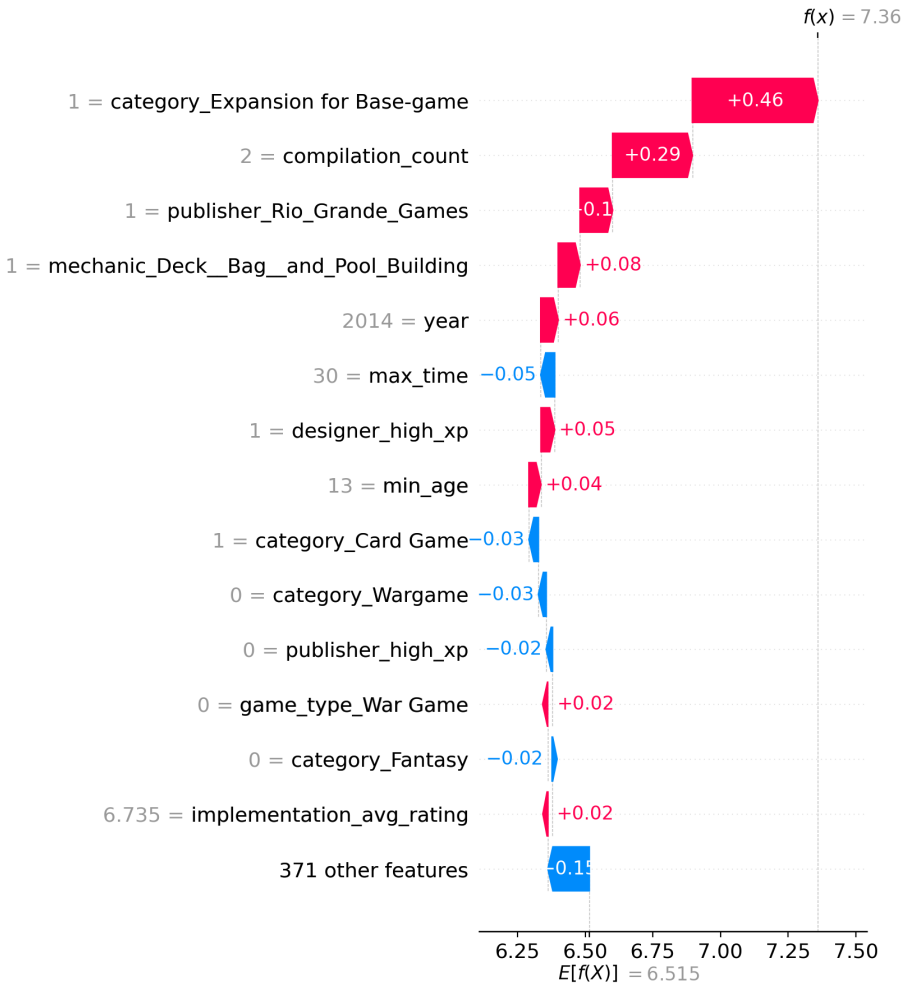


Figure 13: SHAP explanations for Dominion: Fan-Edition I for NN3

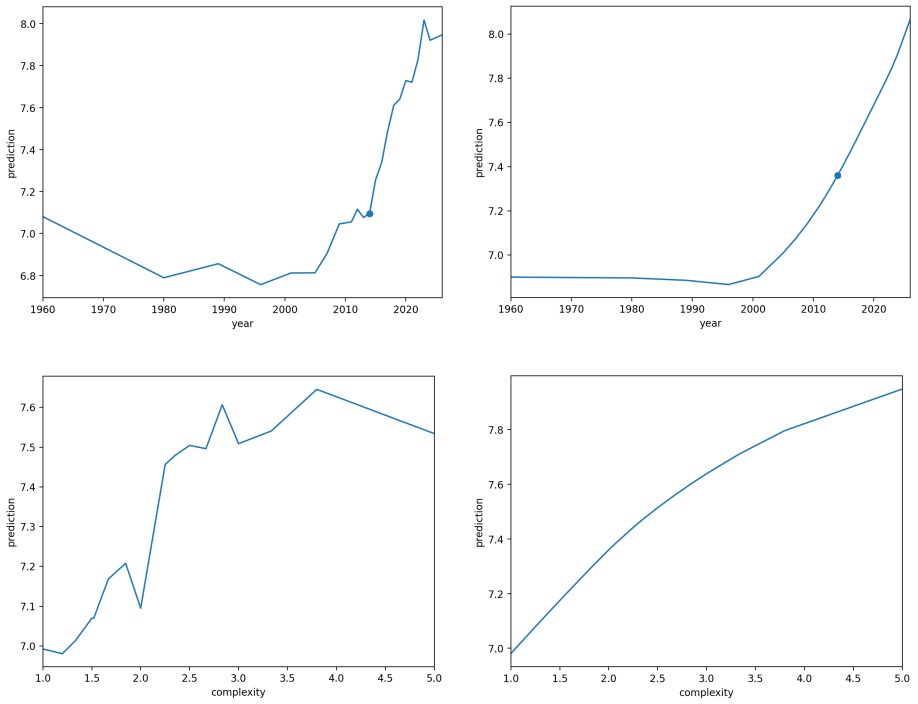


Figure 14: CP curves for `year` and `complexity` for Dominion: Fan-Edition I across XGB (left) and NN3 (right). The blue dot resembles the true value of each feature. The `complexity` value is missing in the data and thus not shown.

the error is structural with respect to the chosen feature set: the models treat expansion + compilation as a strong proxy for high ratings. This does not apply to this specific case however, because the fan edition includes 2 promo cards (which in the data appears as a compilation) but is not a full expansion and does not add gameplay value but rather cosmetic improvements to the game. This might explain the real true rating being much lower than the predicted one.

For the base game Dominion, predictions are close to y (residuals ≈ -0.13 to -0.30), and SHAP assigns meaningful positive mass to deck-building and strategy indicators while the absence of the expansion flag contributes negatively (see Figures 30 and 29 in the Appendix C.4). This supports the interpretation that the fan edition is pushed into a high-rating regime mainly by the `Expansion` and `compilation` signals rather than by deck-building alone.

(2) Systematic underprediction: Karambolage. Karambolage is underpredicted by about 1.9-2.1 points. In Figures 15 and 16, SHAP reveals a coherent mechanism: both models assign large negative contributions to `year` (1976) and to `mechanic_Roll_/_Spin_and_Move`=1, while the absence of `category_Expansion for Base-game` further decreases the prediction. This is a concrete local manifestation of the global pattern that "older" and "roll/spin-and-move" are associated with lower ratings. One problem is that the "roll/spin-and-move" mechanic usually indicates no agency from the player (i.e. roll a 5 and move 5 spaces), whereas in this game the player does indeed roll but then has multiple options on how to move their piece, adding a layer of strategy. This nuance is only slightly captured by the NN3 model by adding positive value to the `category_math` feature and not captured by the XGBoost model and might be a reason for the systematic underprediction of this game.

CP curves show an interesting effect: varying `year` alone yields a large local effect (about 1.8-2.0 points across the grid), moving the prediction substantially closer to the observed rating. In contrast, varying `max_time` has only a moderate effect in the relevant range. This suggests that, for this instance, the underprediction is explained primarily by the roll/spin mechanic and the old publication year.

(3) Extreme outlier: Rock Paper Scissors Game. This game has the lowest true rating among the selected instances ($y = 1.274$), but both models predict around 5.2, i.e. close to the global baseline. The SHAP explanations do identify intuitive negative signals (low `complexity`, relatively old `year`, absence of the expansion flag, and the presence of `mechanic_Rock-Paper-Scissors`). However, the total negative mass is far too small to bridge the gap from ≈ 6.6 down to ≈ 1.3.

CP makes this limitation especially transparent: even when varying the most important numerical drivers (`year` and `complexity`) across wide ranges, the predictions remain bounded well above 5 (the minimum CP predictions are approximately 5.1 for XGBoost and 5.2 for NN3). This suggests that, given the



Figure 15: SHAP explanations for Karambolage for XGBoost.

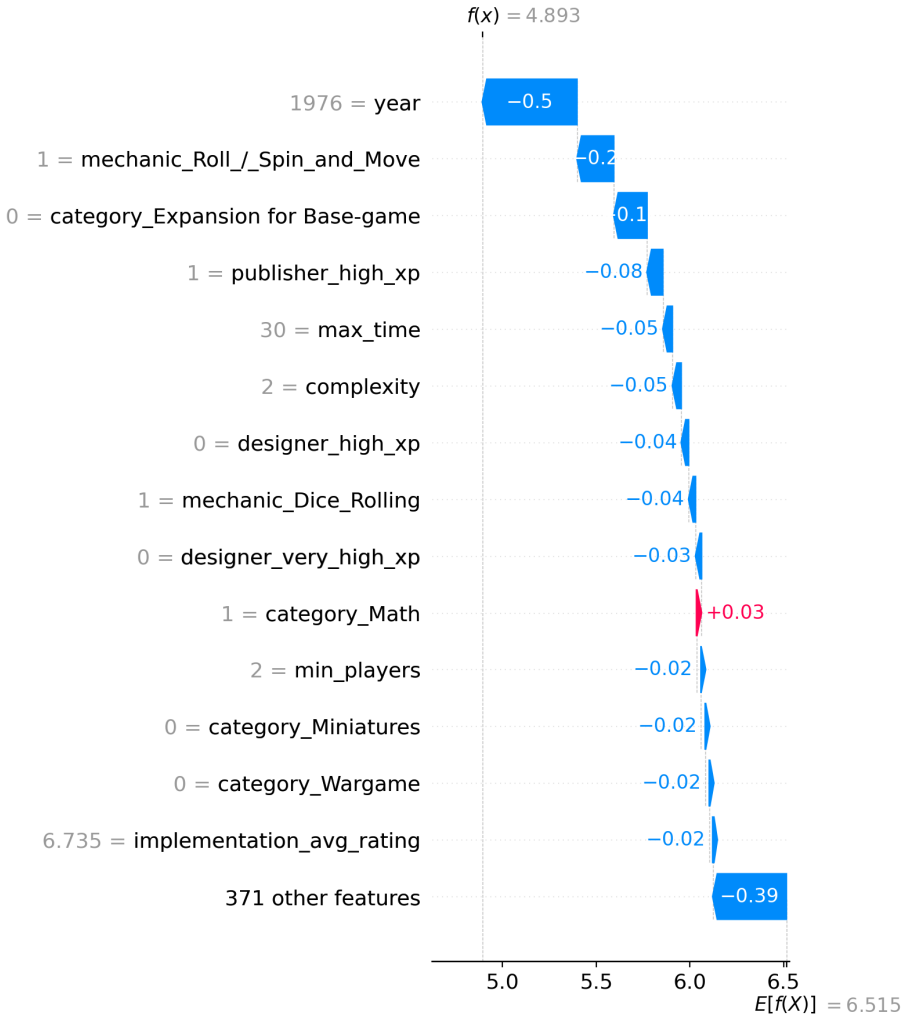


Figure 16: SHAP explanations for Karambolage for NN3

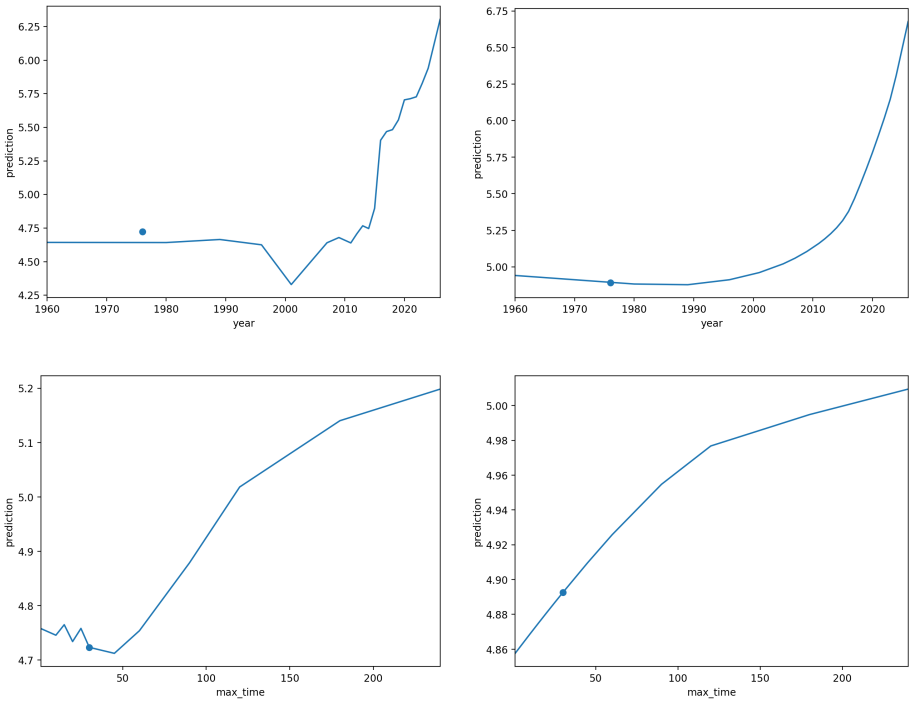


Figure 17: CP curves for `year` and `max_time` for Karambolage across XGB (left) and NN3 (right). The blue dot in the `year` plot for XGB resembles the predicted rating for that year. The CP curve is drawn on a grid of values and then interpolates between them which is why the blue dot is not exactly on the curve.

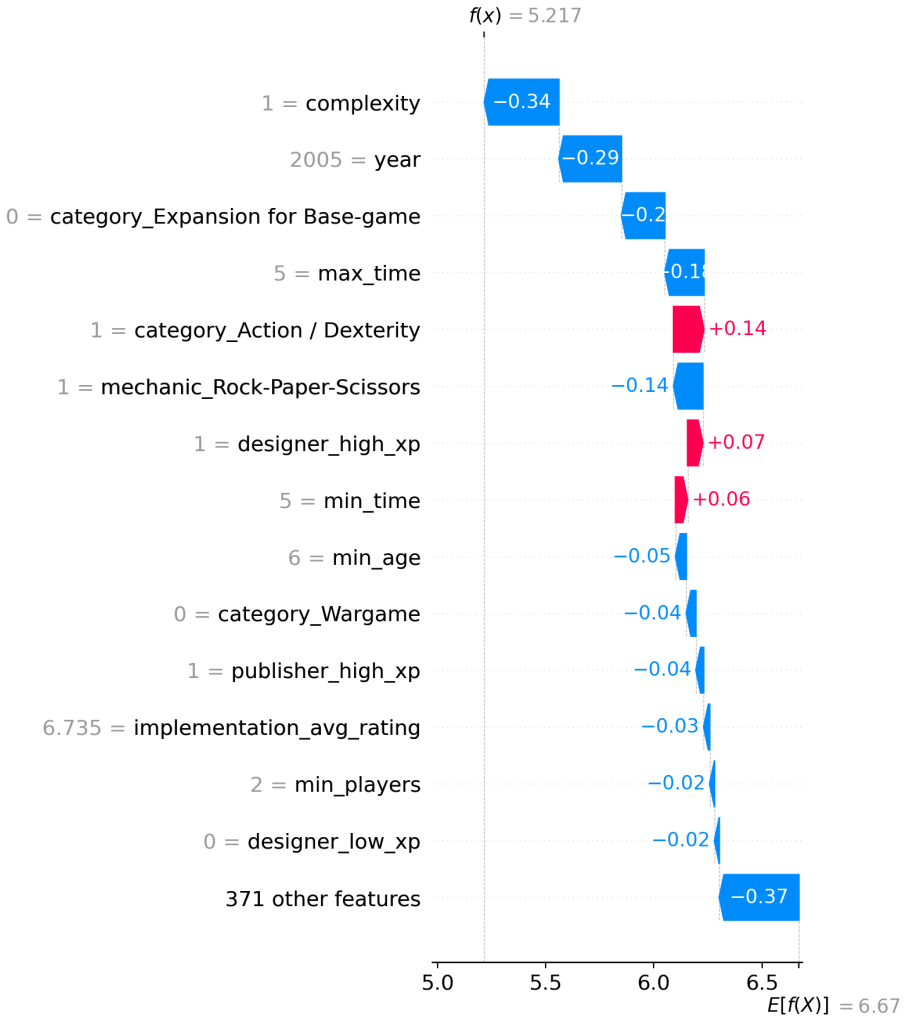


Figure 18: SHAP explanations for Rock Paper Scissors Game for XGBoost

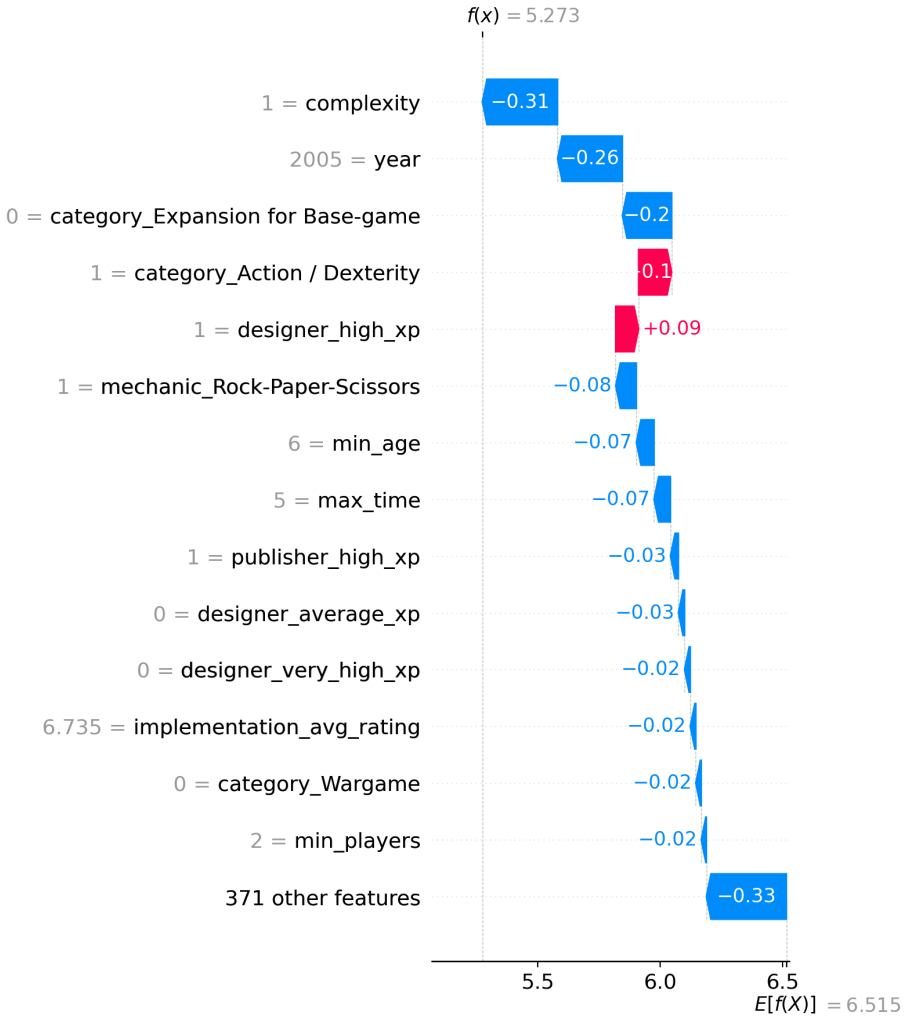


Figure 19: SHAP explanations for Rock Paper Scissors Game for NN3

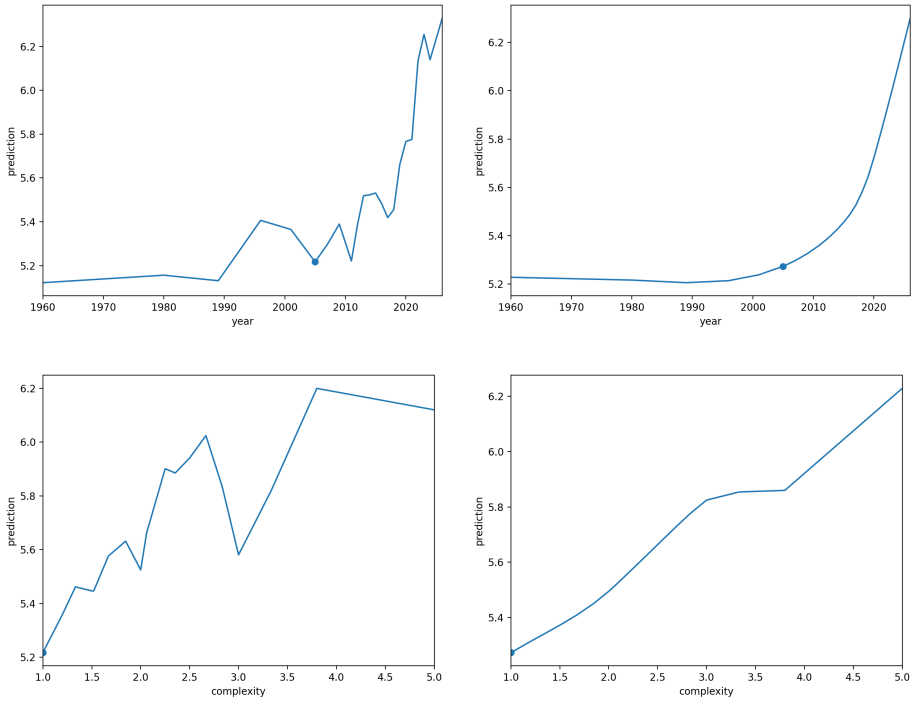


Figure 20: CP curves for `year` and `complexity` for Rock Paper Scissors Game across XGB (left) and NN3 (right).

available features and the training distribution, the models cannot represent the extreme low-rating regime for this instance and instead regress toward a typical score. Put differently, this is a case where local explanations diagnose a representation gap: the observed rating is incompatible with the rating regimes encoded by the features the models rely on.

From user's reviews [34] we can guess why this game is regarded so lowly. It is an implementation of the classic children's game "rock paper scissors" but plays much slower because of physical components and setup time. They also say that the environmental impact of producing plastic components for such a simple game is not justified. These aspects are not captured by the features in the data set and thus cannot be learned by the models.

For the remaining selected instances, the SHAP explanations can be found in Appendix C.4 and the CP plots in Appendix C.5

- **Frosthaven (high prediction).** Both models attribute the high prediction to very large positive effects of `complexity` and `year`, consistent with the global analysis. A notable difference is that XGBoost assigns a substantially larger positive contribution to `max_time` and shows a stronger `max_time` CP slope, whereas NN3's `max_time` CP is comparatively flatter. This aligns with the earlier observation that non-linear models disagree more on time-related features than on `year`/`complexity`.
- **Thunder Road: Vendetta - Turbo Tina's Swag Pack(roll/spin high prediction).** The SHAP explanation illustrates how a negative roll/spin signal can be outweighed by a stack of positive indicators: a very recent `year`, `category_Expansion for Base-game`=1, and signals such as `category_Miniatures` and `family_Crowdfunding`.
- **How to Be a Complete Bastard (roll/spin low prediction).** Here, the low prediction is explained consistently by negative contributions from low `complexity`, older `year`, and `mechanic_Roll_/_Spin_and_Move`=1. This case functions as an example where the global signals yield a plausible local explanation with only moderate residual.
- **Lotto (low prediction).** The local explanation flags an informative data issue: the extremely early `year` value (1530) receives a large negative attribution in both models and largely explains the low prediction. This illustrates how local explanations can also serve as a diagnostic for unusual or potentially noisy entries.

5.3 Discussion and Validity Considerations

Across model classes, the global analyses identify `year`, `complexity`, and `category_-Expansion for Base-game` as dominant drivers with robust positive main effects, while `mechanic_Roll_/_Spin_and_Move` shows a consistently negative association with `avg_rating`. For the best-performing non-linear models, the H-statistics further suggest that these effects are not purely additive; however,

the interaction signal appears to be distributed across many small context effects rather than driven by a single outstanding pairwise interaction.

In line with the bias considerations above, the explanations should be interpreted as describing patterns within the BGG ecosystem rather than population-level preferences. The strong positive role of `complexity` and the negative association for `mechanic_Roll_/_Spin_and_Move` are consistent with an enthusiast-skewed rater community. Moreover, local case analyses illustrate proxy and data-quality effects: Dominion: Fan-Edition I is systematically overpredicted, supporting the interpretation of `category_Expansion for Base-game` as a proxy for prior success, while anomalous metadata (e.g. extreme `year` values) and coarse mechanic indicators can contribute to individual mispredictions. Finally, Rock Paper Scissors Game suggests a representation gap for extremely low-rated titles, indicating that important determinants of very low ratings are not fully captured by the available feature set. Overall, the case study highlights how combining global and local explanations helps distinguish robust associations from artifacts due to selection mechanisms, noisy metadata, and missing features, and the results should be viewed as descriptive rather than causal.

6 Summary

This thesis studied model-agnostic interpretability for supervised regression models. The central theme was to view a trained predictor as a function that can be evaluated on inputs and to develop explanation methods that describe how this function depends on its features. The emphasis was on a mathematical presentation with consistent notation across methods, so that key definitions and properties can be stated and proved within a single framework.

On the local level, Ceteris Paribus and ICE explanations describe sensitivity of predictions by varying a selected feature (or feature subset) while keeping all other features fixed. This provides an instance-based view of how predictions would change under targeted modifications and makes heterogeneity across instances visible. Shapley values provide a complementary local perspective by decomposing a prediction into a baseline plus additive feature contributions. By formulating local feature attribution as a cooperative game, Shapley-based methods inherit a precise axiomatic characterization, and practical estimators arise naturally from the permutation form of the Shapley value together with a coalition evaluation scheme.

On the global level, the thesis considered methods that summarize model behavior across many instances. PDP and ALE were discussed as approaches to visualize feature effects, highlighting both their conceptual differences and their dependence on feature distributions. Permutation-based importance measures were presented as model-agnostic tools to quantify predictive relevance, together with conditional variants designed to reduce artifacts when features are dependent. Finally, feature interaction was addressed by an interaction strength measure that captures departures from additivity and helps identify pairs of features whose joint behavior is important for the model.

In the case study, the methods were applied to predicting board game ratings from a high-dimensional set of engineered metadata features across several different model classes. The global analyses identified stable associations that persist across models, while interaction analyses highlighted where non-linear predictors exploit joint feature structure. The local analyses complemented this by explaining individual predictions, revealing proxy effects and data-quality issues in specific cases, and illustrating how unusual inputs can lead to mispredictions. Taken together, the case study demonstrated that combining global and local model-agnostic explanations yields a more robust understanding than either perspective alone.

References

[1] Takuya Akiba et al. “Optuna: A Next-generation Hyperparameter Optimization Framework”. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2019.

[2] Daniel W. Apley and Jingyu Zhu. “Visualizing the Effects of Predictor Variables in Black Box Supervised Learning Models”. In: *Journal of the Royal Statistical Society Series B: Statistical Methodology* 82.4 (June 2020), pp. 1059–1086. ISSN: 1369-7412. DOI: 10.1111/rssb.12377. eprint: https://academic.oup.com/jrsssb/article-pdf/82/4/1059/49323846/rssb12377-sup-0001-supinfo.pdf. URL: https://doi.org/10.1111/rssb.12377.

[3] Stephen Bates et al. “Metropolized Knockoff Sampling”. In: *Journal of the American Statistical Association* 116.535 (2021), pp. 1413–1427. DOI: 10.1080/01621459.2020.1729163. eprint: https://doi.org/10.1080/01621459.2020.1729163. URL: https://doi.org/10.1080/01621459.2020.1729163.

[4] Moritz Beyer. *Github Repository for Diploma Thesis*. 2025. URL: https://github.com/2Duffman/Diplomarbeit_public.

[5] Ahsan Bilal, David Ebert, and Beiyu Lin. *LLMs for Explainable AI: A Comprehensive Survey*. 2025. arXiv: 2504.00125 [cs.AI]. URL: https://arxiv.org/abs/2504.00125.

[6] Leo Breiman. “Random Forests”. In: *Machine Learning* 45.1 (Oct. 2001), pp. 5–32. ISSN: 1573-0565. DOI: 10.1023/A:1010933404324. URL: https://doi.org/10.1023/A:1010933404324.

[7] Leo Breiman et al. *Classification and Regression Trees*. Belmont, CA: Wadsworth International Group, 1984. ISBN: 978-0-412-04841-8.

[8] Erik Cambria et al. *XAI meets LLMs: A Survey of the Relation between Explainable AI and Large Language Models*. 2024. arXiv: 2407.15248 [cs.CL]. URL: https://arxiv.org/abs/2407.15248.

[9] Emmanuel Candès et al. “Panning for Gold: ‘Model-X’ Knockoffs for High Dimensional Controlled Variable Selection”. In: *Journal of the Royal Statistical Society Series B: Statistical Methodology* 80.3 (Jan. 2018), pp. 551–577. ISSN: 1369-7412. DOI: 10.1111/rssb.12265. eprint: https://academic.oup.com/jrsssb/article-pdf/80/3/551/49274697/rssb12265-sup-0001-suppinfo.pdf. URL: https://doi.org/10.1111/rssb.12265.

[10] Tianqi Chen and Carlos Guestrin. “XGBoost: A Scalable Tree Boosting System”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD ’16. ACM, Aug. 2016, pp. 785–794. DOI: 10.1145/2939672.2939785. URL: http://dx.doi.org/10.1145/2939672.2939785.

[11] Harris Drucker. "Improving Regressors Using Boosting Techniques". In: *Proceedings of the 14th International Conference on Machine Learning* (Aug. 1997).

[12] Aaron Fisher, Cynthia Rudin, and Francesca Dominici. "All Models are Wrong, but Many are Useful: Learning a Variable's Importance by Studying an Entire Class of Prediction Models Simultaneously". In: *Journal of machine learning research : JMLR* 20 (Jan. 2019).

[13] Yoav Freund and Robert E Schapire. "A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting". In: *Journal of Computer and System Sciences* 55.1 (1997), pp. 119–139. ISSN: 0022-0000. DOI: https://doi.org/10.1006/jcss.1997.1504. URL: https://www.sciencedirect.com/science/article/pii/S002200009791504X.

[14] Jerome Friedman and Bogdan Popescu. "Predictive Learning via Rule Ensembles". In: *The Annals of Applied Statistics* 2 (Dec. 2008). DOI: 10.1214/07-AOAS148.

[15] Jerome H. Friedman. "Greedy Function Approximation: A Gradient Boosting Machine". In: *The Annals of Statistics* 29.5 (2001), pp. 1189–1232. ISSN: 00905364, 21688966. URL: http://www.jstor.org/stable/2699986 (visited on 01/09/2025).

[16] Jerome H. Friedman. "Multivariate Adaptive Regression Splines". In: *The Annals of Statistics* 19.1 (1991), pp. 1–67. DOI: 10.1214/aos/1176347963.

[17] Alex Goldstein et al. "Peeking Inside the Black Box: Visualizing Statistical Learning With Plots of Individual Conditional Expectation". In: *Journal of Computational and Graphical Statistics* 24.1 (2015), pp. 44–65. DOI: 10.1080/10618600.2014.907095. eprint: https://doi.org/10.1080/10618600.2014.907095. URL: https://doi.org/10.1080/10618600.2014.907095.

[18] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL: http://www.deeplearningbook.org.

[19] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer Series in Statistics. New York, NY, USA: Springer New York Inc., 2001.

[20] Giles Hooker, Lucas Mentch, and Siyu Zhou. "Unrestricted permutation forces extrapolation: variable importance requires at least one more model, or there is no free variable importance". In: *Statistics and Computing* 31.6 (Oct. 2021), p. 82. ISSN: 1573-1375. DOI: 10.1007/s11222-021-10057-z. URL: https://doi.org/10.1007/s11222-021-10057-z.

[21] Torsten Hothorn and Achim Zeileis. "Predictive Distribution Modeling Using Transformation Forests". In: *Journal of Computational and Graphical Statistics* 30.4 (2021), pp. 1181–1196. DOI: 10.1080/10618600.2021.1872581. eprint: https://doi.org/10.1080/10618600.2021.1872581. URL: https://doi.org/10.1080/10618600.2021.1872581.

[22] Amir-Hossein Karimi et al. "Algorithmic recourse under imperfect causal knowledge: a probabilistic approach". In: *Advances in Neural Information Processing Systems*. Ed. by H. Larochelle et al. Vol. 33. Curran Associates, Inc., 2020, pp. 265–277. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/02a3c7fb3f489288ae6942498498db20-Paper.pdf.

[23] Michał Kuźba, Ewa Baranowska, and Przemyslaw Biecek. "pyCeterisParibus: explaining Machine Learning models with Ceteris Paribus Profiles in Python". In: *Journal of Open Source Software* 4 (May 2019), p. 1389. DOI: 10.21105/joss.01389.

[24] Jing Lei et al. "Distribution-Free Predictive Inference for Regression". In: *Journal of the American Statistical Association* 113.523 (2018), pp. 1094–1111. DOI: 10.1080/01621459.2017.1307116. eprint: https://doi.org/10.1080/01621459.2017.1307116. URL: https://doi.org/10.1080/01621459.2017.1307116.

[25] Zixuan Liang. *Efficient Representations for High-Cardinality Categorical Variables in Machine Learning*. 2025. arXiv: 2501.05646 [cs.LG]. URL: https://arxiv.org/abs/2501.05646.

[26] Scott M. Lundberg, Gabriel G. Erion, and Su-In Lee. *Consistent Individualized Feature Attribution for Tree Ensembles*. 2019. arXiv: 1802.03888 [cs.LG]. URL: https://arxiv.org/abs/1802.03888.

[27] Scott M. Lundberg and Su-In Lee. "A unified approach to interpreting model predictions". In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. NIPS'17. Long Beach, California, USA: Curran Associates Inc., 2017, pp. 4768–4777. ISBN: 9781510860964.

[28] Christoph Molnar. *Interpretable Machine Learning. A Guide for Making Black Box Models Explainable*. 2nd ed. 2022. URL: https://christophm.github.io/interpretable-ml-book.

[29] Christoph Molnar et al. "Model-agnostic feature importance and effects with dependent features: a conditional subgroup approach". In: *Data Mining and Knowledge Discovery* 38.5 (Sept. 2024), pp. 2903–2941. ISSN: 1573-756X. DOI: 10.1007/s10618-022-00901-9. URL: https://doi.org/10.1007/s10618-022-00901-9.

[30] Anirban Mukherjee and Hannah Hanwen Chang. *CAVIAR: Categorical-Variable Embeddings for Accurate and Robust Inference*. 2024. arXiv: 2404.04979 [econ.EM]. URL: https://arxiv.org/abs/2404.04979.

[31] Konstantinos Poulinakis et al. "Machine-Learning Methods on Noisy and Sparse Data". In: *Mathematics* 11.1 (2023). ISSN: 2227-7390. DOI: 10.3390/math11010236. URL: https://www.mdpi.com/2227-7390/11/1/236.

[32] J. R. Quinlan. "Induction of decision trees". In: *Machine Learning* 1.1 (Mar. 1986), pp. 81–106. ISSN: 1573-0565. DOI: 10.1007/BF00116251. URL: https://doi.org/10.1007/BF00116251.

[33] J. Ross Quinlan. *C4.5: Programs for Machine Learning*. San Mateo, CA: Morgan Kaufmann, 1993. ISBN: 1-55860-238-0.

[34] *Review page of Rock Paper Scissors Game*. URL: https://boardgamegeek.com/boardgame/19377/rock-paper-scissors-game/ratings?comment=1 (visited on 12/14/2025).

[35] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. ""Why Should I Trust You?": Explaining the Predictions of Any Classifier". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '16. San Francisco, California, USA: Association for Computing Machinery, 2016, pp. 1135–1144. ISBN: 9781450342322. DOI: 10.1145/2939672.2939778. URL: https://doi.org/10.1145/2939672.2939778.

[36] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "Anchors: High-Precision Model-Agnostic Explanations". In: *Proceedings of the AAAI Conference on Artificial Intelligence* 32.1 (Apr. 2018). DOI: 10.1609/aaai.v32i1.11491. URL: https://ojs.aaai.org/index.php/AAAI/article/view/11491.

[37] Yaniv Romano, Matteo Sesia, and Emmanuel Candès. "Deep Knockoffs". In: *Journal of the American Statistical Association* 115.532 (2020), pp. 1861–1872. DOI: 10.1080/01621459.2019.1660174. eprint: https://doi.org/10.1080/01621459.2019.1660174. URL: https://doi.org/10.1080/01621459.2019.1660174.

[38] M Sesia, C Sabatti, and E J Candès. "Gene hunting with hidden Markov model knockoffs". In: *Biometrika* 106.1 (Aug. 2018), pp. 1–18. ISSN: 0006-3444. DOI: 10.1093/biomet/asy033. eprint: https://academic.oup.com/biomet/article-pdf/106/1/1/27774559/asy033.pdf. URL: https://doi.org/10.1093/biomet/asy033.

[39] Lloyd S Shapley. "A Value for n-Person Games". In: *Contributions to the Theory of Games II*. Ed. by Harold W. Kuhn and Albert W. Tucker. Princeton: Princeton University Press, 1953, pp. 307–317.

[40] Markus Shepherd. *Github Repository for Scraper for BoardGameGeek*. Accessed: 2025-02-01. 2025. URL: https://github.com/recommend-games/board-game-scraper.

[41] D. Shrestha and D. Solomatine. "Experiments with AdaBoost.RT, an Improved Boosting Scheme for Regression". In: *Neural Computation* 18 (2006), pp. 1678–1710. DOI: 10.1162/neco.2006.18.7.1678.

[42] Asher Spector and Lucas Janson. "Powerful knockoffs via minimizing reconstructability". In: *The Annals of Statistics* 50.1 (2022), pp. 252–276. DOI: 10.1214/21-AOS2104. URL: https://doi.org/10.1214/21-AOS2104.

[43] Carolin Strobl et al. “Conditional variable importance for random forests”. In: *BMC Bioinformatics* 9.1 (July 2008), p. 307. ISSN: 1471-2105. DOI: 10.1186/1471-2105-9-307. URL: https://doi.org/10.1186/1471-2105-9-307.

[44] Erik Štrumbelj and Igor Kononenko. “An Efficient Explanation of Individual Classifications using Game Theory”. In: *Journal of Machine Learning Research* 11.1 (2010), pp. 1–18. URL: http://jmlr.org/papers/v11/strumbelj10a.html.

[45] Sandra Wachter, Brent Mittelstadt, and Chris Russell. “Counterfactual Explanations without Opening the Black Box: Automated Decisions and the GDPR”. In: *Harvard Journal of Law & Technology* 31.2 (2018), pp. 841–887. URL: https://arxiv.org/abs/1711.00399.

[46] Jen Wadkins. *Board Games Database from BoardGameGeek*. Accessed: 2025-01-09. 2022. URL: https://www.kaggle.com/datasets/threnjen/board-games-database-from-boardgamegeek.

[47] David S. Watson and Marvin N. Wright. “Testing conditional independence in supervised learning algorithms”. In: *Machine Learning* 110.8 (2021), pp. 2107–2129. ISSN: 1573-0565. DOI: 10.1007/s10994-021-06030-6. URL: https://doi.org/10.1007/s10994-021-06030-6.

[48] Pengbo Zhang and Zhixin Yang. “A Novel AdaBoost Framework With Robust Threshold and Structural Optimization”. In: *IEEE Transactions on Cybernetics* 48 (2018), pp. 64–76. DOI: 10.1109/TCYB.2016.2623900.

List of Appendices

A Feature Engineering **67**

B Model Training **69**
B.1 Preprocessing 69
B.2 Linear Regression (Ordinary Least Squares Regression) 69
B.3 Lasso Regression 69
B.4 Random Forest Regressor 70
B.5 Boosted Decision Tree (Adaboost) 71
B.6 Extreme Gradient Boosting (XGBoost) 72
B.7 Neural Networks (Feed-forward Regression) 73
B.8 Shallow Neural Network (one hidden layer) 74
B.9 Neural Network with two hidden layers (MLP) 74

C Case Study **75**
C.1 Permutation Feature Importance Tables 75
C.2 ALE Plots 83
C.3 Friedman's H-Statistic Tables 91
C.4 SHAP 102
C.5 CP Plots 113

D Hinweis zur Verwendung von KI-Tools **124**

A Feature Engineering

To begin with, we had to find a dataset suitable for the case study. We wanted one where we have a deeper understanding of the data and the features. This is why we chose the topic of board games. We have been playing board games for a long time and are familiar with the different attributes used to describe them, such as the number of players, the playing time, the complexity, or the theme. We also know that there are many different types of board games, such as card games, strategy games, or party games. This makes board games an interesting application domain for interpretability methods.

We initially found a dataset on Kaggle [46] containing information scraped from the BoardGameGeek website. The dataset included attributes such as the name, year of publication, player counts, playing time, complexity, and thematic information, and we decided to use it as a starting point.

However, the dataset was not fully up to date and was missing several features that we considered relevant for the machine learning tasks. For this reason, we decided to scrape the data ourselves. We used and modified the scraper provided by Markus Shepherd [40] to crawl BoardGameGeek. The modified code can be found in our GitHub repository [4]. We scraped all games with at least 10 user ratings, since we expected that this would reduce extreme outliers driven by very small sample sizes. This resulted in a dataset of 61,530 games.

At this point, the dataset contained 64 features. Many of these were irrelevant for prediction (e.g. URLs) and were removed. Others were high-cardinality categorical variables (e.g. designer names) or contained multiple categories per game (e.g. mechanics). We therefore put substantial effort into finding suitable encodings. In general, we aimed to create binary indicator features that occur in at least 300 games (approximately 0.5% of the dataset). This threshold was chosen so that the resulting indicators are not excessively sparse, which can be problematic for many learning algorithms [31]. In practice, this often required merging semantically related categories to meet the threshold, or dropping them entirely.

One of the biggest challenges was the `family` column. It contained heterogeneous information. For example, for the well-known game Monopoly it looks like this:

> ['Game: Monopoly (Official):62857', 'Series: Nostalgia (Hasbro):10149', 'Brands: Groceries:62234', 'Cities: Atlantic City (New Jersey, USA):64788', 'Cities: Reykjavík (Iceland):67865', 'Components: 11 x 11 Grids:78565', 'Country: Poland:10317', 'Country: Spain:10310', 'Country: Sweden:10313', 'Country: Switzerland:10311', 'Mechanic: Circuit-Based Trading Games:79835', 'Misc: Dice Tower Hall of Fame:105030', 'Misc: Watch It Played How To Videos:78198', 'Promotional: Board Games:11505', 'Series: De Eftel-ing:36967', 'Series: Monopoly Derivatives (Official):4462', 'Series: Monopoly Villes et Régions:65824', 'Series: Monopoly-Like:100', 'States: New Jer-sey:43662', 'Theme: Cereal Games:28535']

This column contains several different keywords (e.g. `Cities`, `Components`) together with a finer description for each keyword. Some keywords such as `Game`

were judged irrelevant and were removed completely. For other keywords, such as `Cities`, we kept the keyword but removed the finer description because each city appeared far less than 300 times. For other keywords, such as `Components`, we could create new features from the finer descriptions (e.g. grid sizes).

The following keywords were removed: States, Game, Webcomics, Interwar period (Nov. 1918 – Aug. 1939), Fictional Events, Traditional Games, Series, War Level of Command, Word Games, Comic Strips, Celebrities, Containers, Mountains, Authors, Cities, Games, War Battlespace, Music, Neon Comet Games map sets, Contests, Promotional, Occupation, Rivers, Versions & Editions, Setting, Admin, Trivia, Construction, Mechanic, Characters, Magazine, Digital Implementations, Islands, Traditional Dice Games, Mountain, 1900 (Looping Games), Magenta (CMYK), Organizations, Constructions.

The following keywords were turned into their own binary columns (without using the finer descriptions): Crowdfunding, Movies, Political, Traditional Card Games, Holidays, Historical Figures, Space, Sports Teams, Sports, Food & Drink, Video Game Theme, TV Shows, Medical, Region, Toys, Comic Books, Mythology, Folk Tales & Fairy Tales, Books, Religious, Ancient, Collectible.

Finally, for the following keywords we created subcategories from the finer descriptions whenever applicable: Card Games, Creatures, Components, Category, Animals, Continents, Country, Decades, Players, Brands, Mechanism, History, Misc, Theme.

For `game_type` and `category`, we were able to keep all keywords and turn them into binary features. For `mechanic`, we kept all keywords but had to bundle some of them to reach the threshold. For `designer` and `publisher`, we only kept individual designers with more than 500 games and publishers with more than 600 games in the dataset. The remainder were bundled into four experience categories (very high experience, high experience, average experience, low experience). The cutoffs for designers were 100, 10, and 3; for publishers they were 300, 50, and 3.

Next, there was an `implementation` column that contained names of games implemented within the game. We transformed this column into a numerical feature by computing the mean `avg_rating` of the referenced implemented games. This representation retains information that is relevant for prediction while remaining compatible with standard ML models.

Finally, there were `min_players` and `min_players_rec` columns. The first contains the minimum number of players stated by the publisher, while the second contains the minimum number recommended by BoardGameGeek users. In many cases these coincide, but the disagreement is potentially informative. We therefore removed `min_players_rec` and added `min_players_rec_diff`, which is 0 if there is no difference and otherwise equals `min_players_rec - min_players`. We did not take the absolute value, because the direction of the difference may matter. We applied the analogous transformation to `max_players` and `max_players_rec`.

After these steps, we performed a variance inflation factor (VIF) analysis to identify collinearity among predictors. It turned out that `cooperative` and `mechanic_cooperative_game` are duplicates, so we removed `cooperative`.

B Model Training

In this section some choices regarding the model training will be explained. All code used in training and data processing can be found in the repository [4]. Hyperparameters were chosen with an Optuna study [1]. Finally, each model was trained on the training split and evaluated on the holdout test split. The evaluation metrics are found in Table 1 in Chapter 5.

B.1 Preprocessing

A single 80/20 train-test split was frozen a priori to avoid information leakage from the test set into the training data. Then, a pipeline was used to impute missing values for the numeric features by the median. For the Linear and Neural Network models, those features were then standardised to have zero mean and unit variance. The pipeline was fitted only on the training data to avoid information leakage.

B.2 Linear Regression (Ordinary Least Squares Regression)

The baseline model fits the ordinary least squares estimator

$$\hat{f}(\mathbf{x}) = \mathbf{x}^\top \hat{\beta}, \text{ where } \hat{\beta} \;=\; \underset{\beta \in \mathbb{R}^p}{\arg\min} \, \|\mathbf{y} - \mathbf{X}\beta\|_2^2.$$

Interaction terms were not included: experiments showed a negligible reduction in cross-validated MSE, while they complicate the interpretation of individual coefficients. Retaining the strictly additive structure keeps this model a transparent reference point for the more expressive tree and neural architectures. No hyper-parameters require tuning, so the linear regression serves purely as a benchmark for gauging the benefit of regularisation and non-linear function classes introduced later.

B.3 Lasso Regression

The least absolute shrinkage and selection operator (Lasso) solves

$$\hat{\beta}(\alpha) \;=\; \underset{\beta \in \mathbb{R}^p}{\arg\min} \; \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \alpha \|\beta\|_1,$$

where $\alpha > 0$ trades prediction error against coefficient sparsity. A grid of 50 log-spaced values for α was created in the range $[1.2 \cdot 10^{-5}, 1]$. Each α_k was then evaluated by 5-fold cross-validation on the training split while recording the mean squared error $\mathrm{MSE}(\alpha_k)$ and the number of non-zero coefficients $s(\alpha_k) = |\{j : \hat{\beta}_j(\alpha_k) \neq 0\}|$.

Let $\alpha^\star$ minimise the cross-validated MSE and let $\sigma^\star$ be the corresponding standard error across the 5 folds. The *one-standard-error rule* (1-SE) selects

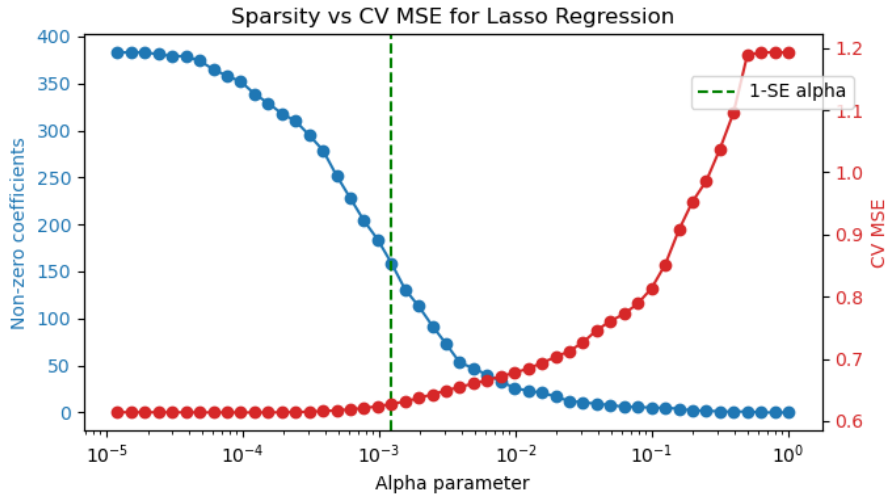


Figure 21: Number of non-zero coefficients (left axis, blue) and cross–validated MSE (right axis, red dashed) as a function of the regularisation strength α. The vertical line marks the selected $\alpha_{\text{1-SE}}$

the largest α satisfying

$$\text{MSE}(\alpha) \;\leq\; \text{MSE}(\alpha^\star) + \sigma^\star.$$

According to Hastie, Tibshirani, and Friedman [19, p.244], choosing α via the 1-SE rule produces a sparse model while incurring only a minor loss in predictive accuracy. Figure 21 shows that the rule selects $\alpha_{\text{1-SE}} = 0.0012$, resulting in 159 non-zero coefficients (compared with 352 at $\alpha^\star = 0.0001$) and raising the cross-validated MSE by merely 0.0135.

B.4 Random Forest Regressor

Definition B.4.1 (Random Forest Regressor, after Breiman [6])**.** A Random Forest for regression is a model consisting of a collection of tree-structured predictors $\{h(\mathbf{x}, \Theta_k), \quad k = 1, \ldots, K\}$, where the Θ_k are independent, identically distributed random vectors. The forest prediction at an input $\mathbf{x}$ is given by the average

$$\hat{f}(\mathbf{x}) \;=\; \frac{1}{K} \sum_{k=1}^{K} h(\mathbf{x}, \Theta_k).$$

There are different algorithms for creating the K base learners in a random forest, e.g. ID3 [32], C4.5 [33], CART [7] or MARS [16]. In this case study the CART algorithm was used with MSE as the splitting criterion. Unlike the canonical Random Forest algorithm, experiments showed that not using

bootstrap sampling improved performance, so each base learner was trained on the entire training set. The final fitted model parameters were:

$$\begin{aligned} \texttt{n_estimators} &= 433, \\ \texttt{max_depth} &= 34, \\ \texttt{min_samples_split} &= 8, \\ \texttt{min_samples_leaf} &= 1, \\ \texttt{max_features} &= 0.16, \\ \texttt{bootstrap} &= \texttt{False} \end{aligned}$$

B.5 Boosted Decision Tree (Adaboost)

While Random Forest uses Bootstrap sampling ("Bagging") to train its base learners, Adaboost was the first widely adopted algorithm focussing on "Boosting", where the training instances are reweighted at each iteration. There have been proposed various modifications and improvements to the original algorithm by Freund and Schapire [13], e.g. Adaboost.R2 [11], Adaboost.RT [41] or Robust AdaBoost.RT [48]. For this case study the Adaboost.R2 algorithm was chosen.

Definition B.5.1 (AdaBoost.R2 Regressor)**.** An Adaboost.R2 Regressor is a model consisting of a sequence of weak learners $\{h_k\}_{k=1}^K$ trained on reweighted samples from the training set. The model prediction at an input $\mathbf{x}$ is given by the weighted median

$$\hat{f}(\mathbf{x}) = \inf_{y \in \mathbf{Y}} \left\{ \sum_{k:h_k(\mathbf{x}) \leq y} \log\left(\frac{1}{\beta_k}\right) \geq \frac{1}{2} \sum_{k=1}^{K} \log\left(\frac{1}{\beta_k}\right) \right\}.$$

The base learners $\{h_k\}_{k=1}^K$ and weights $\{\beta_k\}_{k=1}^K$ are obtained from the following algorithm where w_i are initialized as 1:

1. Pick (with replacement) N instances $\mathbf{x}^{(i)}$ from $\mathbf{X}_{\mathcal{D}}$, where
$$P(\mathbf{x}^{(i)}) = \frac{w_i}{\sum_{j=1}^N w_j}$$
2. Fit a weak learner h_k to this data
3. Calculate the loss $\ell_i = \ell(\mathbf{y}^{(i)}, h_k(\mathbf{x}^{(i)}))$ for each instance $\mathbf{x}^{(i)} \in \mathbf{X}_{\mathcal{D}}$ (Note that any loss function $\ell \in [0,1]$ is permissible, Drucker [11] suggests normalized linear, square or exponential)
4. Calculate the average loss $L_k = \sum_{i=1}^N P(\mathbf{x}^{(i)}) \cdot \ell_i$
5. Set
$$\beta_k = \frac{L_k}{1 - L_k}, \quad w_i = w_i \cdot \beta_k^{1-\ell_i}$$

Steps 1-5 are repeated until $L_k < 0.5$.

Although AdaBoost is traditionally paired with weak learners such as decision stumps, our experiments showed that allowing deeper CART trees as base learners markedly improved validation performance (Table 3). We then conducted a targeted hyperparameter search, which selected a configuration with `max_depth` $= 31$ and $K = 495$ estimators as best on validation; however, training this model required approximately 30 minutes per fit, which is impractical for the repeated retraining needed in this case study. Because the accuracy gain over leaner settings was small, we adopt the following faster configuration for all subsequent analyses:

$$\begin{aligned} \texttt{n_estimators} &= 100, \\ \texttt{learning_rate} &= 1, \\ \texttt{loss} &= \texttt{linear}, \\ \texttt{max_depth} &= 30. \end{aligned}$$

Table 3: AdaBoost.R2 validation MSE by tree depth and loss; depth 31 (square) best, depth 30 (linear) chosen.

Tree depth	Loss	MSE
1	exponential	0.975
2	exponential	0.676
3	exponential	0.633
5	exponential	0.580
10	linear	0.592
20	linear	0.440
30	linear	0.432
31	square	0.422
40	linear	0.438

B.6 Extreme Gradient Boosting (XGBoost)

Definition B.6.1 (XGBoost Regressor, after Chen and Guestrin [10])**.** An XGBoost Regressor is an additive ensemble of K CART regression trees:

$$\hat{f}_i = \phi(\mathbf{x}^{(i)}) = \sum_{k=1}^{K} h_k(\mathbf{x}^{(i)})$$

The base learners $\{h_k\}_{k=1}^{K}$ are obtained by minimizing the regularized objective

$$\mathcal{L}(\phi) = \sum_{i=1}^{N} \ell(f_i, \hat{f}_i) + \sum_{k=1}^{K} \Omega(h_k), \quad \Omega(h) = \gamma T + \frac{\lambda}{2}\|w\|_2^2,$$

where ℓ is a convex differentiable loss function, T the number of leaves in h, w the (continuous) leaf weights, and γ and λ regularization hyperparameters controlling model complexity.

This minimization is approximated via second-order Taylor expansion of the loss function; for further details see [10]. In addition to shrinkage (learning rate η) and row subsampling, XGBoost applies column subsampling at each split, and supports sparsity-aware split finding and weighted quantile sketching for approximate split proposals.

In this case study, the loss ℓ was chosen as MSE and the final model parameters were:

$$\begin{aligned}
\texttt{n_estimators} &= 592,\\
\texttt{max_depth} &= 36,\\
\texttt{learning_rate} &= 0.0342,\\
\texttt{subsample} &= 0.8732,\\
\texttt{colsample_bytree} &= 0.3596,\\
\texttt{reg_lambda} &= 6.9773,\\
\texttt{reg_gamma} &= 0.0305,\\
\texttt{reg_alpha} &= 2.7000,\\
\texttt{min_child_weight} &= 7.
\end{aligned}$$

B.7 Neural Networks (Feed-forward Regression)

Neural networks provide a flexible class of nonlinear predictors by composing affine maps with element-wise nonlinearities. In this case study, we use standard fully-connected feed-forward networks (multilayer perceptrons, MLPs) and fit them by empirical risk minimisation with squared loss; for background and further details see Goodfellow, Bengio, and Courville [18].

Definition B.7.1 (Feed-forward neural network regressor (MLP), after Goodfellow, Bengio, and Courville [18, pp. 164-166])**.** Let $L \geq 2$ and let $\sigma_1, \dots, \sigma_{L-1}$ be element-wise activation functions. For parameters $\theta = \{(W_\ell, b_\ell)\}_{\ell=1}^{L}$ with compatible dimensions, define the hidden representations recursively by

$$z_0(\mathbf{x}) = \mathbf{x}, \qquad z_\ell(\mathbf{x}) = \sigma_\ell\big(W_\ell z_{\ell-1}(\mathbf{x}) + b_\ell\big), \quad \ell = 1, \dots, L-1,$$

and the scalar regression output by the final affine map

$$\hat{f}(\mathbf{x}) = f_\theta(\mathbf{x}) = W_L z_{L-1}(\mathbf{x}) + b_L.$$

Given training data $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$, the fitted parameters are obtained as

$$\hat{\theta} \in \arg\min_{\theta} \; \frac{1}{N} \sum_{i=1}^{N} \big(y^{(i)} - f_\theta(\mathbf{x}^{(i)})\big)^2,$$

approximately solved by stochastic gradient methods.

All neural networks were trained in TensorFlow/Keras with the Adam optimiser and mean squared error loss. As described in the preprocessing subsection, numerical features were median-imputed and standardised; boolean/non-numeric columns were passed through without scaling in the default configuration.

B.8 Shallow Neural Network (one hidden layer)

The first architecture is an MLP with a single hidden layer and ReLU nonlinearity,

$$\mathbf{x} \;\mapsto\; \mathrm{Dense}(6963, \mathrm{ReLU}) \;\mapsto\; \mathrm{Dense}(1, \mathrm{linear}).$$

The selected training hyperparameters were

$$\begin{aligned} \texttt{epochs} &= 17, \\ \texttt{batch_size} &= 64, \\ \texttt{optimizer} &= \texttt{Adam}, \\ \texttt{learning_rate} &= 1.303 \cdot 10^{-5}. \end{aligned}$$

B.9 Neural Network with two hidden layers (MLP)

The second architecture is a deeper MLP with ELU nonlinearities and dropout regularisation between the hidden layers,

$$\mathbf{x} \;\mapsto\; \mathrm{Dense}(166, \mathrm{ELU}) \;\mapsto\; \mathrm{Dropout}(0.4685) \;\mapsto\; \mathrm{Dense}(99, \mathrm{ELU}) \;\mapsto\; \mathrm{Dense}(1, \mathrm{linear}).$$

The selected training hyperparameters were

$$\begin{aligned} \texttt{epochs} &= 84, \\ \texttt{batch_size} &= 64, \\ \texttt{optimizer} &= \texttt{Adam}, \\ \texttt{learning_rate} &= 0.0022962 \end{aligned}$$

C Case Study

C.1 Permutation Feature Importance Tables

To compute permutation feature importance (PFI) scores (Definition 4.3.2) for each fitted model, we evaluate the drop in predictive performance when a single feature column is randomly permuted in the test set while all other columns are kept fixed. Since all predictors are implemented as `sklearn`-style pipelines, the permutation is applied to the raw test-set feature table and predictions are then recomputed through the full preprocessing pipeline; in particular, missing numerical entries are imputed as in model training, so the reported PFI values quantify the sensitivity of the complete trained pipeline to each original input feature.

Concretely, for each feature we repeat the permutation procedure `n_repeats`= 10 times with a fixed random seed and score the permuted data using the `neg_mean_squared_error` loss. For each feature, the tables report the empirical mean (`PFI`) and standard deviation (`PFI SD`) of the score decrease across repeats. To facilitate comparisons within a model, we additionally report a normalized importance `PFI norm`, defined as the mean importance divided by the sum of absolute mean importances over all features. All PFI computations are performed on the full holdout test split (without subsampling).

Table 4: Top 20 features by PFI for Linear Regression

Feature	PFI	PFI SD	PFI norm	Rank
category_Expansion for Base-game	0.214278	0.003783	0.376592	1
complexity	0.104074	0.002883	0.182909	2
mechanic_Roll_/_Spin_and_Move	0.018391	0.001054	0.032322	3
category_Wargame	0.018073	0.001032	0.031764	4
game_type_War Game	0.013416	0.000775	0.023578	5
min_age	0.012206	0.000880	0.021452	6
category_Miniatures	0.009742	0.000671	0.017121	7
implementation_avg_rating	0.008434	0.000879	0.014823	8
family_Crowdfunding	0.007479	0.000847	0.013144	9
min_players	0.006417	0.000727	0.011277	10
mechanic_Cooperative_Game	0.006010	0.000503	0.010563	11
game_type_Abstract Game	0.006009	0.000611	0.010561	12
designer_high_xp	0.005630	0.000918	0.009895	13
compilation_count	0.005041	0.000766	0.008859	14
category_Movies / TV / Radio theme	0.004807	0.000747	0.008447	15
category_Trivia	0.004411	0.000647	0.007752	16
publisher_(Self-Published)	0.004117	0.000549	0.007236	17
mechanic_Deck_Construction	0.003443	0.000315	0.006052	18
category_Humor	0.003242	0.000323	0.005697	19
year	0.002994	0.000436	0.005262	20

Table 5: Top 20 features by PFI for Lasso Regression

Feature	PFI	PFI SD	PFI norm	Rank
category_Expansion for Base-game	0.203789	0.003665	0.382294	1
complexity	0.116074	0.003013	0.217747	2
mechanic_Roll_/_Spin_and_Move	0.023294	0.001201	0.043698	3
category_Wargame	0.016406	0.001070	0.030776	4
min_age	0.015049	0.000963	0.028231	5
category_Miniatures	0.012095	0.000815	0.022689	6
implementation_avg_rating	0.011378	0.000961	0.021345	7
family_Crowdfunding	0.010255	0.001030	0.019238	8
min_players	0.007346	0.000849	0.013780	9
game_type_War Game	0.006592	0.000658	0.012365	10
mechanic_Solo_/_Solitaire_Game	0.005744	0.000657	0.010776	11
designer_high_xp	0.005641	0.000865	0.010582	12
compilation_count	0.005362	0.000846	0.010058	13
mechanic_Cooperative_Game	0.005119	0.000503	0.009602	14
category_Trivia	0.005116	0.000660	0.009598	15
category_Movies / TV / Radio theme	0.004847	0.000676	0.009093	16
year	0.003857	0.000500	0.007235	17
publisher_(Self-Published)	0.003769	0.000506	0.007070	18
category_Humor	0.003385	0.000335	0.006350	19
game_type_Abstract Game	0.003064	0.000370	0.005748	20

Table 6: Top 20 features by PFI for Random Forest

Feature	PFI	PFI SD	PFI norm	Rank
year	0.297615	0.007120	0.349719	1
category_Expansion for Base-game	0.178254	0.002927	0.209461	2
complexity	0.123581	0.002313	0.145217	3
max_time	0.028458	0.001864	0.033440	4
min_age	0.018478	0.000482	0.021712	5
min_players	0.016431	0.001004	0.019307	6
category_Miniatures	0.015553	0.000575	0.018276	7
category_Wargame	0.013111	0.000993	0.015406	8
max_players	0.010113	0.000703	0.011884	9
mechanic_Roll_/_Spin_and_Move	0.009401	0.001222	0.011047	10
min_time	0.008190	0.000761	0.009624	11
implementation_avg_rating	0.007557	0.001149	0.008880	12
family_Crowdfunding	0.006341	0.000590	0.007451	13
designer_high_xp	0.005796	0.000717	0.006811	14
mechanic_Variable_Player_Powers	0.004608	0.000438	0.005414	15
game_type_Family Game	0.004028	0.000413	0.004733	16
category_Children's Game	0.003421	0.000347	0.004020	17
mechanic_Cooperative_Game	0.003251	0.000205	0.003820	18
category_Movies / TV / Radio theme	0.003130	0.000466	0.003677	19
publisher_high_xp	0.003044	0.000297	0.003577	20

Table 7: Top 20 features by PFI for AdaBoost

Feature	PFI	PFI SD	PFI norm	Rank
year	0.393457	0.008960	0.391255	1
category_Expansion for Base-game	0.198449	0.004032	0.197339	2
complexity	0.179740	0.004794	0.178735	3
max_time	0.028575	0.002158	0.028415	4
category_Miniatures	0.013175	0.000548	0.013101	5
min_age	0.012256	0.000676	0.012188	6
max_players	0.012087	0.001310	0.012019	7
category_Wargame	0.010669	0.000741	0.010610	8
min_players	0.009869	0.000578	0.009814	9
min_time	0.008215	0.001216	0.008169	10
designer_high_xp	0.006830	0.000755	0.006792	11
mechanic_Roll_/_Spin_and_Move	0.006463	0.000959	0.006426	12
implementation_avg_rating	0.006132	0.000699	0.006097	13
game_type_Family Game	0.004972	0.000553	0.004944	14
publisher_low_xp	0.004556	0.000736	0.004530	15
publisher_average_xp	0.004527	0.000520	0.004501	16
publisher_high_xp	0.004388	0.000635	0.004364	17
designer_average_xp	0.004144	0.000709	0.004121	18
mechanic_Variable_Player_Powers	0.004076	0.000533	0.004053	19
category_Movies / TV / Radio theme	0.003429	0.000519	0.003410	20

Table 8: Top 20 features by PFI for XGBoost

Feature	PFI	PFI SD	PFI norm	Rank
year	0.244927	0.005652	0.311475	1
category_Expansion for Base-game	0.150057	0.003292	0.190829	2
complexity	0.090681	0.002511	0.115320	3
max_time	0.028669	0.001688	0.036458	4
min_age	0.018513	0.000761	0.023543	5
category_Wargame	0.014160	0.001341	0.018007	6
min_players	0.012536	0.000812	0.015943	7
min_time	0.012030	0.000766	0.015299	8
max_players	0.011715	0.000781	0.014898	9
implementation_avg_rating	0.011329	0.001229	0.014407	10
category_Miniatures	0.010191	0.000680	0.012959	11
designer_high_xp	0.008665	0.001038	0.011019	12
mechanic_Roll_/_Spin_and_Move	0.007241	0.000740	0.009208	13
publisher_high_xp	0.007063	0.001051	0.008982	14
designer_average_xp	0.006444	0.000624	0.008195	15
family_Crowdfunding	0.006090	0.000572	0.007744	16
compilation_count	0.005985	0.000639	0.007611	17
designer_low_xp	0.005549	0.000587	0.007057	18
publisher_(Self-Published)	0.005501	0.000481	0.006995	19
publisher_very_high_xp	0.004926	0.000384	0.006265	20

Table 9: Top 20 features by PFI for NN1

Feature	PFI	PFI SD	PFI norm	Rank
category_Expansion for Base-game	0.216204	0.003806	0.255047	1
year	0.213508	0.004315	0.251867	2
complexity	0.112039	0.004011	0.132168	3
category_Wargame	0.030046	0.001303	0.035444	4
min_players	0.022040	0.001362	0.025999	5
max_players	0.017115	0.000999	0.020190	6
category_Miniatures	0.014174	0.001267	0.016721	7
min_age	0.009189	0.000771	0.010839	8
implementation_avg_rating	0.008635	0.001191	0.010186	9
designer_high_xp	0.008153	0.000734	0.009618	10
publisher_(Self-Published)	0.007840	0.000581	0.009248	11
publisher_average_xp	0.007531	0.000460	0.008884	12
publisher_low_xp	0.006364	0.000581	0.007507	13
compilation_count	0.005587	0.001073	0.006590	14
designer_low_xp	0.004881	0.000537	0.005758	15
designer_average_xp	0.004838	0.000707	0.005708	16
publisher_high_xp	0.004254	0.000497	0.005018	17
mechanic_Roll_/_Spin_and_Move	0.004140	0.000630	0.004884	18
designer_very_high_xp	0.003952	0.000343	0.004663	19
category_Movies / TV / Radio theme	0.003743	0.000376	0.004415	20

Table 10: Top 20 features by PFI for NN3

Feature	PFI	PFI SD	PFI norm	Rank
year	0.235062	0.003805	0.274348	1
category_Expansion for Base-game	0.165147	0.003115	0.192748	2
complexity	0.081932	0.003430	0.095625	3
implementation_avg_rating	0.015129	0.001922	0.017658	4
max_time	0.012592	0.000919	0.014697	5
min_players	0.012174	0.002173	0.014209	6
designer_low_xp	0.012102	0.001548	0.014124	7
designer_average_xp	0.011032	0.000994	0.012876	8
category_Wargame	0.010339	0.000991	0.012067	9
publisher_high_xp	0.010132	0.000907	0.011825	10
category_Miniatures	0.009662	0.001737	0.011276	11
min_age	0.009497	0.000795	0.011084	12
publisher_very_high_xp	0.008509	0.001028	0.009931	13
designer_high_xp	0.008255	0.000875	0.009635	14
compilation_count	0.007963	0.000997	0.009294	15
category_Book	0.005942	0.000886	0.006936	16
mechanic_Dice_Rolling	0.005112	0.000572	0.005966	17
designer_very_high_xp	0.005055	0.001231	0.005900	18
category_Card Game	0.004308	0.000546	0.005028	19
mechanic_Hand_Management	0.004244	0.000467	0.004953	20

C.2 ALE Plots

The following pages contain the ALE plots for all models for features of interest. For continuous features we use Equation (28), with $K = 50$. For binary features we run the same calculation but with $K = 2$ (i.e., splitting the data into two groups based on the feature value). For Plotting, we then calculate the Delta between the two groups. This can be interpreted as the change of estimated effect when switching the feature from 0 to 1. We show a positive change in green and a negative change in red.

Figure 22: ALE plots for Linear Regression. Rows 1–2: continuous features. Rows 3–4: binary indicators.

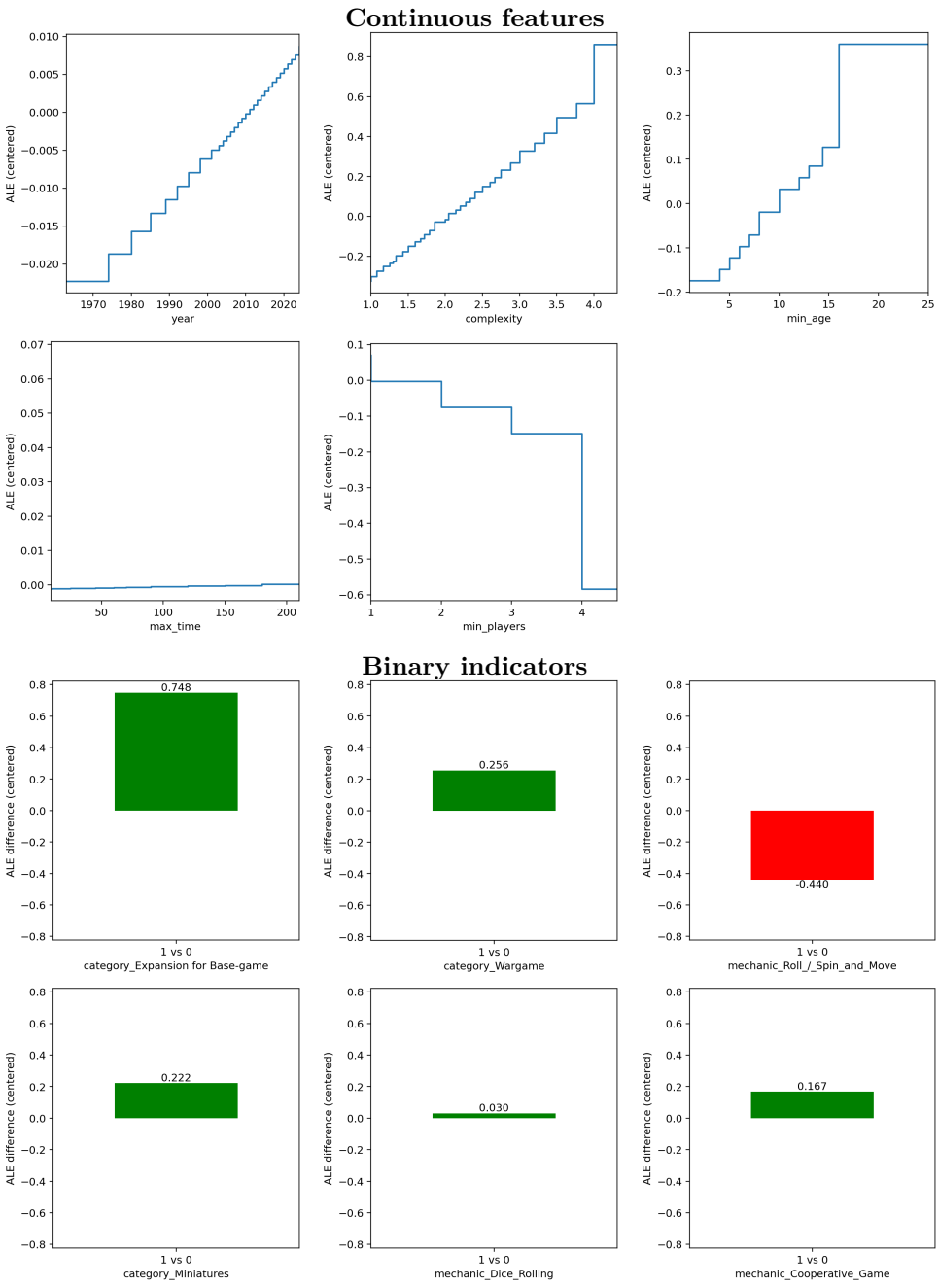


Figure 23: ALE plots for Lasso Regression. Rows 1–2: continuous features. Rows 3–4: binary indicators.

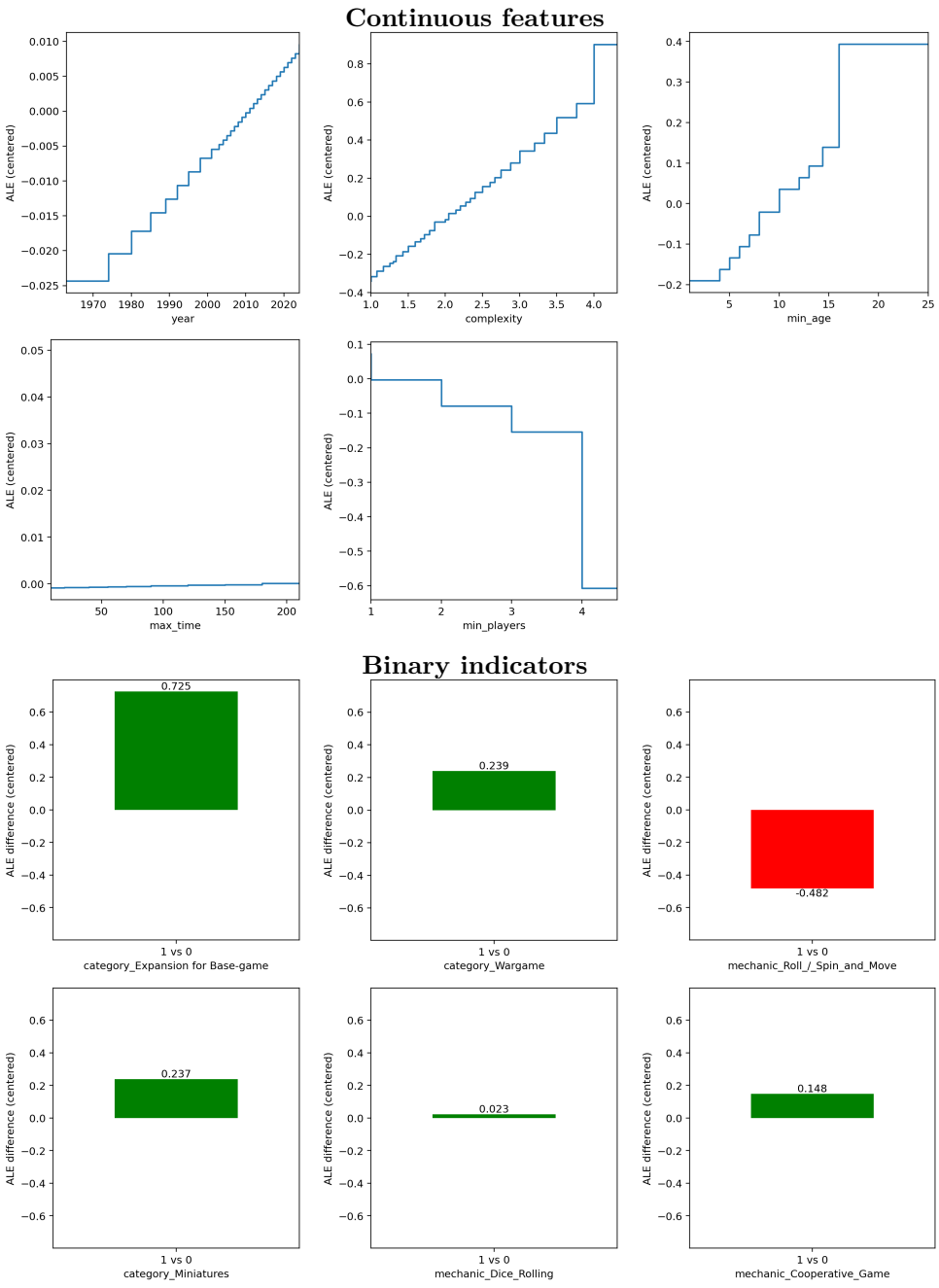


Figure 24: ALE plots for Random Forest. Rows 1–2: continuous features. Rows 3–4: binary indicators.

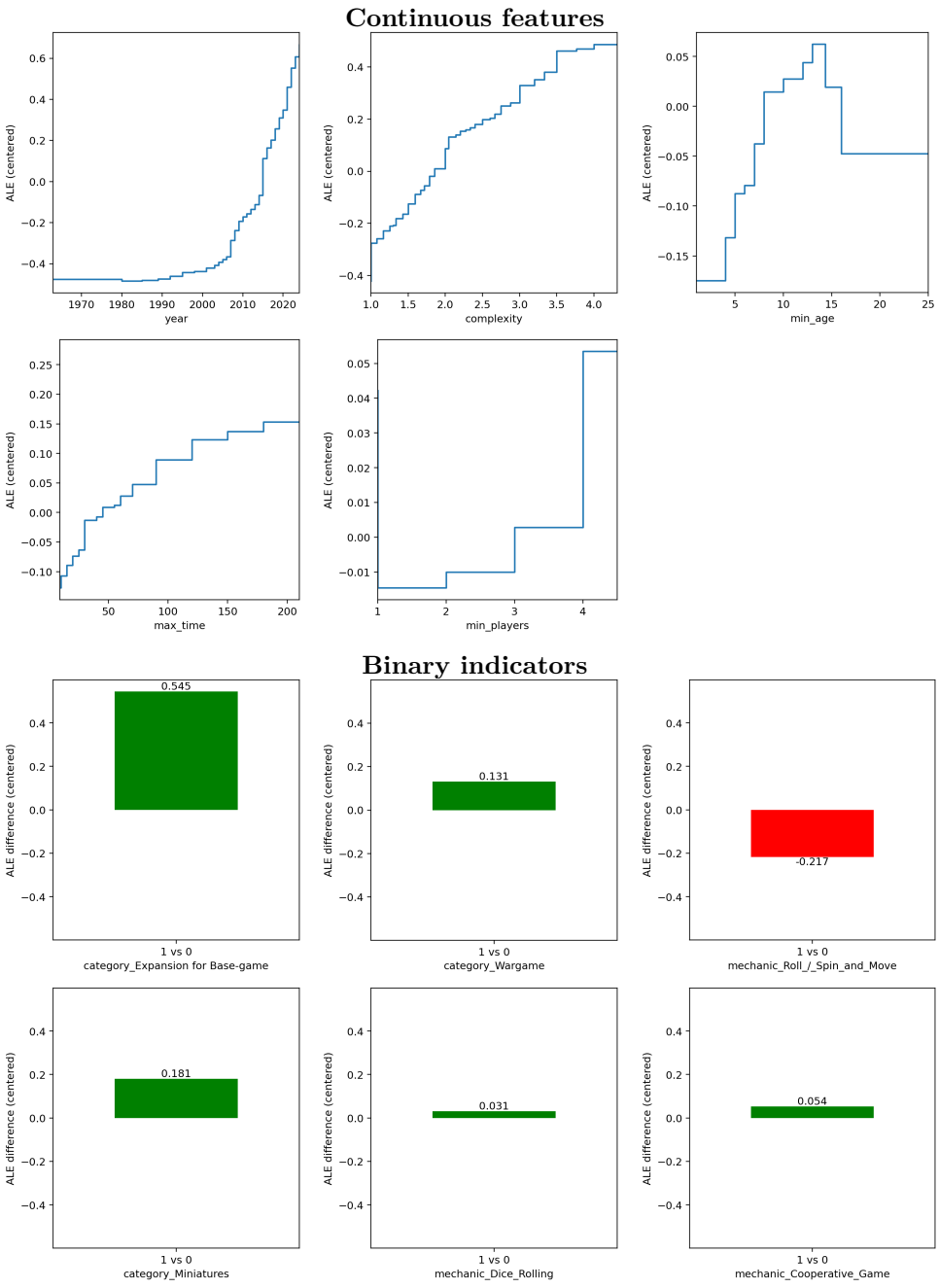


Figure 25: ALE plots for AdaBoost. Rows 1–2: continuous features. Rows 3–4: binary indicators.

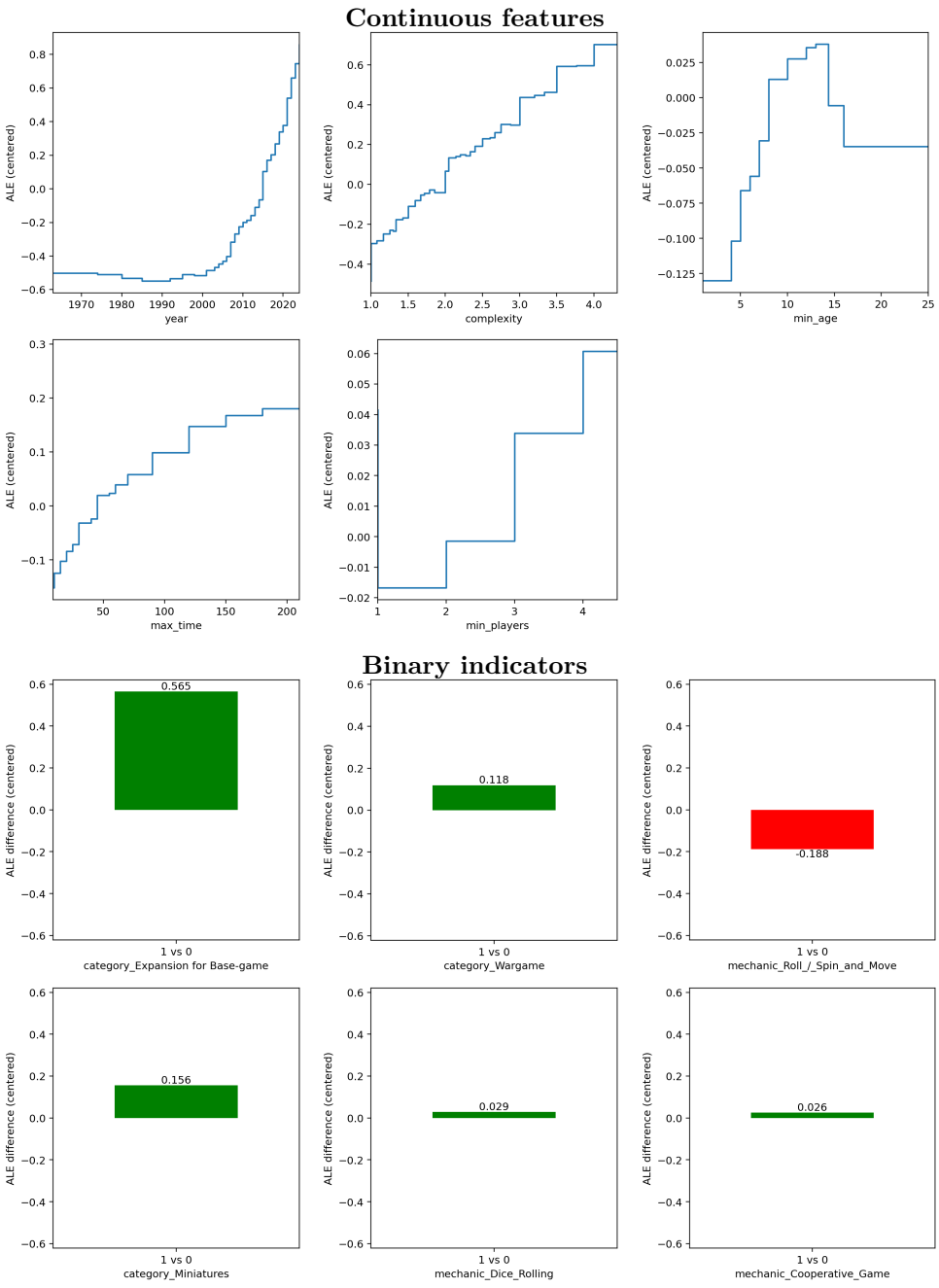


Figure 26: ALE plots for XGBoost. Rows 1–2: continuous features. Rows 3–4: binary indicators.

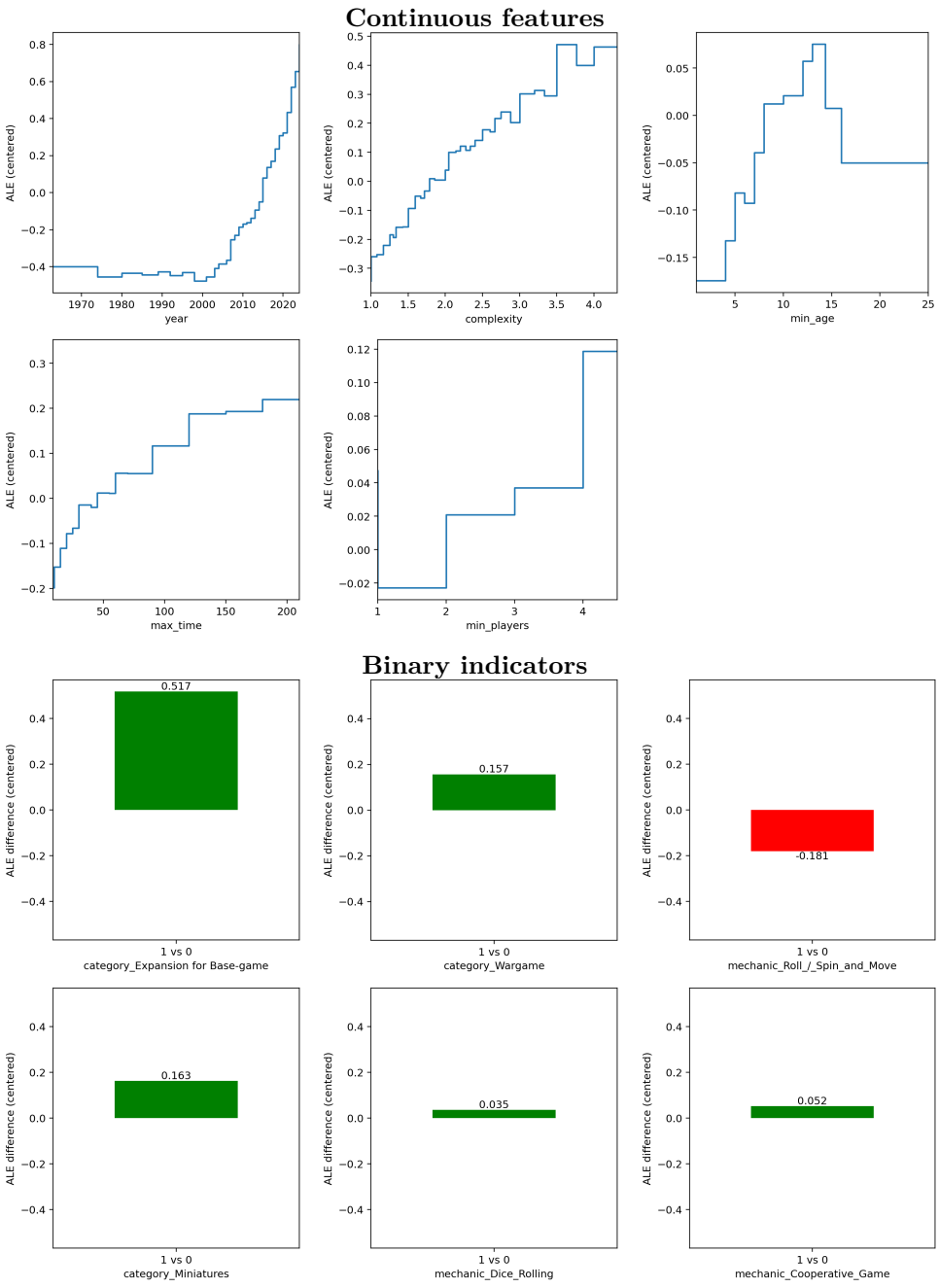


Figure 27: ALE plots for NN1. Rows 1–2: continuous features. Rows 3–4: binary indicators.

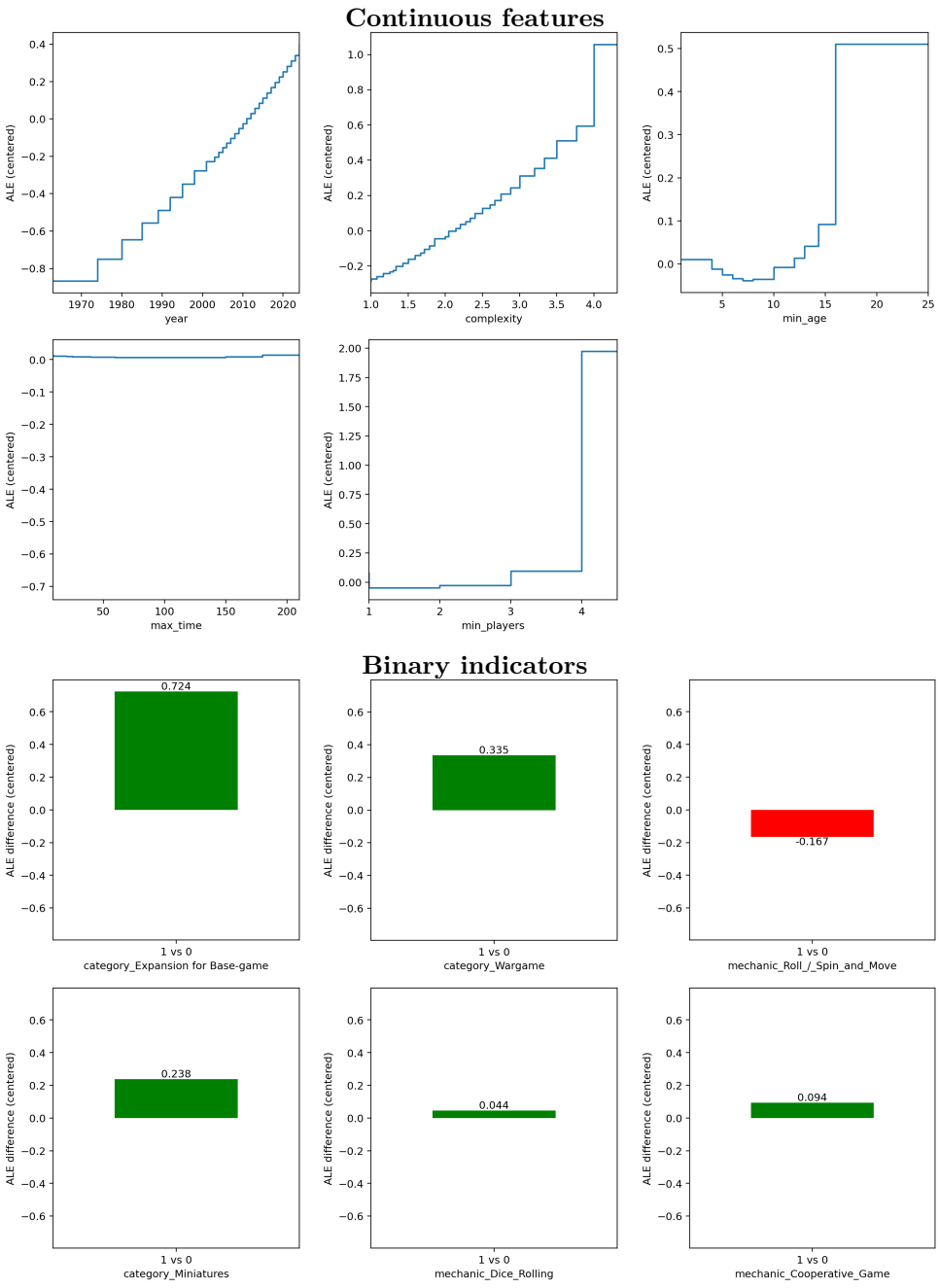
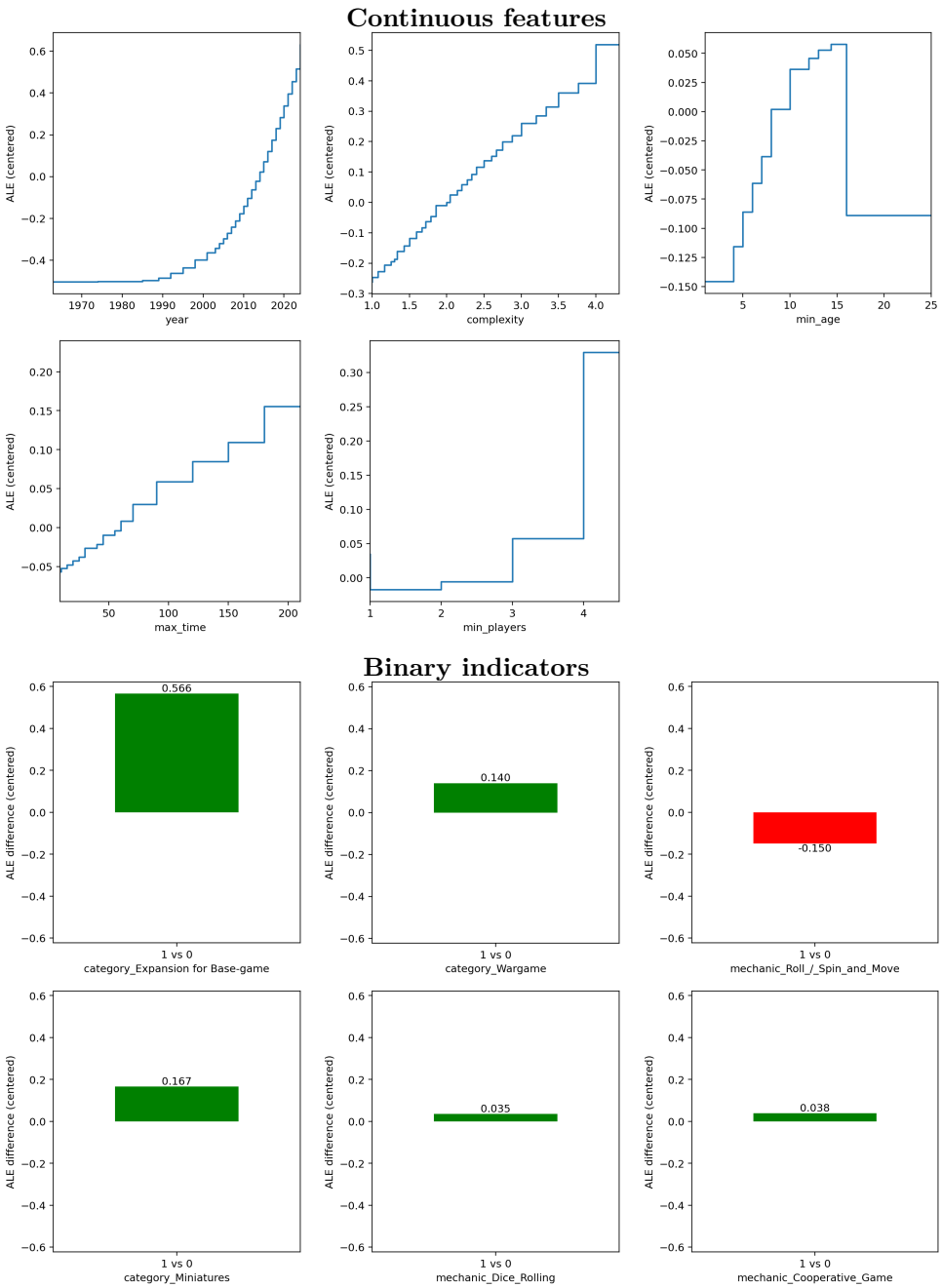


Figure 28: ALE plots for NN3. Rows 1–2: continuous features. Rows 3–4: binary indicators.



C.3 Friedman's H-Statistic Tables

To estimate the feature-wise interaction strengths H_j (Definition 4.4.3) and the pairwise interaction strengths H_{jk} (Definition 4.4.2) on the board game dataset, we approximate all required partial dependence terms by Monte-Carlo sampling, since exact evaluation becomes costly for large N and many feature pairs. For a given model $\hat{f}$, we draw a background set of size $M = 64$ uniformly at random from the training set and approximate partial dependence expectations by averaging predictions over this background set. Interaction strengths are then evaluated on a random subsample of the test set of size $N_{\text{eval}} = 1500$. For continuous features, $PD_j(x_j^{(i)})$ and $PD_{jk}(x_j^{(i)}, x_k^{(i)})$ are estimated by replacing the corresponding feature values in the background rows by the evaluation values $x_j^{(i)}$ (and $(x_j^{(i)}, x_k^{(i)})$) and averaging $\hat{f}$'s predictions over the M modified background rows. For binary features, the computation is accelerated by evaluating the partial dependence only at the two levels $\{0, 1\}$ and using a lookup for the evaluation points. All partial dependence quantities are centered with respect to the evaluation set to ensure identifiability up to additive constants. Finally, the expectations in H_j^2 and H_{jk}^2 are computed as empirical means over the evaluation sample. For reproducibility, the evaluation sample and background subsamples are drawn with a fixed random seed, and predictions are computed in batches (batch size = 256).

Table 11: Top 20 features by H_j for Linear Regression

Feature	H_j^2	H_j	Rank
designer_low_xp	1.946190e-29	4.411564e-15	1
designer_average_xp	1.871397e-29	4.325965e-15	2
mechanic_Roll_/_Spin_and_Move	1.806156e-29	4.250006e-15	3
publisher_high_xp	1.752166e-29	4.186605e-15	4
publisher_(Self-Published)	1.617523e-29	4.021846e-15	5
category_Expansion for Base-game	1.615977e-29	4.019921e-15	6
mechanic_Dice_Rolling	1.537359e-29	3.920663e-15	7
category_Miniatures	1.453823e-29	3.812636e-15	8
family_Crowdfunding	1.423834e-29	3.773112e-15	9
category_Wargame	1.361740e-29	3.689095e-15	10
implementation_avg_rating	1.356721e-29	3.682287e-15	11
year	1.303839e-29	3.611703e-15	12
compilation_count	1.184934e-29	3.442289e-15	13
min_age	1.094379e-29	3.308140e-15	14
min_players	1.033820e-29	3.215000e-15	15
max_players	9.462153e-30	3.076062e-15	16
min_time	7.990223e-30	2.826345e-15	17
complexity	6.338928e-30	2.517721e-15	18
max_time	5.572647e-30	2.360646e-15	19
designer_high_xp	4.475215e-31	6.690440e-16	20

Table 12: Top 20 features by H_j for Lasso Regression

Feature	H_j^2	H_j	Rank
max_time	1.289503e-29	3.590965e-15	1
mechanic_Roll_/_Spin_and_Move	1.017174e-29	3.189317e-15	2
category_Miniatures	9.465076e-30	3.076862e-15	3
designer_average_xp	8.840027e-30	2.973217e-15	4
family_Crowdfunding	8.319684e-30	2.884386e-15	5
publisher_high_xp	8.065608e-30	2.840001e-15	6
designer_high_xp	7.794944e-30	2.791941e-15	7
category_Wargame	7.689953e-30	2.773076e-15	8
publisher_(Self-Published)	7.648217e-30	2.765546e-15	9
category_Expansion for Base-game	7.310330e-30	2.703761e-15	10
year	6.986936e-30	2.643660e-15	11
designer_low_xp	6.793214e-30	2.606377e-15	12
min_age	6.712063e-30	2.590765e-15	13
implementation_avg_rating	6.687275e-30	2.585966e-15	14
max_players	6.524930e-30	2.554394e-15	15
min_players	6.278011e-30	2.505595e-15	16
compilation_count	6.192780e-30	2.488529e-15	17
min_time	6.158177e-31	7.847406e-16	18
complexity	6.158177e-31	7.847406e-16	19
mechanic_Dice_Rolling	6.158177e-31	7.847406e-16	20

Table 13: Top 20 features by H_j for Random Forest

Feature	H_j^2	H_j	Rank
year	0.037341	0.193238	1
complexity	0.034877	0.186754	2
max_time	0.022645	0.150482	3
category_Expansion for Base-game	0.022290	0.149298	4
max_players	0.017015	0.130441	5
min_players	0.012740	0.112870	6
min_age	0.012409	0.111398	7
min_time	0.010718	0.103528	8
designer_high_xp	0.005546	0.074471	9
category_Miniatures	0.005525	0.074331	10
mechanic_Roll_/_Spin_and_Move	0.005474	0.074000	11
category_Wargame	0.004179	0.064646	12
publisher_high_xp	0.004160	0.064501	13
designer_average_xp	0.003778	0.061469	14
publisher_(Self-Published)	0.002427	0.049262	15
family_Crowdfunding	0.002364	0.048616	16
implementation_avg_rating	0.002361	0.048591	17
designer_low_xp	0.002350	0.048476	18
mechanic_Dice_Rolling	0.002291	0.047865	19
compilation_count	0.001954	0.044201	20

Table 14: Top 20 features by H_j for AdaBoost

Feature	H_j^2	H_j	Rank
year	0.072623	0.269487	1
complexity	0.061450	0.247892	2
category_Expansion for Base-game	0.036943	0.192205	3
max_time	0.019908	0.141094	4
max_players	0.013502	0.116199	5
min_age	0.010107	0.100535	6
min_time	0.007921	0.089000	7
designer_high_xp	0.005908	0.076867	8
min_players	0.005301	0.072811	9
category_Miniatures	0.005029	0.070919	10
mechanic_Roll_/_Spin_and_Move	0.005003	0.070731	11
category_Wargame	0.004846	0.069613	12
publisher_high_xp	0.004169	0.064571	13
designer_average_xp	0.003733	0.061097	14
family_Crowdfunding	0.002643	0.051408	15
publisher_(Self-Published)	0.002571	0.050705	16
implementation_avg_rating	0.002561	0.050602	17
designer_low_xp	0.002258	0.047515	18
mechanic_Dice_Rolling	0.002157	0.046444	19
compilation_count	0.002038	0.045143	20

Table 15: Top 20 features by H_j for XGBoost

Feature	H_j^2	H_j	Rank
year	0.026764	0.163598	1
complexity	0.020243	0.142278	2
category_Expansion for Base-game	0.017244	0.131315	3
max_time	0.009856	0.099277	4
min_age	0.009364	0.096766	5
min_time	0.006949	0.083361	6
max_players	0.005354	0.073168	7
min_players	0.005106	0.071459	8
category_Wargame	0.005039	0.071000	9
category_Miniatures	0.003781	0.061491	10
publisher_high_xp	0.003780	0.061489	11
designer_high_xp	0.002551	0.050511	12
designer_average_xp	0.002452	0.049513	13
publisher_(Self-Published)	0.002359	0.048570	14
designer_low_xp	0.002060	0.045384	15
mechanic_Roll_/_Spin_and_Move	0.001967	0.044349	16
family_Crowdfunding	0.001808	0.042523	17
implementation_avg_rating	0.001759	0.041941	18
compilation_count	0.001756	0.041911	19
mechanic_Dice_Rolling	0.001681	0.041002	20

Table 16: Top 20 features by H_j for NN1

Feature	H_j^2	H_j	Rank
complexity	0.010549	0.102710	1
min_players	0.004943	0.070306	2
year	0.004481	0.066939	3
category_Expansion for Base-game	0.003713	0.060935	4
max_time	0.003101	0.055685	5
max_players	0.002784	0.052765	6
min_time	0.002426	0.049254	7
min_age	0.002374	0.048717	8
compilation_count	0.002326	0.048229	9
designer_high_xp	0.002296	0.047916	10
designer_average_xp	0.002058	0.045360	11
mechanic_Roll_/_Spin_and_Move	0.001803	0.042460	12
category_Miniatures	0.001802	0.042448	13
publisher_high_xp	0.001722	0.041495	14
category_Wargame	0.001634	0.040425	15
mechanic_Dice_Rolling	0.001625	0.040310	16
designer_low_xp	0.001601	0.040011	17
family_Crowdfunding	0.001483	0.038510	18
publisher_(Self-Published)	0.001445	0.038010	19
implementation_avg_rating	0.001432	0.037848	20

Table 17: Top 20 features by H_j for NN3

Feature	H_j^2	H_j	Rank
year	0.030293	0.174049	1
category_Expansion for Base-game	0.021346	0.146103	2
complexity	0.014226	0.119273	3
publisher_high_xp	0.011808	0.108664	4
designer_high_xp	0.010969	0.104734	5
min_players	0.010432	0.102137	6
min_age	0.008638	0.092945	7
max_players	0.007718	0.087856	8
max_time	0.007416	0.086114	9
min_time	0.007012	0.083742	10
category_Miniatures	0.006162	0.078500	11
category_Wargame	0.006100	0.078104	12
designer_average_xp	0.005664	0.075261	13
designer_low_xp	0.004994	0.070667	14
mechanic_Roll_/_Spin_and_Move	0.003235	0.056880	15
publisher_very_high_xp	0.003193	0.056512	16
publisher_(Self-Published)	0.002785	0.052777	17
compilation_count	0.002700	0.051961	18
family_Crowdfunding	0.002496	0.049955	19
implementation_avg_rating	0.002370	0.048679	20

Table 18: Pairwise interaction strengths H_{jk} for XGBoost on the profile feature set

Feature j	Feature k	H_{jk}^2	H_{jk}	Rank
min_time	max_players	0.053221	0.230696	1
min_age	min_time	0.046830	0.216403	2
max_players	min_players	0.037392	0.193370	3
min_age	min_players	0.030911	0.175815	4
min_age	max_players	0.028873	0.169919	5
max_time	min_players	0.028589	0.169082	6
max_time	min_age	0.025653	0.160166	7
max_time	max_players	0.024428	0.156294	8
min_players	min_time	0.010771	0.103783	9
max_time	min_time	0.010086	0.100431	10
year	category_Expansion for Base-game	0.007228	0.084999	11
category_Expansion for Base-game	max_players	0.006228	0.078920	12
year	max_time	0.005778	0.076012	13
year	complexity	0.005373	0.073301	14
complexity	max_time	0.004968	0.070483	15
complexity	category_Expansion for Base-game	0.004587	0.067729	16
year	min_time	0.003611	0.060088	17
complexity	min_time	0.003550	0.059578	18
category_Expansion for Base-game	min_time	0.003483	0.059018	19
year	max_players	0.003039	0.055125	20
year	min_players	0.002974	0.054537	21
complexity	max_players	0.002176	0.046649	22
complexity	min_players	0.002041	0.045176	23
category_Expansion for Base-game	max_time	0.001599	0.039984	24
category_Expansion for Base-game	min_players	0.001516	0.038928	25
min_players	max_players	0.000866	0.029434	26
year	min_age	0.000528	0.022989	27
complexity	min_age	0.000266	0.016314	28

Table 19: Pairwise interaction strengths H_{jk} for XGBoost on the mechanics-and-categories feature set, Part 1

Feature j	Feature k	H_{jk}^2	H_{jk}	Rank
category_Miniatures	mechanic_Dice_Rolling	0.025830	0.160717	1
mechanic_Dice_Rolling	min_players	0.021017	0.144971	2
category_Wargame	min_players	0.013682	0.116971	3
complexity	max_time	0.009983	0.099914	4
category_Wargame	mechanic_Dice_Rolling	0.009750	0.098742	5
complexity	category_Expansion for Base-game	0.009274	0.096302	6
mechanic_Cooperative_Game	min_players	0.007248	0.085134	7
max_time	min_players	0.007095	0.084234	8
category_Wargame	complexity	0.006525	0.080775	9
mechanic_Dice_Rolling	mechanic_Roll_/_Spin_and_Move	0.006362	0.079762	10
year	category_Expansion for Base-game	0.005982	0.077343	11
year	max_time	0.005918	0.076929	12
category_Expansion for Base-game	max_time	0.005547	0.074477	13
year	complexity	0.005289	0.072724	14
category_Wargame	max_time	0.005277	0.072644	15
complexity	category_Miniatures	0.004965	0.070462	16
complexity	mechanic_Dice_Rolling	0.004704	0.068585	17
category_Expansion for Base-game	min_players	0.004341	0.065884	18
year	min_players	0.004287	0.065471	19
max_time	mechanic_Dice_Rolling	0.004139	0.064333	20
category_Expansion for Base-game	category_Miniatures	0.003992	0.063183	21
category_Miniatures	min_players	0.003964	0.062961	22
mechanic_Cooperative_Game	max_time	0.003846	0.062015	23
category_Miniatures	max_time	0.003640	0.060333	24
year	category_Miniatures	0.003556	0.059635	25
year	category_Wargame	0.003437	0.058627	26
category_Expansion for Base-game	mechanic_Dice_Rolling	0.003197	0.056539	27
category_Expansion for Base-game	category_Wargame	0.003167	0.056275	28

Table 20: Pairwise interaction strengths H_{jk} for XGBoost on the mechanics-and-categories feature set, Part 2

Feature j	Feature k	H_{jk}^2	H_{jk}	Rank
year	mechanic_Dice_Rolling	0.003077	0.055469	29
max_time	mechanic_Roll_/_Spin_and_Move	0.002762	0.052554	30
category_Wargame	category_Miniatures	0.002631	0.051290	31
complexity	mechanic_Cooperative_Game	0.002596	0.050952	32
year	mechanic_Cooperative_Game	0.002523	0.050230	33
mechanic_Cooperative_Game	mechanic_Dice_Rolling	0.002453	0.049527	34
category_Wargame	mechanic_Cooperative_Game	0.002390	0.048885	35
year	mechanic_Roll_/_Spin_and_Move	0.002311	0.048076	36
complexity	mechanic_Roll_/_Spin_and_Move	0.002187	0.046764	37
mechanic_Cooperative_Game	mechanic_Roll_/_Spin_and_Move	0.001983	0.044536	38
category_Miniatures	mechanic_Roll_/_Spin_and_Move	0.001971	0.044401	39
category_Miniatures	mechanic_Cooperative_Game	0.001943	0.044082	40
mechanic_Roll_/_Spin_and_Move	min_players	0.001638	0.040470	41
year	category_Expansion for Base-game	0.001556	0.039448	42
category_Expansion for Base-game	mechanic_Roll_/_Spin_and_Move	0.001494	0.038654	43
category_Wargame	mechanic_Roll_/_Spin_and_Move	0.001452	0.038106	44
year	category_Wargame	0.000798	0.028255	45

C.4 SHAP

To compute local feature attributions by SHAP (Definition 3.2.16) on the board game dataset, we use the `shap` Python package provided by Lundberg and Lee [27] with the model-agnostic permutation estimator (`shap.Explainer`). Since all models are implemented as `sklearn`-style pipelines, SHAP values are computed with respect to the model input after preprocessing. In particular, the pipelines were trained on data with an imputation step applied to missing numerical values; therefore, missing entries in a game's feature vector are imputed before SHAP values are evaluated. Consequently, the explanation corresponds to the prediction of the pipeline for the imputed instance actually seen by the model. For transparency, we store both the raw feature values and the imputed values used for SHAP in the output tables (see appendix files for each explained instance).

For a given instance $\mathbf{x}$ to be explained, the permutation SHAP estimator approximates the Shapley values by evaluating the model on random feature coalitions relative to a background distribution. As background dataset we sample $M = 200$ observations uniformly at random from the training set (fixed random seed). The baseline value ϕ_0 is the model's expected prediction on this background set. We bound the computational cost by limiting the number of model evaluations per explanation to `max_evals=2000` (default SHAP sampling scheme), and we compute explanations independently for XGBoost and NN3. The corresponding waterfall plots are presented in Figures 29-38.

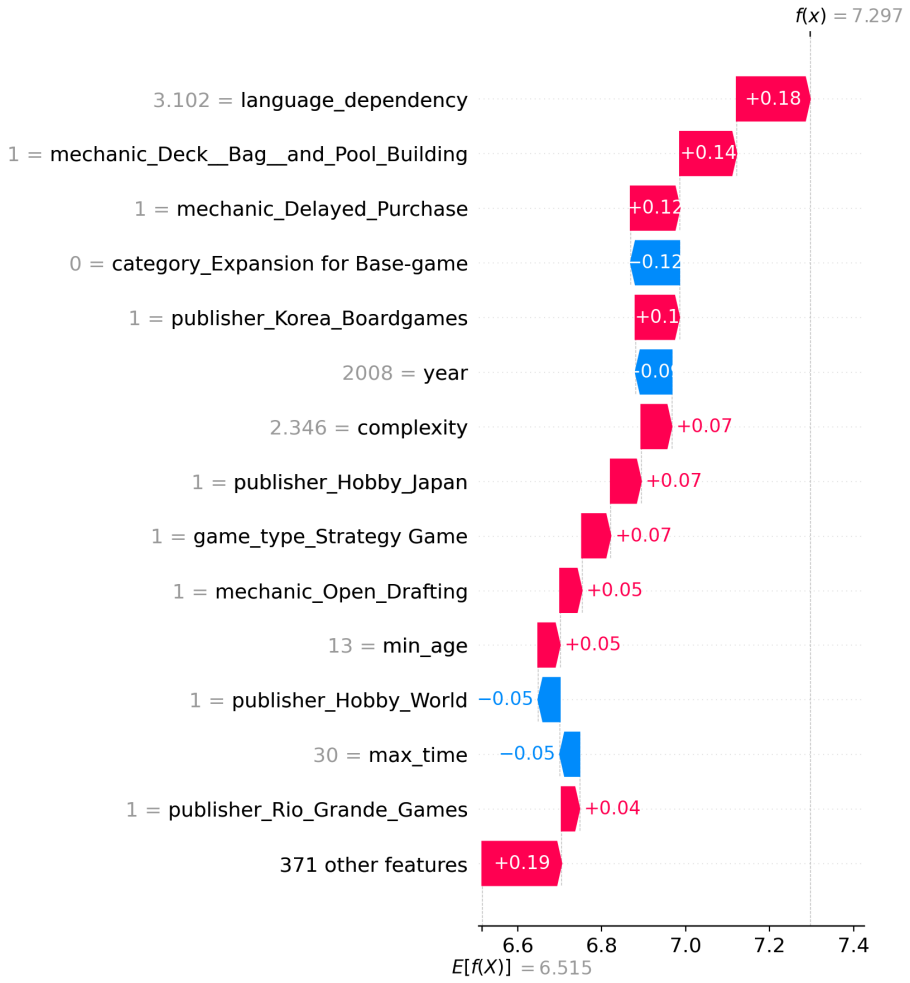


Figure 29: SHAP explanations for Dominion for NN3 model

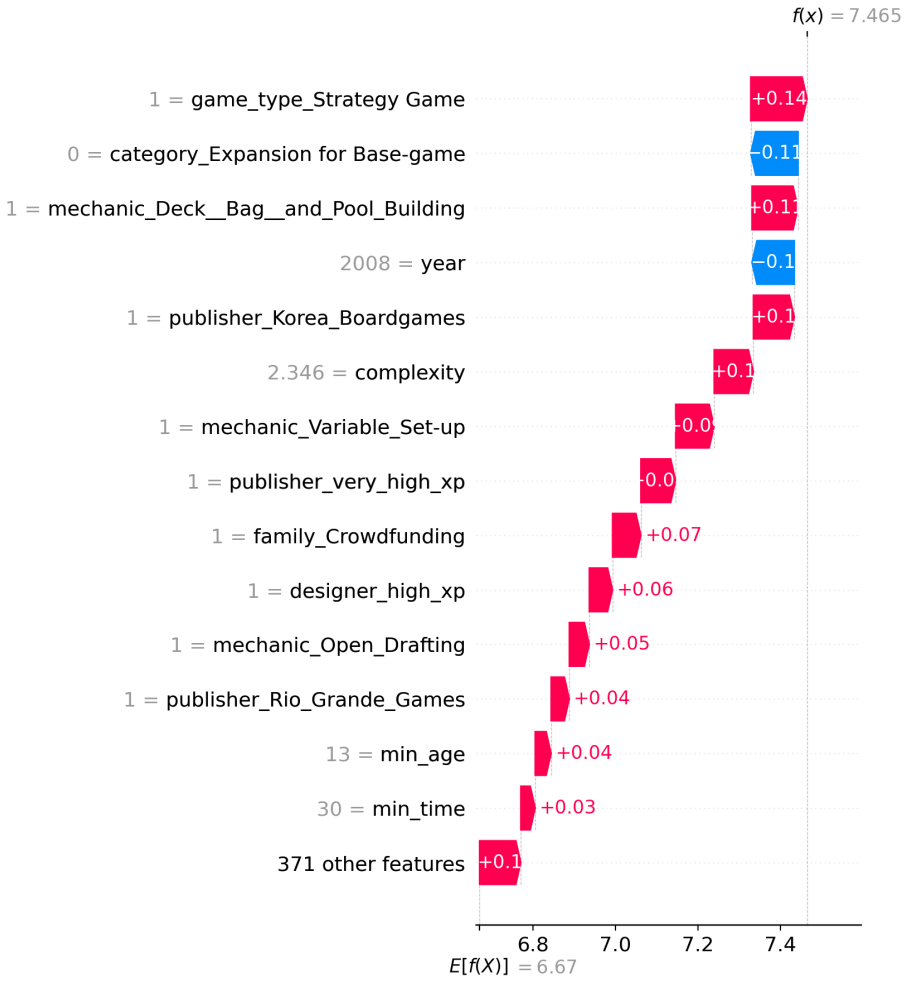


Figure 30: SHAP explanations for Dominion for XGBoost model

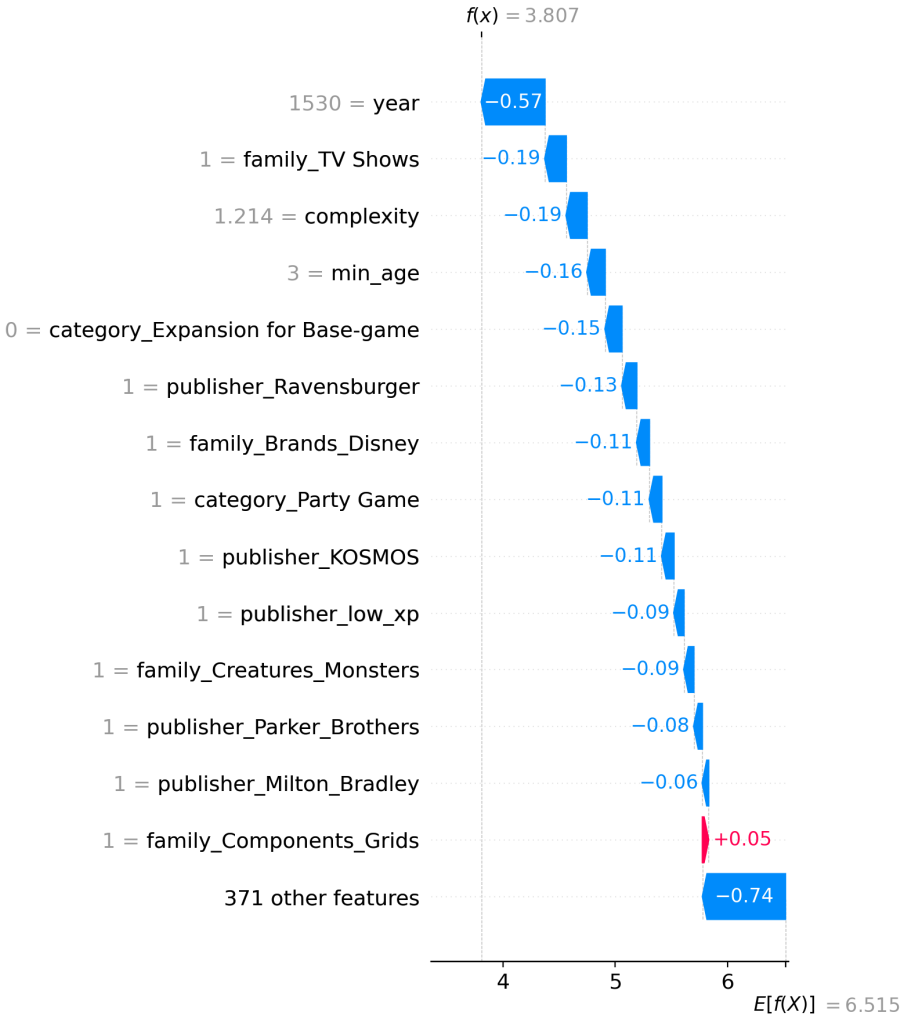


Figure 31: SHAP explanations for Lotto for NN3 model



Figure 32: SHAP explanations for Lotto for XGBoost model

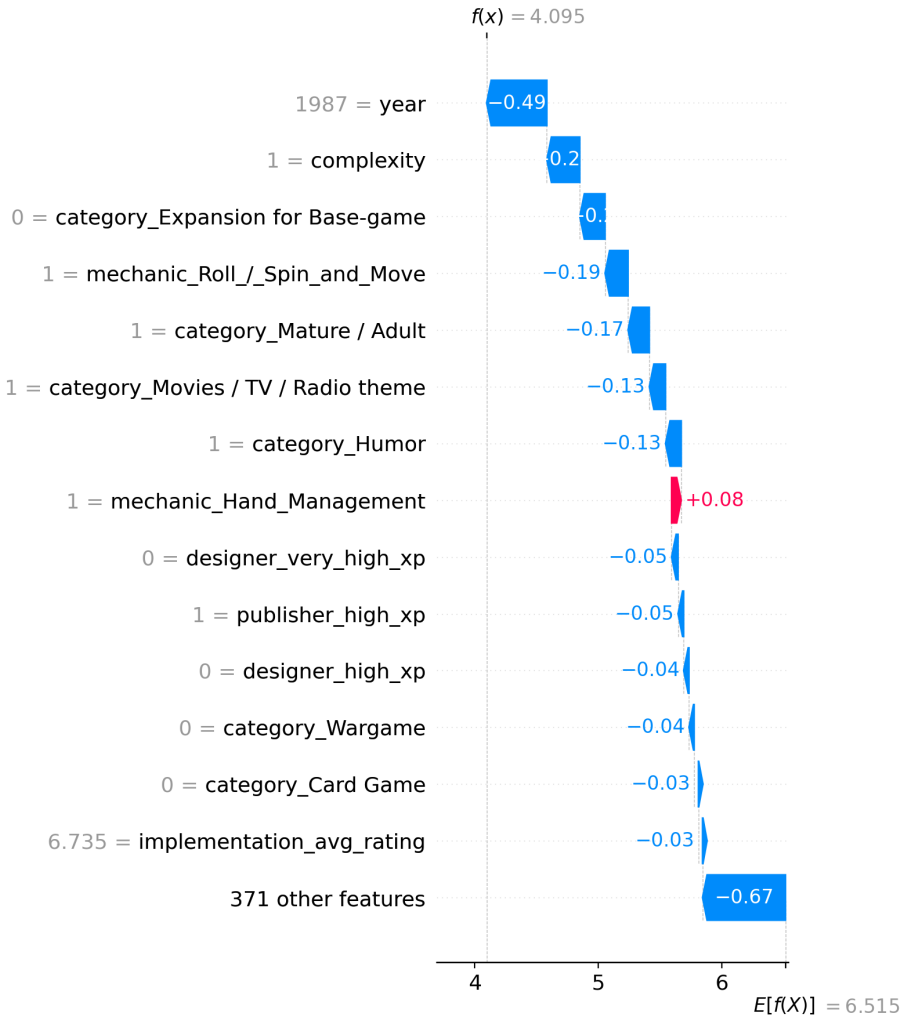


Figure 33: SHAP explanations for How to Be a Complete Bastard for NN3 model



Figure 34: SHAP explanations for How to Be a Complete Bastard for XGBoost model

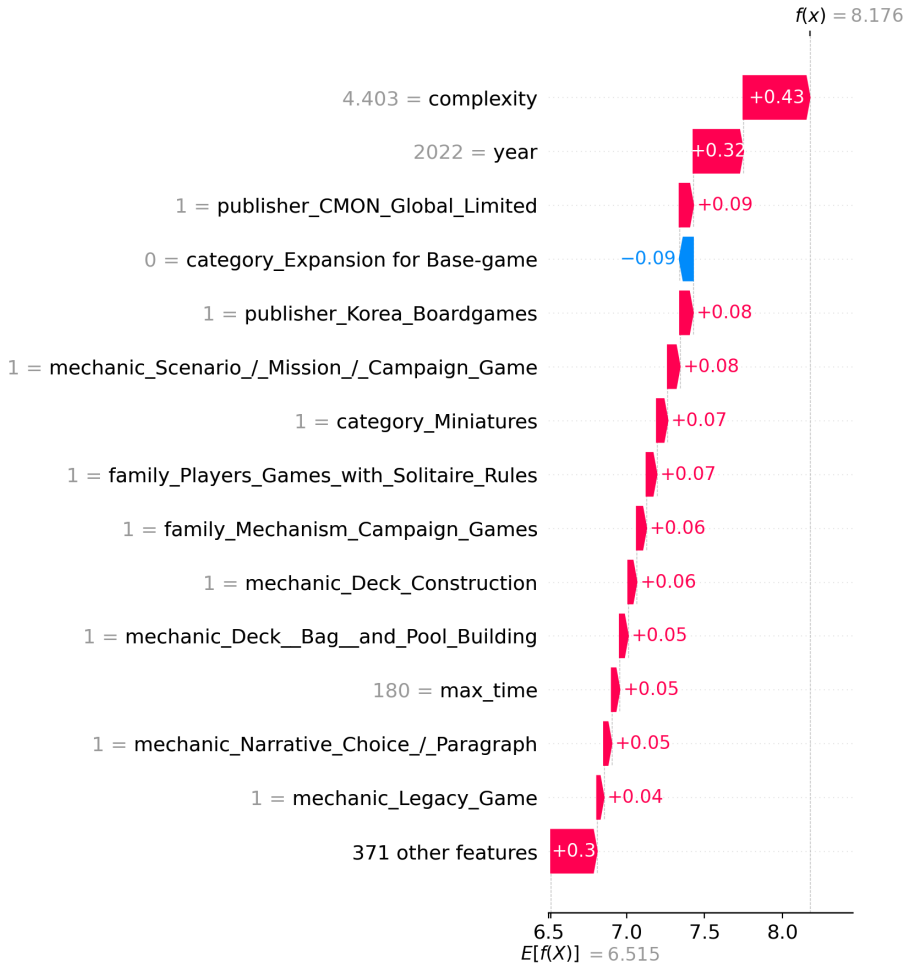


Figure 35: SHAP explanations for Frosthaven for NN3 model

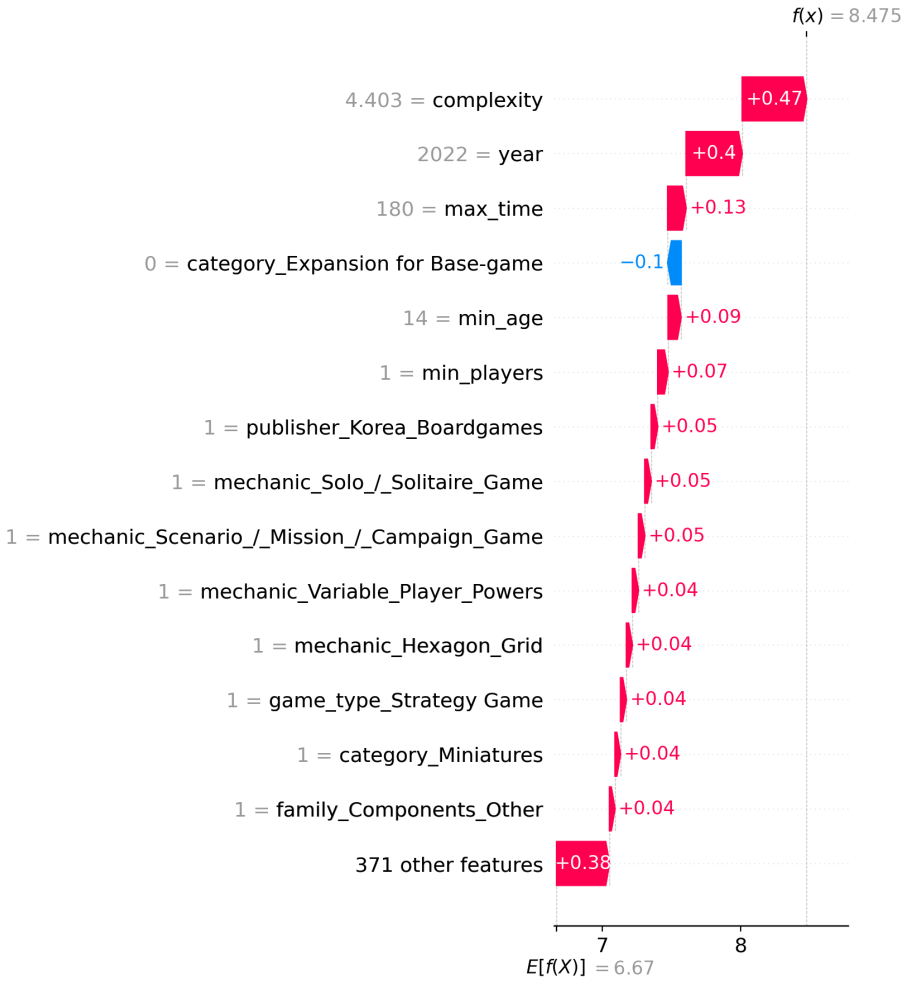


Figure 36: SHAP explanations for Frosthaven for XGBoost model

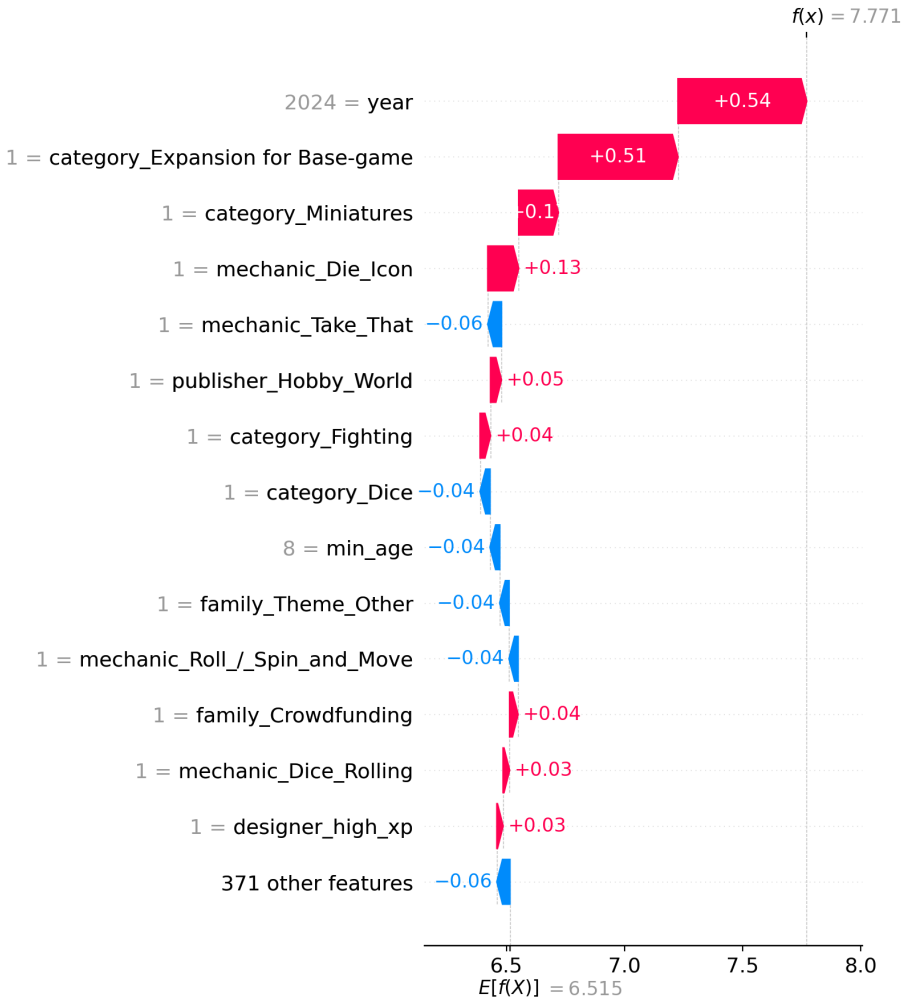


Figure 37: SHAP explanations for Thunder Road: Vendetta – Turbo Tina's Swag Pack for NN3 model

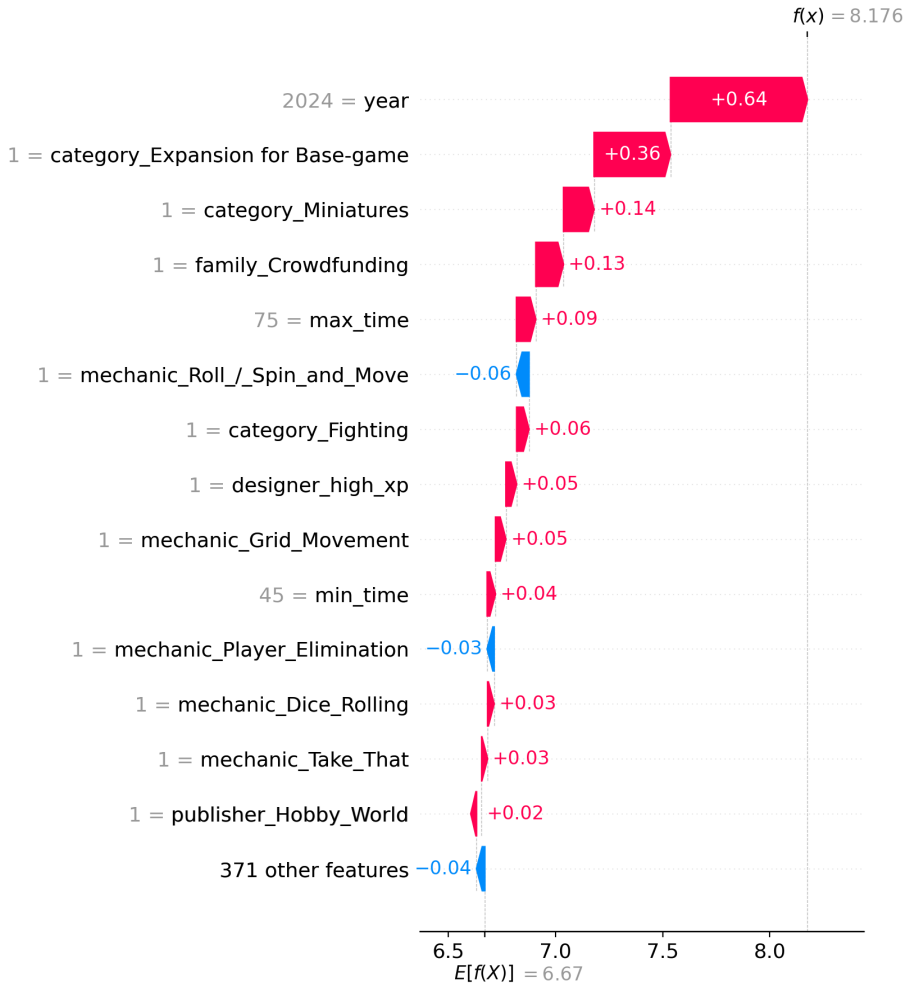


Figure 38: SHAP explanations for Thunder Road: Vendetta – Turbo Tina's Swag Pack for XGBoost model

C.5 CP Plots

To compute Ceteris Paribus (CP) curves (Definition 3.1.1) for the local analysis, we follow the standard one-feature intervention scheme. For a fixed test instance $\mathbf{x} \in \mathbf{X}$ and a numerical feature index j, we construct a grid $\mathcal{G}_j$ of plausible values for x_j from the empirical training distribution of feature j. Concretely, we use either the set of unique observed values if the feature is sufficiently discrete, or otherwise a quantile grid of fixed size (here: $|\mathcal{G}_j| = 25$). The CP curve for feature j is then obtained by evaluating the model pipeline on the counterfactual inputs (x_{-j}, g) for all $g \in \mathcal{G}_j$, i.e. we replace only the j-th feature value by g while keeping all remaining features fixed to the original instance. Predictions are computed through the full preprocessing pipeline; in particular, missing numerical values are imputed as in model training, so the CP curves describe the model's local behaviour with respect to the preprocessed input representation.

To avoid CP grids being dominated by rare outliers and to keep the resulting curves interpretable, we restrict the grid ranges for selected features by clipping to fixed bounds: `max_players`≤ 10, `min_players`≤ 5, `max_time`≤ 240, `min_time`≤ 240, `min_age`≤ 25, and `year`≥ 1960. Note that for tree-based models the response as a function of a single feature can be non-smooth; when plotting a CP curve from evaluations on a finite grid, linear interpolation between grid points may cause the instance marker to not lie exactly on the drawn line unless the instance value is included in the grid.

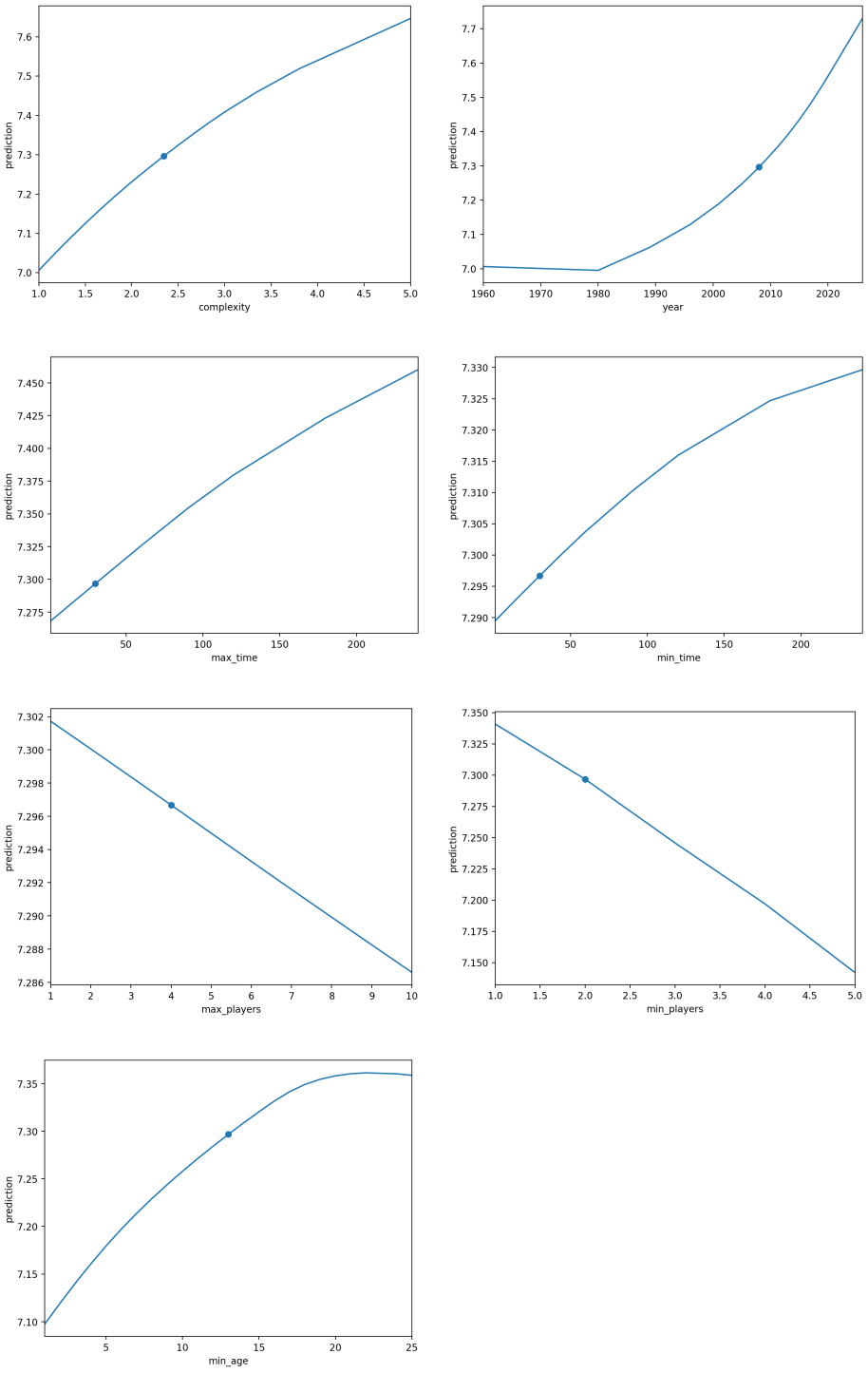


Figure 39: CP plots for Dominion for NN3 model

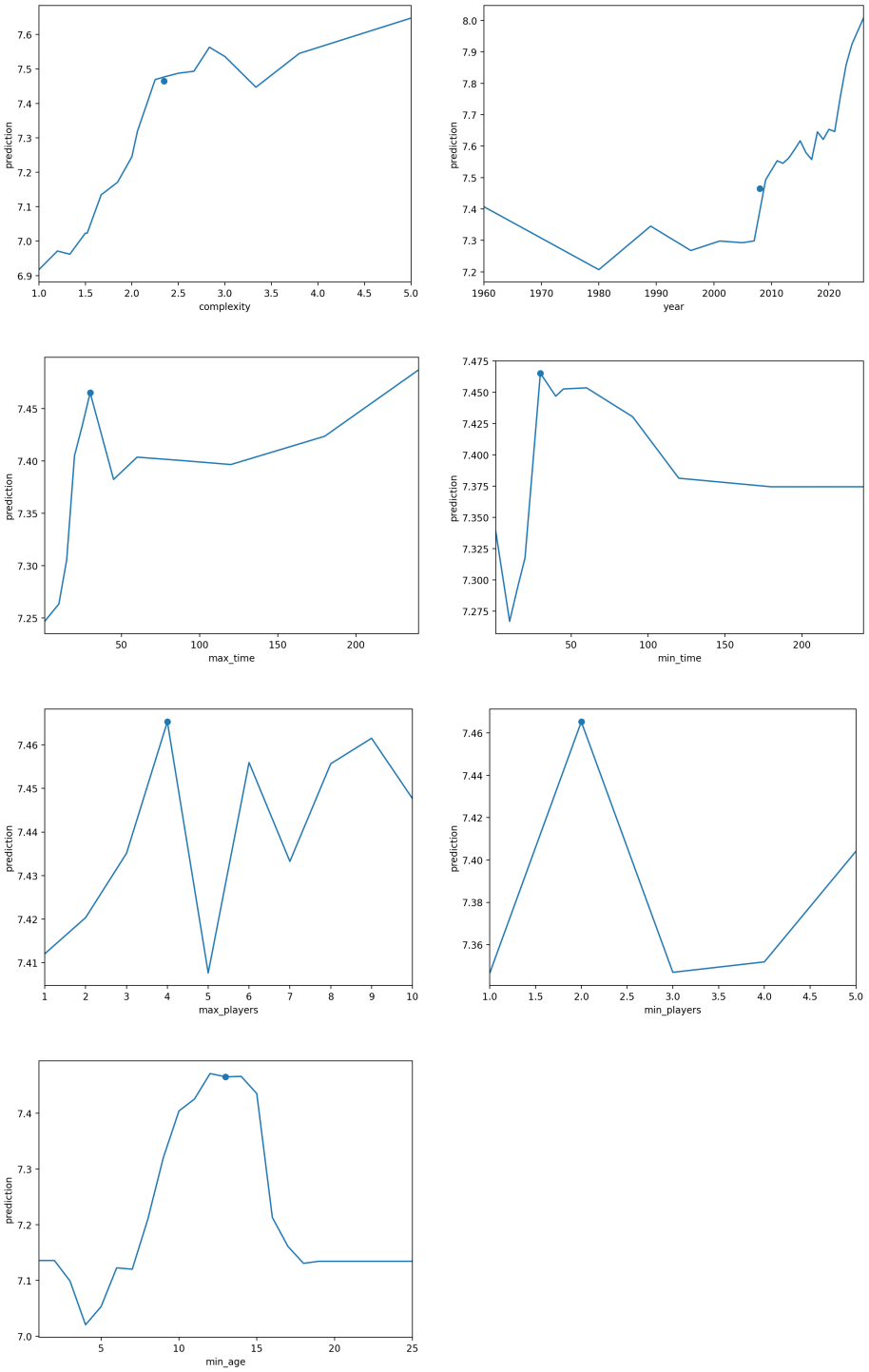


Figure 40: CP plots for Dominion for XGB model

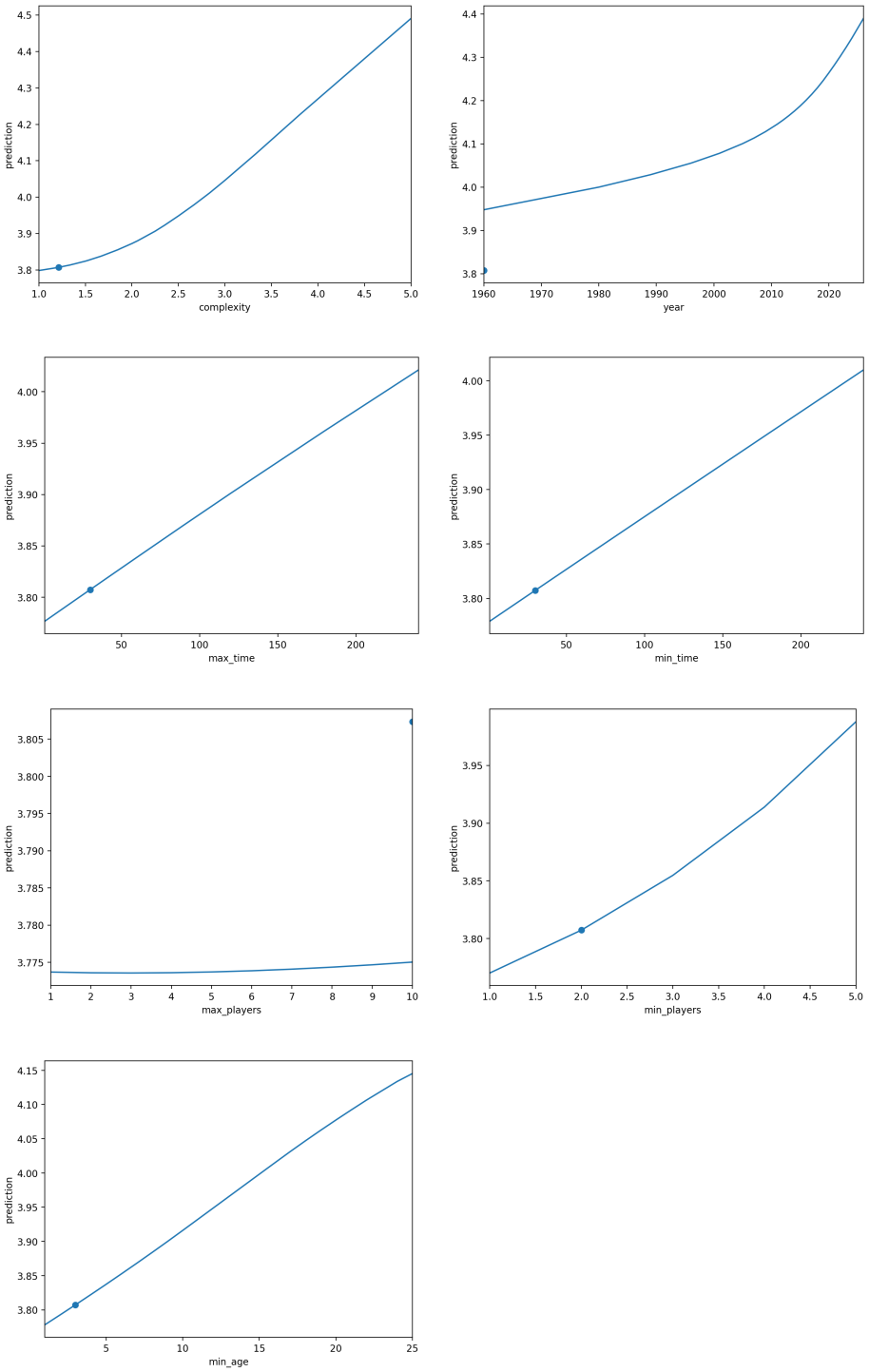


Figure 41: CP plots for Lotto for NN3 model

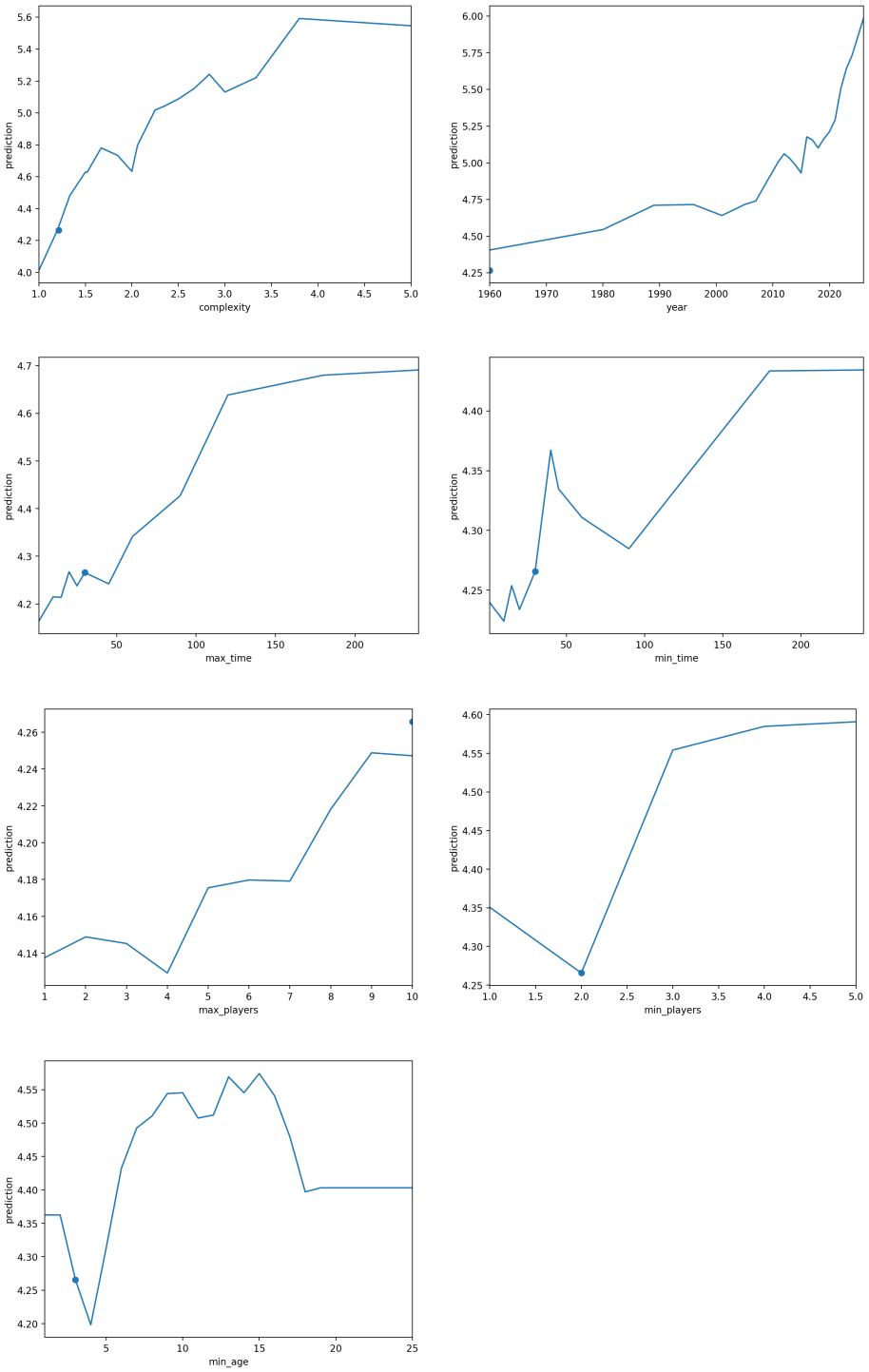


Figure 42: CP plots for Lotto for XGB model

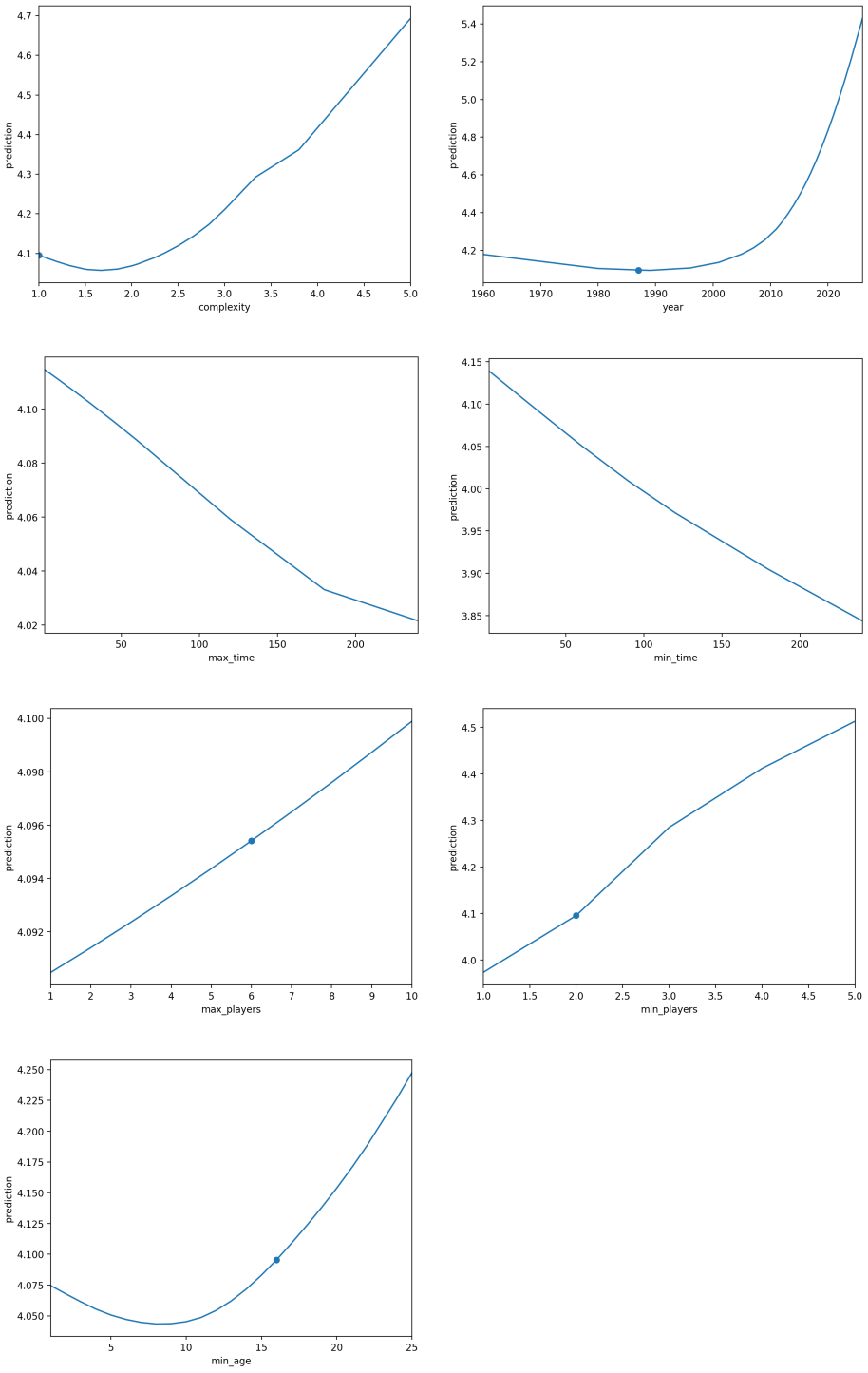


Figure 43: CP plots for How to Be a Complete Bastard for NN3 model

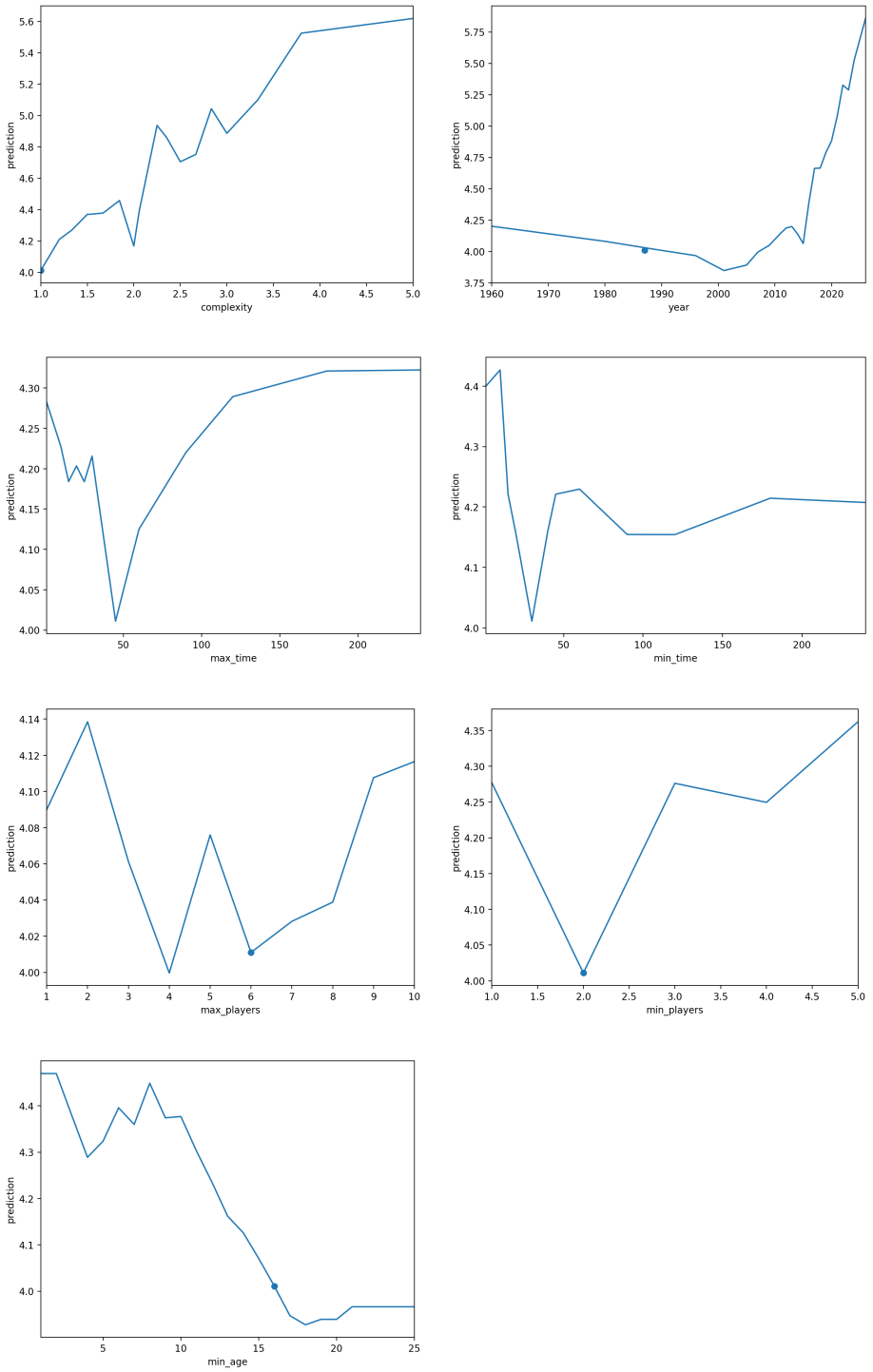


Figure 44: CP plots for How to Be a Complete Bastard for XGB model

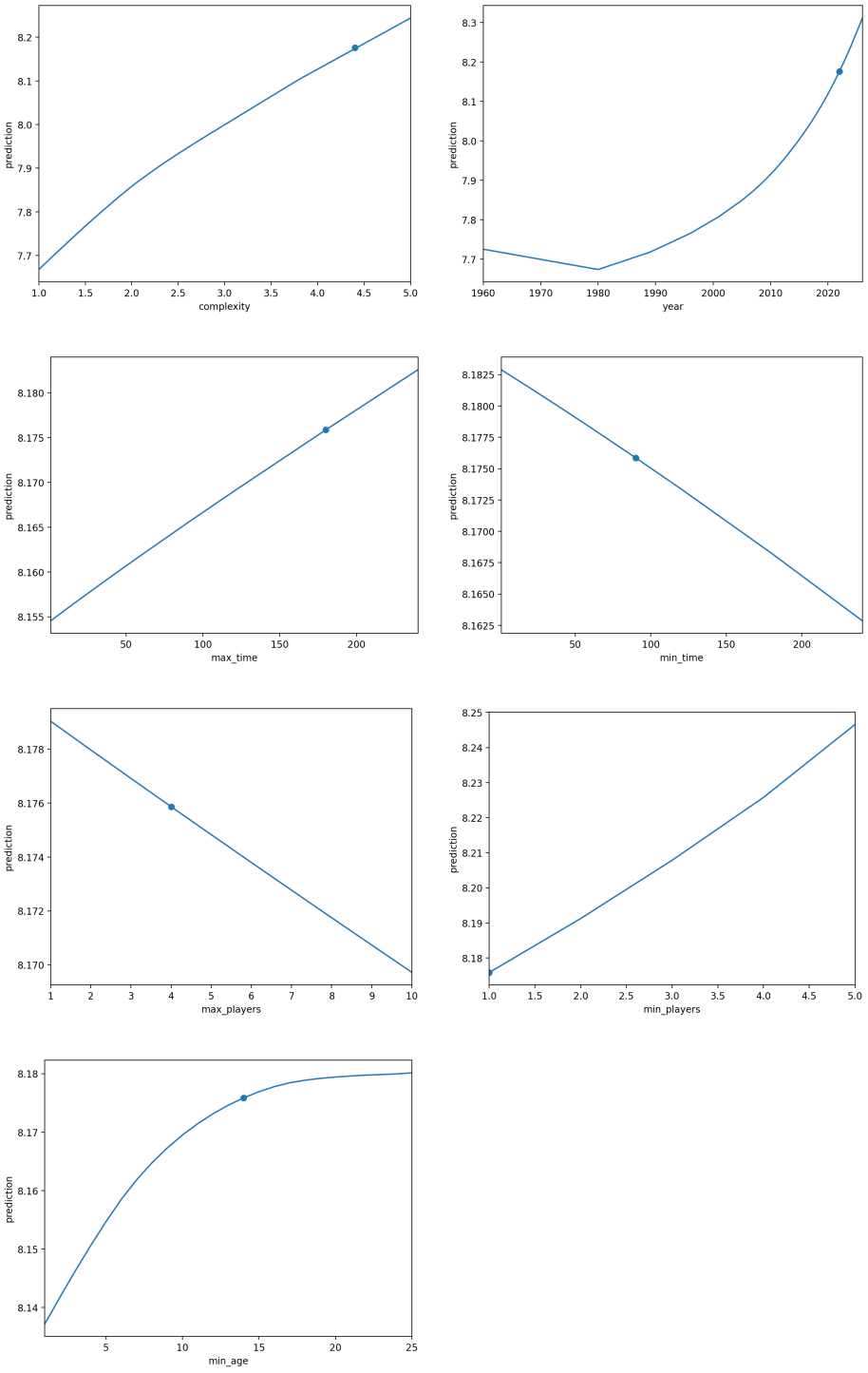


Figure 45: CP plots for Frosthaven for NN3 model

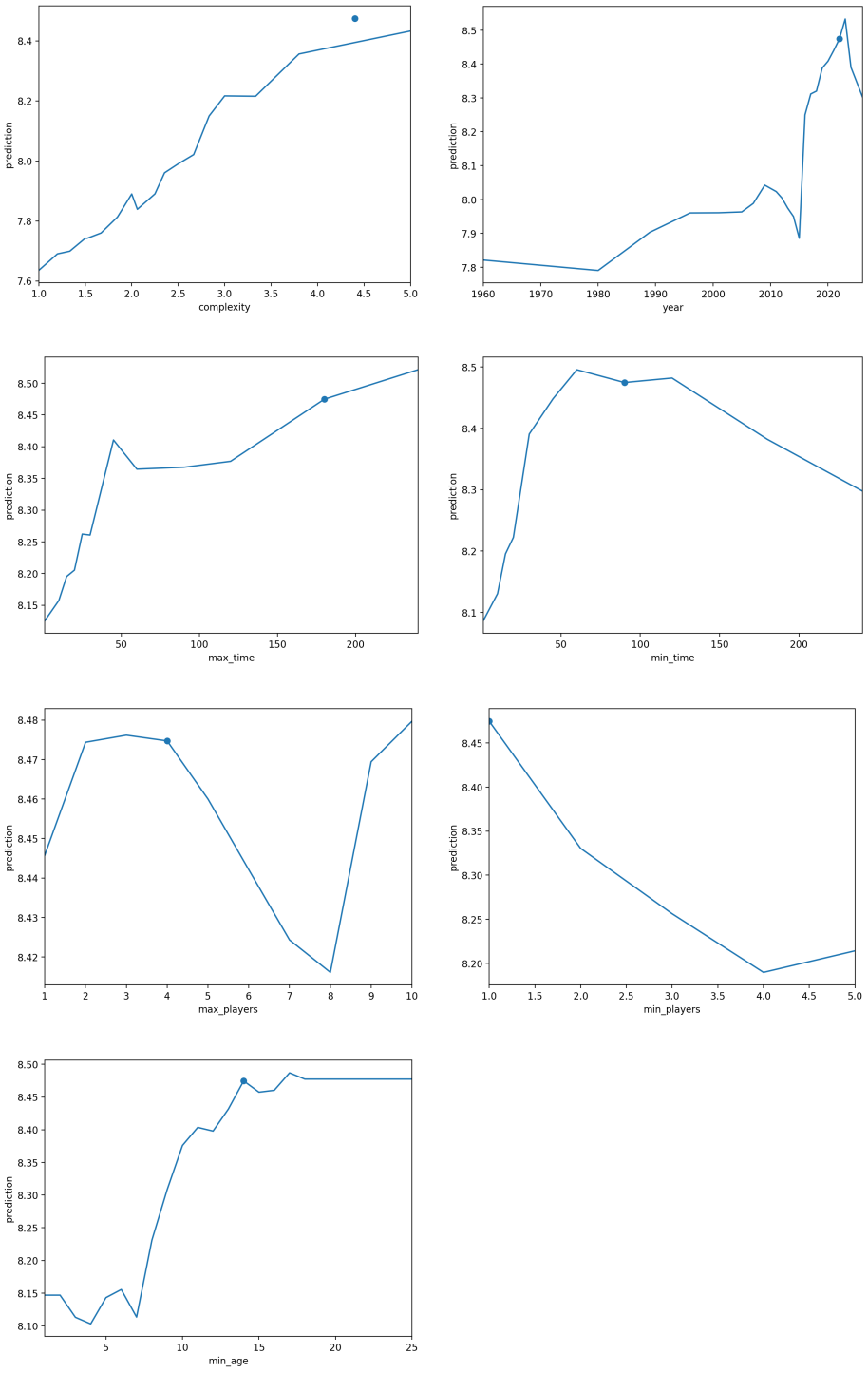


Figure 46: CP plots for Frosthaven for XGB model

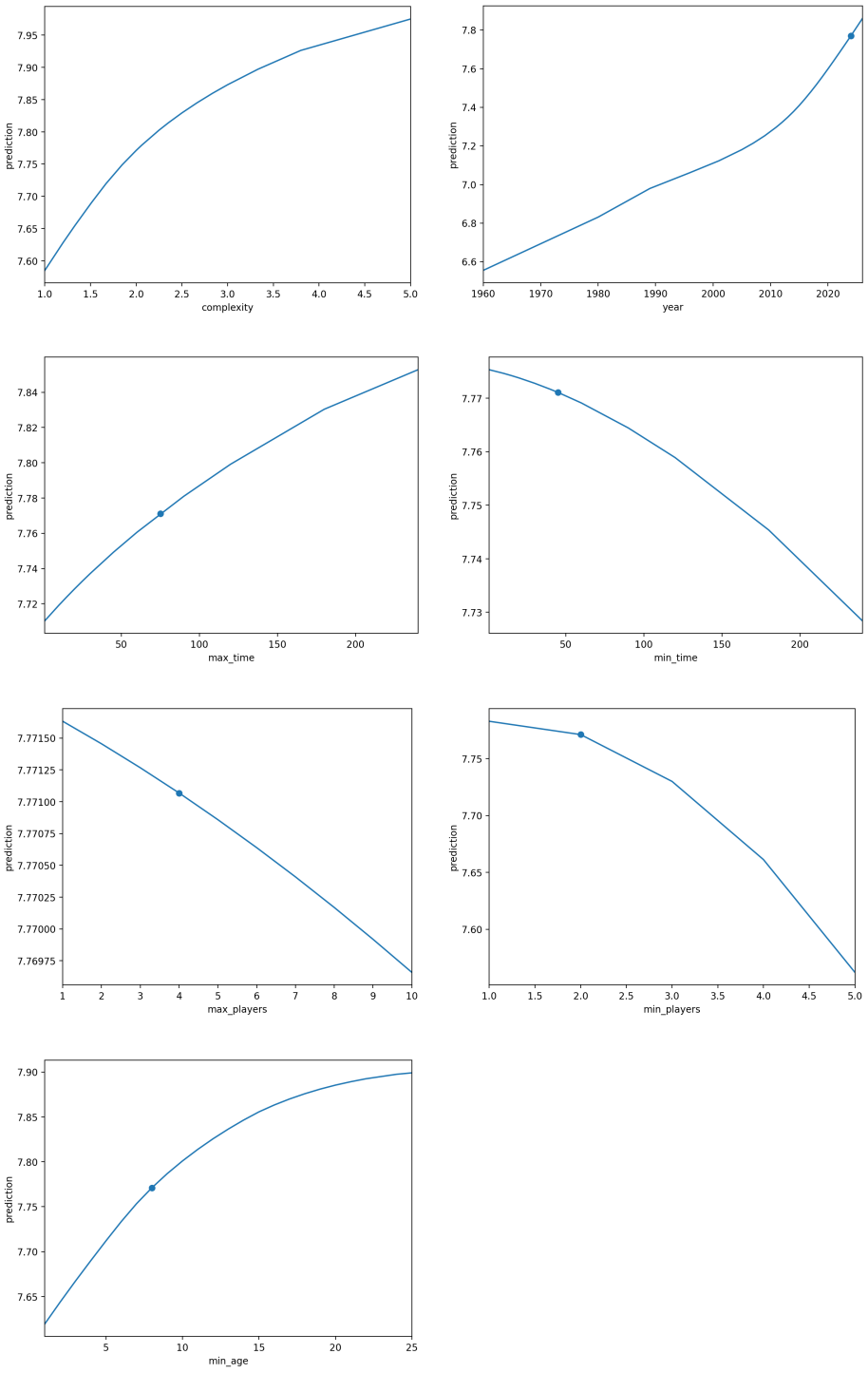


Figure 47: CP plots for Thunder Road: Vendetta – Turbo Tina's Swag Pack for NN3 model

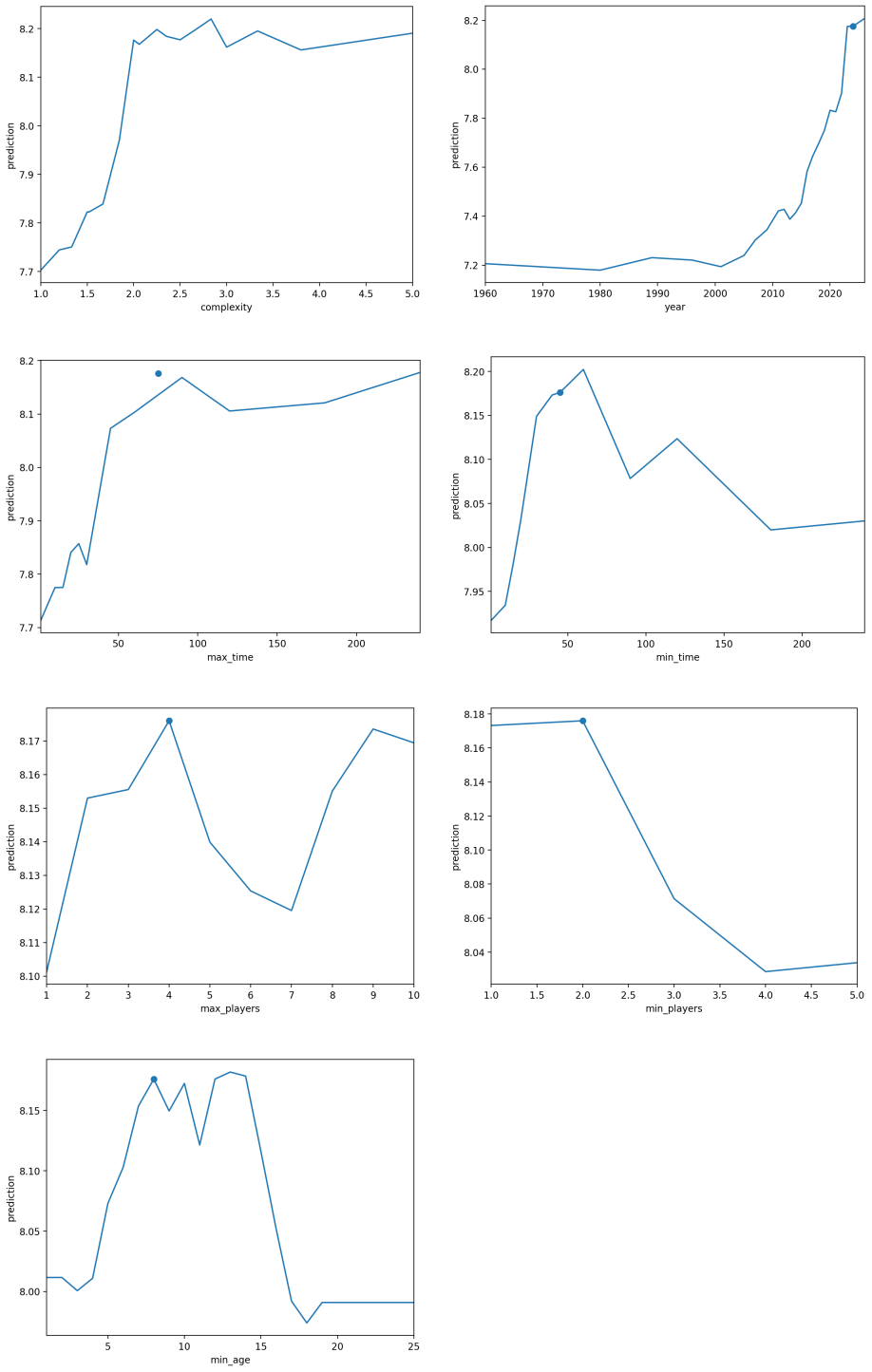


Figure 48: CP plots for Thunder Road: Vendetta – Turbo Tina's Swag Pack for XGB model

D Hinweis zur Verwendung von KI-Tools

Diese Arbeit wurde unter punktueller Nutzung von KI-gestützten Werkzeugen erstellt. Für die Literatursuche wurde die KI-basierte Suchmaschine *scite.ai* eingesetzt. Darüber hinaus wurde *ChatGPT* (Modelle GPT-4, GPT-5, GPT-5.1 und GPT-5.2) in mehreren Rollen verwendet: (i) als Formulierungshilfe zur Verbesserung der englischen Ausdrucksweise sowie zur sprachlichen Überarbeitung und Korrektur einzelner Textpassagen, (ii) zur Unterstützung bei der Strukturierung von Abschnitten, (iii) zur Programmierunterstützung einschließlich Debugging und der Erläuterung technischer Konzepte, sowie (iv) als dialogbasiertes Lern- und Verständniswerkzeug, bei dem Rückfragen gestellt wurden, bis die zugrunde liegenden mathematischen bzw. algorithmischen Zusammenhänge nachvollzogen waren. Zudem war *GitHub Copilot* mit Modell GPT-4.1 als Autovervollständigung beim Schreiben von Python- und LaTeX-Code aktiviert.

Die wissenschaftliche Eigenleistung bleibt hiervon unberührt: Inhaltliche Entscheidungen, Herleitungen, Beweise, Experimente und Interpretationen wurden eigenständig vorgenommen. Aussagen mit fachlichem Gehalt wurden nicht aus KI-Ausgaben übernommen, sondern anhand von Fachliteratur verifiziert. KI-Tools dienten ausschließlich als unterstützende Werkzeuge für Sprache, Struktur, technische Umsetzung und Verständnisarbeit.

E Selbstständigkeitserklärung

Ich versichere, dass ich die vorliegende Arbeit selbstständig und nur unter Verwendung der angegebenen Quellen und Hilfsmittel angefertigt habe, insbesondere sind wörtliche oder sinngemäße Zitate als solche gekennzeichnet. Mir ist bekannt, dass Zuwiderhandlung auch nachträglich zur Aberkennung des Abschlusses führen kann. Ich versichere, dass das elektronische Exemplar mit den gedruckten Exemplaren übereinstimmt.

Ort, Datum

Unterschrift